

Foundations, Architectures, and Emerging Frontiers of LLM-based Multi-Agent Systems: A Comprehensive Survey

Generated by AutoSurvey

Contents

1	Introduction and Foundational Concepts	3
1.1	Definitions, Scope, and Foundational Concepts of LLM-MAS	3
1.2	Evolution from Single-Agent to Multi-Agent Paradigms	5
1.3	Core Motivations and Theoretical Foundations for Multi-Agent Collaboration	7
1.4	Unified Taxonomy of Agent Roles, Coordination Architectures, and Interaction Topologies	9
2	Core Architectures, Communication Protocols, and Coordination Mechanisms	11
2.1	Architectural Taxonomy and Coordination Topologies	11
2.2	Communication Protocols and Inter-Agent Interaction Paradigms	14
2.3	Coordination Strategies, Decision-Making, and Interaction Dynamics	17
2.4	Design Choices, Heterogeneity, and System-Level Trade-offs	19
3	Memory, Learning, and Self-Evolution in Multi-Agent Systems	22
3.1	Memory Taxonomies and Hierarchical Architectures for Multi-Agent Systems	22
3.2	Experience Accumulation, Retrieval, and Procedural Skill Formation	25
3.3	Self-Evolution Frameworks and Multi-Agent Learning Mechanisms	28
3.4	Lifelong Learning, Test-Time Adaptation, and Continuous Improvement	31
4	Planning, Reasoning, and Decision-Making in Multi-Agent Collaboration	33
4.1	Advanced Planning Methodologies in Multi-Agent Collaboration	33
4.2	Reasoning Enhancement Through Debate, Consensus, and Critique Refinement	35
4.3	Game-Theoretic Decision-Making, Strategic Reasoning, and Theory of Mind	38
4.4	Emergent Coordination Behaviors, Task Complexity, and Generalization Theory	42
5	Optimization, Efficiency, and Resource Management	44
5.1	Automated Architecture Construction and Search for Cost-Effective MAS	44
5.2	Routing, Model Selection, and Budget-Aware Collaboration Strategies	47
5.3	Infrastructure-Aware Scheduling, Serving, and Resource Management	50
5.4	KV-Cache Management, Parallel Execution, and Performance-Cost Modeling	52
6	Evaluation, Benchmarking, and Failure Diagnosis	54
6.1	Evaluation Metrics, Taxonomies, and Process-Oriented Assessment Frameworks	54
6.2	Benchmarks, Scalability, and Comparative Analysis of Single-Agent vs. Multi-Agent Systems	56
6.3	Failure Localization, Attribution, and Error Propagation Diagnosis	58
6.4	Reliability Engineering, Robustness Testing, and Defense-Aware Evaluation	61

7	Security, Robustness, and Responsible AI	64
7.1	Threat Taxonomy and Attack Surfaces in LLM-MAS	64
7.2	Defense Mechanisms and Mitigation Strategies	66
7.3	Evaluation, Verification, and Red-Teaming for Security	69
7.4	Governance, Alignment, and Responsible AI Frameworks	72
	References	75

1 Introduction and Foundational Concepts

1.1 Definitions, Scope, and Foundational Concepts of LLM-MAS

The advent of large language models (LLMs) has catalyzed a paradigm shift in artificial intelligence, moving beyond the deployment of isolated models toward the construction of intricate, collaborative systems. At the forefront of this evolution are LLM-based Multi-Agent Systems (LLM-MAS), which represent a fundamental departure from both classical multi-agent systems and single-agent paradigms. Establishing a formal and precise definition of LLM-MAS is essential for delineating the scope of this survey and for providing a robust foundation upon which to analyze their architectures, mechanisms, and applications.

We define an LLM-based Multi-Agent System (LLM-MAS) as a computational framework comprising multiple interacting intelligent agents, where the core cognitive and decision-making capabilities of at least a subset of these agents are powered by one or more large language models. These agents operate within a shared or partially shared environment, perceiving inputs, reasoning about their state, and taking actions to achieve individual or collective objectives. The defining feature is the orchestration of these LLM-powered agents through structured or emergent communication and coordination protocols, enabling them to tackle tasks that exceed the capabilities of any single agent [1, 2]. This definition distinguishes LLM-MAS from simpler LLM-centric orchestration, where a single LLM sequentially invokes tools or prompts, by emphasizing the presence of multiple, distinct cognitive entities that interact dynamically.

To fully appreciate the nature of LLM-MAS, it is crucial to contrast them with both their classical predecessors and single-agent counterparts. Classical multi-agent systems (MAS) have a rich history in artificial intelligence, grounded in formal methods, game theory, and distributed control, with agents often relying on explicit symbolic reasoning, predefined rules, and low-level state exchanges [3]. While they share the foundational concept of multiple interacting entities, LLM-MAS diverge significantly by leveraging the semantic understanding and generative capabilities of LLMs. This elevates collaboration from simple state-based coordination to high-level, language-driven reasoning, enabling agents to interpret nuanced instructions, negotiate complex plans, and generate novel solutions in ways that were previously unattainable [4]. From the perspective of single-agent systems, the limitations are well-documented: a single agent is constrained by its inherent biases, its finite context window, and its inability to parallelize distinct aspects of a task, leading to bottlenecks in complex, long-horizon scenarios [2]. The motivation for LLM-MAS thus arises from a need to overcome these single-agent bottlenecks through distributed intelligence, where multiple agents can offer diverse perspectives, specialize in different sub-tasks, and engage in iterative critique to improve robustness and accuracy [5].

A deeper exploration reveals the core characteristics that define and distinguish LLM-MAS. Four foundational pillars—agent autonomy, social interaction, environmental perception, and communication-driven collaboration—are paramount. Agent autonomy refers to the capacity of an LLM-powered agent to operate without continuous external control, making its own decisions based on its internal model, accumulated memory, and perceived environment. This autonomy, however, is not absolute; it is bounded by the system’s overarching coordination architecture and the specific instructions embedded in the agent’s persona and goals [2]. Social interaction is the *sine qua non* of a multi-agent system. It encompasses the complex dynamics that arise when multiple autonomous entities coexist and influence each other, such as competition, cooperation, or negotiation [6]. This is a critical point of distinction, as many supposedly “multi-agent” systems in the LLM literature lack genuine social interaction, instead relying on a single LLM to sequentially generate the responses of different “agents,” thereby simulating a social process without enacting it. Environmental

perception refers to an agent’s ability to gather information from and act upon its surroundings. This environment can be external, such as a code repository, a web browser, or a physical simulation, or internal, such as the shared context of a conversation. A well-designed LLM-MAS provides agents with structured feedback from their environment, enabling them to observe the consequences of their actions and adjust their strategies accordingly [3]. Finally, communication-driven collaboration is the engine that integrates the other characteristics. The ability of agents to exchange messages, share insights, delegate tasks, and reach a consensus through natural language is what enables them to pool their individual intelligences into a cohesive problem-solving unit [7].

This leads to a critical examination of terminological ambiguities and the boundaries between genuine multi-agent collaboration and what can be considered “simplistic LLM-centric orchestration.” The conceptual evolution from single-agent to multi-agent paradigms, as traced in the preceding discussion, has been driven by the recognition that collective intelligence can emerge from agent interactions, cooperation, and societal structures. Yet, a significant portion of the recent literature is critiqued for appropriating the terminology of classical MAS without engaging with its foundational principles, resulting in systems that are multi-agent in name only [5]. A key indicator of this is the absence of true autonomy and social interaction. For example, a system where a central orchestration prompt instructs a single LLM to “act as a planner, then a coder, then a critic” by sequentially generating different role-based responses is essentially a single-agent system with a sophisticated prompting strategy. While it may produce compelling results, it lacks the emergent dynamics, robustness, and division of cognitive labor that characterize a true MAS, where each agent has its own persistent state, localized context, and capacity for independent initiative. The distinction lies in the locus of control and the granularity of cognition: in a genuine LLM-MAS, each agent runs its own inference process, maintains its own context and memory, and is a separate “mind” [8]. Conversely, in LLM-centric orchestration, there is only one “mind” that is context-switching. Furthermore, the social aspect of agency is often overlooked; agents are increasingly viewed not just as problem solvers but as social actors whose behavior is shaped by their perceived role within a team and the presence of other agents, a phenomenon that can give rise to emergent behaviors such as social loafing and the bystander effect, where individual effort diminishes in a group setting [9]. This highlights that the “social” dimension is not just about communication but about the psychological and sociological dynamics that emerge from interaction.

The implications of this definition are profound for the analysis of LLM-MAS. It moves the focus from the capabilities of individual models to the emergent properties of the system as a whole. It necessitates a shift in evaluation from simple outcome-based metrics to process-oriented and architecture-aware assessments that consider how collaboration patterns, information flow, and coordination strategies influence final performance [1]. This perspective is echoed in contemporary research that calls for a paradigm shift from viewing LLM-MAS as merely large numbers of aligned individuals to understanding them as unified, dynamic socio-technical systems where global agreement, uncertainty, and security are system-level properties [10]. By establishing this formal definition and clarifying its boundaries, we set the stage for a comprehensive exploration of the architectures, communication protocols, cognitive mechanisms, and evaluation frameworks that define the current state-of-the-art in this rapidly evolving field, separating substantive contributions from mere semantic repackaging. This foundational understanding directly informs the subsequent examination of how LLM-MAS have evolved from early influential frameworks into sophisticated collaborative architectures, and how their design principles continue to shape the trajectory of collective intelligence research.

1.2 Evolution from Single-Agent to Multi-Agent Paradigms

The evolution from single-agent LLM frameworks to collaborative multi-agent architectures represents one of the most significant paradigm shifts in contemporary artificial intelligence research. This transition did not occur in isolation but rather emerged from a confluence of technological advances, conceptual insights, and practical limitations that progressively revealed the boundaries of individual agent capabilities [11].

Historically, the development of AI systems followed a trajectory from specialized, rule-based programs to versatile, learning-driven autonomous systems [12]. Early AI research was characterized by an ambition to reproduce the human mind, but as the scale of that ambition became manifest, the field retreated into either studying specialized intelligent behaviors or proposing overarching architectural concepts for interfacing specialized intelligent behavior components, conceived of as agents in a kind of organization [13]. This agent-based modeling paradigm, in turn, proved to have interesting applications in understanding, simulating, and predicting the behavior of social and legal structures on an aggregate level [13].

The advent of large language models fundamentally altered this landscape. LLMs catalyzed a transformative shift in artificial intelligence, paving the way for advanced intelligent agents capable of sophisticated reasoning, robust perception, and versatile action across diverse domains [11]. These models, initially conceived as passive text generators, evolved into autonomous, goal-driven systems that integrate planning, memory, tool use, and iterative reasoning to operate in complex environments [14]. This evolution from statistical models to transformer-based systems identified capabilities that enable agentic behavior: long-range reasoning, contextual awareness, and adaptive decision-making [14]. As established in the preceding subsection, the core characteristics of LLM-MAS—agent autonomy, social interaction, environmental perception, and communication-driven collaboration—emerge directly from these enhanced capabilities, which will be further examined through the motivations that drive multi-agent adoption in the following subsection.

The progression from single-agent LLM systems to multi-agent architectures was driven by several interconnected factors that directly extend the foundational definition and characteristics established earlier. First, researchers recognized that individual agents, despite their impressive capabilities, remained limited when tasks required sustained coordination across roles, tools, and environments [2]. While LLMs demonstrated remarkable capabilities across diverse tasks, they remained fundamentally static, unable to adapt their internal parameters to novel tasks, evolving knowledge domains, or dynamic interaction contexts [15]. This static nature became a critical bottleneck as models were increasingly deployed in open-ended, interactive environments [15]. This limitation directly underscores the motivation for multi-agent architectures that can overcome the intrinsic bottlenecks of single-agent systems through distributed cognition and persistent environmental engagement—a theme elaborated in the following subsection’s examination of concrete motivations.

Second, the recognition that many real-world scenarios require cooperation among individuals to enhance efficiency and effectiveness motivated the exploration of collaborative frameworks [16]. The emergence of large language models catalyzed two distinct yet interconnected paradigms: standalone AI Agents and collaborative Agentic AI ecosystems [17]. While AI Agents were characterized as specialized, tool-enhanced systems leveraging foundation models for targeted automation within constrained environments, Agentic AI represented sophisticated multi-entity frameworks where distributed agents exhibit emergent collective intelligence through coordinated interaction protocols [17]. This distinction proved crucial for understanding the architectural and conceptual differences between single-agent and multi-agent systems, aligning with the earlier clarification that genuine LLM-MAS require truly autonomous agents engaging in social interaction rather than a single LLM simulating multiple roles through sequential prompting—a differentiation that shapes how

motivations for multi-agent adoption are framed and evaluated.

Third, the limitations of single-agent approaches in addressing task complexity and scalability demands became increasingly apparent. As applications demanded long-term adaptability and real-time interaction, single-agent frameworks struggled to balance the growing complexity of foundation models with limited resources [18]. Multi-agent systems offered a compelling alternative by enabling groups of intelligent agents to coordinate and solve complex tasks collectively at scale, transitioning from isolated models to collaboration-centric approaches [1]. This shift recognized that collective intelligence could emerge from agent interactions, cooperation, and societal structures [11]. This recognition of distributed intelligence as a solution to complexity directly anticipates the following subsection’s detailed analysis of how role specialization and heterogeneity enable multi-agent systems to address tasks that exceed single-agent capabilities.

The transition from single-agent to multi-agent paradigms also reflected a deeper conceptual evolution. Researchers began to argue that the ‘agent’ paradigm itself might be a limiting framework, distinguished between agentic systems (AI inspired by agency, often semi-autonomous), agential systems (fully autonomous, self-producing systems, currently only biological), and non-agentic systems (tools without the impression of agency) [19]. This critical re-evaluation suggested that while the ‘agentic’ framing of many AI systems was heuristically useful, it could be misleading and obscure underlying computational mechanisms, particularly in LLMs [19]. This perspective motivated exploration of system-level dynamics and frameworks grounded in complex systems thinking, reinforcing the earlier emphasis on understanding LLM-MAS as unified socio-technical systems rather than mere collections of aligned individuals—a perspective that informs the theoretical foundations for principled design discussed in the following subsection.

Early influential frameworks played a pivotal role in establishing the foundations for contemporary LLM-MAS research, transforming the conceptual motivations above into concrete architectural patterns. AgentVerse proposed a multi-agent framework that could collaboratively and dynamically adjust its composition as a greater-than-the-sum-of-its-parts system, demonstrating that effective deployment of multi-agent groups could outperform a single agent [16]. Importantly, AgentVerse also delved into the emergence of social behaviors among individual agents within a group during collaborative task accomplishment, discussing strategies to leverage positive behaviors and mitigate negative ones for improving collaborative potential [16]. This focus on emergent social dynamics highlighted the unique characteristics of multi-agent systems beyond mere performance improvements, illustrating the social interaction pillar that distinguishes genuine LLM-MAS from single-agent orchestration.

AutoGen and CAMEL represented other foundational contributions. AutoGen established a framework for orchestrating conversations between multiple agents, enabling flexible interaction patterns and structured collaboration [2]. CAMEL (Communicative Agents for “Mind” Exploration of Large Language Model Society) explored the potential of role-playing between LLM agents to tackle complex tasks through guided communication [1]. These frameworks demonstrated that multi-agent systems could effectively decompose tasks, distribute responsibilities, and achieve results that single agents could not, embodying the communication-driven collaboration central to the formal definition of LLM-MAS. The development of these early systems laid the groundwork for the subsequent explosion of research in LLM-MAS, establishing design patterns and architectural principles that continue to influence contemporary research [20]. These foundational architectures also foreshadow the coordination architectures and interaction topologies that the following subsection systematically categorizes, providing concrete instantiations of how agent roles and communication structures shape system behavior.

The evolution toward multi-agent systems also reflected a broader scientific agenda. The field recognized the need to transition from blind trial-and-error to rigorous science, advocating for

systematic frameworks to isolate intrinsic gains from increased budgets and to identify collaboration-driving factors [21]. This scientific approach acknowledged that the field still relied heavily on empirical trial-and-error and lacked a unified and principled framework necessary for systematic optimization and improvement [21]. This call for scientific rigor directly connects to the theoretical foundations—game theory, Theory of Mind, and organizational theory—that the following subsection presents as principled guidance for design, moving the field from ad hoc construction toward systematic engineering grounded in established scientific disciplines.

The progression from single-agent to multi-agent systems ultimately represents a fundamental reconceptualization of how intelligent systems can be designed, deployed, and improved. Rather than attempting to create increasingly powerful individual agents, researchers recognized that organizing LLMs into community-based structures could enhance their collective intelligence and problem-solving capabilities [22]. This paradigm shift from isolated to synergistic operational frameworks continues to shape the field, motivating research into how multiple agents can collaborate, compete, learn, and evolve together toward increasingly sophisticated forms of collective intelligence. This reconceptualization, grounded in the formal definition and distinguishing characteristics of LLM-MAS established in the preceding subsection, sets the stage for the following subsection’s systematic examination of the concrete motivations—reasoning enhancement, task complexity, robustness, and theoretical grounding—that justify the adoption of multi-agent architectures over monolithic single-agent approaches.

1.3 Core Motivations and Theoretical Foundations for Multi-Agent Collaboration

The transition from single-agent LLM systems to multi-agent architectures is not merely a technical convenience but is driven by a set of fundamental limitations inherent to the former and unique capabilities unlocked by the latter. Building on the conceptual evolution outlined above—where the shift from isolated to synergistic operational frameworks emerged from recognizing the static nature of individual agents and the need for collective problem-solving—this subsection systematically examines the concrete motivations that justify adopting LLM-based Multi-Agent Systems (LLM-MAS). The primary motivation is to overcome the intrinsic bottlenecks of individual agents. Single-agent systems, even with advanced reasoning abilities, are constrained by their limited context windows, susceptibility to cognitive biases, and inability to simultaneously represent diverse expertise. These systems often struggle with complex tasks that demand extensive interactions and substantial computational resources, as standalone agents frequently encounter limitations when handling them [23]. Moreover, there is a growing recognition that scaling a single model faces significant challenges such as diminishing returns and high costs, making the distribution of tasks among specialized agents a more promising and economically viable solution [24]. This economic and practical rationale directly extends the earlier observation that single-agent frameworks struggled to balance the growing complexity of foundation models with limited resources. The paradigm shift is further motivated by the observation that while single agents perform well in isolated tasks, they lack the higher-order cognition required for adaptive collaboration in dynamic, real-world scenarios [25]. Thus, LLM-MAS moves beyond the constraints of monolithic intelligence by enabling collaborative problem-solving, where the interaction itself is a source of capability [2].

A central motivation for multi-agent collaboration is the enhancement of reasoning quality through structured debate, consensus-building, and critique-refinement loops. These mechanisms represent a concrete instantiation of the emergent collective intelligence discussed in the previous subsection, where the interaction among agents generates capabilities that no single agent possesses. Multi-agent debate (MAD) systems allow agents to present, critique, and refine arguments, offering a potential for improved reasoning and robustness over monolithic models [26]. This process is

designed to facilitate cognitive synergy, where the collective intelligence of the system exceeds the sum of its parts, not just through knowledge sharing but through recursive reasoning and structured critique [25]. However, the effectiveness of such debate is not unconditional; research indicates that while it can be beneficial for solving complex, multi-step reasoning tasks, its advantages over strong self-agent test-time scaling methods can be limited or even counterproductive in simpler scenarios or safety-related tasks [26, 27]. This conditional effectiveness highlights the need for principled frameworks to understand when and why collective reasoning is beneficial, moving beyond the assumption that more agents invariably lead to better outcomes [28, 27]. This nuance directly motivates the scientific rigor advocated in the previous subsection—transitioning from blind trial-and-error to systematic understanding of collaboration-driving factors.

Beyond reasoning enhancement, LLM-MAS are motivated by the need to address task complexity and scalability demands. The recognition in the previous subsection that multi-agent systems offered a compelling alternative for coordinating and solving complex tasks collectively at scale is here given a more precise formulation. Complex real-world challenges often require the integration of diverse capabilities and sequential reasoning steps that are beyond the scope of a single agent. Multi-agent systems address this through task decomposition and role specialization, enabling a “divide-and-conquer” approach to complex and long-horizon tasks [29]. Theoretically, the benefit of LLM-MAS over single-agent systems increases with both task depth (reasoning length) and width (capability diversity), clarifying that collective structures are most advantageous when tasks are inherently complex and multifaceted [27]. Furthermore, the principle of heterogeneity—powering agents with different LLMs—can elevate the system’s potential to the collective intelligence of diverse models, achieving significant performance boosts without requiring structural redesign [30]. This addresses the scalability bottleneck of homogeneous scaling, which suffers from diminishing returns due to correlated outputs, whereas heterogeneous configurations contribute complementary evidence and continue to yield substantial gains [31]. This emphasis on heterogeneity and role specialization foreshadows the taxonomy of agent roles and coordination architectures presented in the following subsection, which systematically characterizes these design dimensions.

Robustness and reliability constitute another critical motivation for adopting multi-agent frameworks. By leveraging redundancy and diverse perspectives, MAS can mitigate single points of failure and reduce the risk of cascading errors—a concern that becomes increasingly salient as systems are deployed in open-ended, interactive environments, as noted in the previous subsection. The inclusion of heterogeneous agents with different strengths creates a system that is more resilient to the failure of any one component [30, 32]. However, this robustness is not guaranteed; simply adding more agents can introduce coordination overhead and new failure modes such as error propagation across agents and interaction rounds [2, 28]. Cooperative dynamics, where agents help each other at minimal cost, are not always realized, and capable models may actively withhold information despite instructions to cooperate, revealing that “more capable” models can be “less cooperative” [33]. This highlights that robustness and effective synergy are emergent properties that require deliberate design of coordination and interaction [28]. These design considerations directly connect to the interaction topologies and coordination architectures that the following subsection systematically categorizes, emphasizing that the choice of organizational structure fundamentally shapes system behavior and reliability.

Finally, the design and analysis of LLM-MAS are deeply informed by a rich tapestry of theoretical foundations that provide principled guidance for addressing the motivations and challenges discussed above. These theoretical lenses transform the motivations into actionable design principles, bridging the gap between empirical observations and systematic engineering. Game theory provides a mathematical framework for modeling the strategic interactions, payoffs, and information structures among rational agents, organizing research around the key elements of players, strategies, payoffs, and

information [6]. Concepts such as cooperative games are used to formalize credit assignment among agents [34, 35], while Fictitious Play offers a paradigm for equilibrium-seeking in decision-making scenarios with entangled stances [36]. The cognitive science concept of Theory of Mind (ToM) is crucial for enabling agents to infer others’ intentions and rationale, which is fundamental for adaptive coordination [25, 37]. The framework of the Analysis of Competing Hypotheses (ACH) introduces structured reasoning to mitigate cognitive biases in collective decision-making [38]. Furthermore, collective intelligence and organizational theory offer analogies to human team dynamics, emphasizing the role of structure, diversity, and interaction dynamics in team performance [39]—insights that directly inform the coordination architectures and interaction topologies categorized in the following subsection. Finally, computer-supported cooperative work (CSCW) provides a grounding for defining and analyzing “collaborative competence”—the capacity of agents to establish common ground and repair misalignment—which is vital for long-term coordination [40]. These theoretical underpinnings collectively inform the architecture, strategies, and evaluation of LLM-MAS, and along with the taxonomy presented in the following subsection, move the field from blind trial-and-error toward a rigorous design science [21].

1.4 Unified Taxonomy of Agent Roles, Coordination Architectures, and Interaction Topologies

A systematic comparison of diverse LLM-based Multi-Agent Systems (LLM-MAS) designs requires a structured analytical lens that can capture the fundamental design dimensions shared across otherwise heterogeneous implementations. The preceding discussion has established that the motivations for adopting LLM-MAS—enhanced reasoning, task scalability, robustness, and theoretical grounding—are compelling, but realizing these benefits hinges critically on *how* the multi-agent system is actually designed and instantiated. While existing surveys have proposed taxonomies grounded in application domains, communication mechanisms, or capability classifications, the field lacks a unified framework that simultaneously characterizes *what agents are* (their functional roles), *how the system is organized* (its coordination architecture), and *how agents interact* (their communication topology). Without such a framework, the strategic design choices that determine whether the theoretical advantages of collaboration—discussed in the previous subsection—are actually achieved remain obscured. In this subsection, we present a taxonomy spanning three orthogonal dimensions—agent roles, coordination architectures, and interaction topologies—synthesized from an analysis of foundational surveys and influential early frameworks. This taxonomy directly operationalizes the theoretical foundations and design principles highlighted earlier, transforming them into concrete analytical categories that will guide the evaluation, comparison, and construction of LLM-MAS throughout the remainder of this survey.

Agent Roles. The first dimension characterizes the functional specialization of individual agents within a system, directly addressing the motivation for role specialization and heterogeneity discussed in the previous subsection. Based on an analysis of prominent frameworks, we identify several recurring archetypal roles. *Workers* (or executors) are agents that directly perform task-specific operations, such as generating code, retrieving information, or executing tool calls. *Planners* decompose complex tasks into subtasks and design execution strategies, often operating at a higher level of abstraction than workers—this role is the concrete instantiation of the task decomposition and “divide-and-conquer” approach identified as a key advantage of LLM-MAS. *Critics* (or evaluators) review outputs produced by other agents, providing feedback for iterative refinement, thereby operationalizing the critique-refinement loops and consensus-building mechanisms discussed earlier. *Orchestrators* (or supervisors) coordinate the activities of multiple agents, managing task assignment, information routing, and execution order. *Judges* make final decisions on outputs, often selecting

among competing candidates or resolving disagreements. This functional decomposition aligns with the observation that common LLM-profiled roles include *policy models*, *evaluators*, and *dynamic models* [41]. Furthermore, recent frameworks such as PartnerMAS demonstrate the practical value of role specialization through hierarchical decomposition into planner, specialized, and supervisor layers, where each role contributes complementary capabilities to the overall decision-making process [42]. Notably, roles are not necessarily static; self-evolving systems allow agents to dynamically refine their profiles and capabilities during task execution, guided by performance feedback and team complementarity [43]. Similarly, skill-centric designs construct agents as task-conditioned bundles of reusable skills, enabling flexible role composition that generalizes to unseen task requirements [44]. This flexibility in role definition directly supports the heterogeneity principle discussed earlier, which showed that diverse agent capabilities contribute complementary evidence and yield substantial performance gains.

Coordination Architectures. The second dimension captures the system-level organizational structure that governs how agents are arranged and how authority and information flow are distributed. This dimension directly addresses the design considerations for robustness and reliability raised in the previous subsection, where effective synergy was identified as an emergent property requiring deliberate design of coordination. We distinguish five archetypal architectures. *Centralized* architectures feature a single orchestrator or controller that coordinates all agents, making decisions about task decomposition, assignment, and result aggregation. This design offers strong control and global consistency but introduces potential communication bottlenecks and a single point of failure—a concern directly linked to the robustness motivations discussed earlier. *Decentralized* architectures distribute control among agents, enabling peer-to-peer interaction without a central coordinator. While offering greater autonomy and fault tolerance, decentralized designs may suffer from coordination overhead and inconsistent global behavior [45]. *Hierarchical* architectures organize agents into layered structures with varying levels of control, information flow, role delegation, and temporal abstraction [46]. Hierarchical designs can achieve global efficiency while preserving local autonomy, though the balance is delicate and context-dependent. *Flat* architectures treat all agents as equals, often engaging them in symmetric collaboration or debate, which connects directly to the multi-agent debate mechanisms discussed previously. *Dynamic* architectures adapt their organizational structure during execution based on task requirements, feedback, or environmental changes. For instance, AgentConductor employs reinforcement learning to dynamically generate task-adapted, density-aware layered DAG topologies, adjusting topology density based on inferred task difficulty [47]. Similarly, DRAMA implements a modular architecture with separation between control and worker planes, enabling real-time task reassignment as agents join, depart, or become unavailable [48].

It is important to note that these architectural categories are not mutually exclusive; hybrid designs are common. For example, hierarchical frameworks can incorporate both centralized control at higher levels and decentralized coordination at lower levels, and dynamic architectures may transition between centralized and decentralized modes. Furthermore, the choice of architecture entails fundamental trade-offs among control, autonomy, communication overhead, fault tolerance, and scalability—trade-offs that directly determine whether the robustness and scalability benefits motivated in the previous subsection are realized in practice. Research on hierarchical multi-agent systems emphasizes that hierarchy is not a one-size-fits-all solution but rather a design choice with both benefits and costs across multiple dimensions including control hierarchy, information flow, and temporal layering [46]. Empirical studies comparing flat versus hierarchical team structures in LLM-based systems have found that flat teams tend to perform better on certain reasoning tasks, suggesting that architectural choices interact with task characteristics in complex ways [39]. This finding resonates with the conditional effectiveness of multi-agent debate discussed earlier,

reinforcing that architectural benefits are task-dependent rather than absolute.

Interaction Topologies. The third dimension characterizes the structural patterns of communication and information flow between agents. This dimension operationalizes the interaction mechanisms—such as structured debate, critique-refinement loops, and consensus-building—that were identified in the previous subsection as central motivations for adopting LLM-MAS. Common topologies include *star* (hub-and-spoke) configurations where all agents communicate through a central node, *chain* structures where information passes sequentially from one agent to the next, *mesh* (or fully connected) topologies where every agent can communicate with every other agent, *layered DAG* structures where information flows through ordered layers with directed connections, and *tree* or *hierarchical* communication structures that mirror the organizational hierarchy. The choice of topology significantly affects communication efficiency, information propagation, and error propagation patterns—factors that directly determine whether the error propagation failure modes identified in the previous subsection are amplified or mitigated. AgentConductor explicitly models topology density—the degree of connectivity—as a tunable parameter that should be adapted to task difficulty, demonstrating that excessive communication can lead to redundant information and performance bottlenecks [47]. Research on scaling LLM-driven multi-agent systems has proposed design principles including simplicity, elastic feedback, sequential workflows with optional loops, and summary-based communication, which translate into specific topological constraints modeled as constrained directed workflow graphs [49]. Moreover, the interaction topology is not independent of the coordination architecture; for example, centralized architectures often employ star topologies, while fully decentralized systems may adopt mesh structures. This interdependence between topology and architecture constitutes a critical design consideration that must be explicitly evaluated when analyzing system behavior.

The three dimensions of our taxonomy are orthogonal in the sense that different combinations yield architecturally distinct systems with different failure modes and design trade-offs. For instance, the same orchestrator-worker topology can implement fundamentally different patterns—plan-and-execute, hierarchical delegation, or adversarial verification—depending on the cognitive functions assigned to each role [50]. Existing frameworks often conflate these dimensions, leading to ambiguous characterizations of systems that are architecturally different despite superficial similarities. Our unified taxonomy provides a structured lens for systematically comparing diverse MAS designs, enabling researchers to identify precisely where systems differ and what design choices drive observed behavioral differences. This analytical clarity directly supports the scientific rigor advocated in the previous subsection—moving the field from blind trial-and-error toward a rigorous design science—by establishing a common vocabulary and analytical framework for characterizing, comparing, and ultimately predicting the behavior of LLM-MAS systems. This analytical clarity is essential for advancing the field from ad-hoc system building toward principled, evidence-based design methodologies, and it provides the foundational framework upon which the subsequent discussion of system capabilities, evaluation methodologies, and future directions will build.

2 Core Architectures, Communication Protocols, and Coordination Mechanisms

2.1 Architectural Taxonomy and Coordination Topologies

The architectural design of LLM-based Multi-Agent Systems (LLM-MAS) is a critical determinant of their performance, scalability, and robustness. A systematic categorization of these architectures is essential for understanding the vast and rapidly expanding design space. This subsection presents a

taxonomy that distinguishes centralized orchestrator-led designs, decentralized peer-to-peer systems, hierarchical structures, flat configurations, and dynamic graph-based topologies. By analyzing the design trade-offs inherent in each category, we aim to provide a structured lens through which to compare and evaluate the diverse architectural choices available to system designers. This architectural perspective is intrinsically linked to the communication mechanisms discussed in the next subsection: the chosen topology dictates the pathways, protocols, and information flow that enable agents to coordinate, while the available communication paradigm in turn constrains what architectural configurations are feasible or efficient.

Centralized Architectures represent the most prevalent paradigm in current LLM-MAS frameworks. In these systems, a central orchestrator or coordinator agent is responsible for task decomposition, role assignment, and the management of inter-agent communication. This design centralizes control, simplifying coordination and enabling global optimization of the workflow. However, it also introduces potential bottlenecks; the orchestrator becomes a single point of failure and a communication bottleneck as the system scales. For instance, the Hierarchical Adaptive Consensus Network addresses scalability in consensus tasks by using a global orchestration framework at its third layer, demonstrating the utility of central arbitration for final decision-making [51]. Similarly, frameworks like HieraMAS, while introducing intra-node LLM mixtures, still rely on a high-level orchestration mechanism to manage the inter-node topology, highlighting the continued relevance of central coordination even in more complex designs [52]. The appeal of centralized designs lies in their ability to maintain a coherent global strategy, as seen in the AgentConductor framework, where an LLM-based orchestrator generates task-adapted, density-aware layered DAG topologies, enabling dynamic control over the collaboration process [47]. In such systems, communication is typically mediated through the orchestrator, which routes messages and consolidates information, a pattern that aligns with the natural language message passing paradigm described in the following subsection.

In contrast, **Decentralized Architectures** distribute control and decision-making across all agents, eliminating the need for a central coordinator. This peer-to-peer paradigm enhances robustness by removing the single point of failure and improves scalability by reducing communication bottlenecks. The Symphony framework exemplifies this approach, enabling lightweight LLMs on consumer-grade GPUs to coordinate through a decentralized ledger and a Beacon-selection protocol for dynamic task allocation, thereby achieving a privacy-saving, scalable, and fault-tolerant orchestration [53]. Similarly, the Matrix framework for synthetic data generation employs a decentralized design where both control and data flow are represented as serialized messages passed through distributed queues, eliminating the central orchestrator to allow for massive concurrent workflows [54]. This shift towards decentralization is also echoed in the AgentNet framework, which uses a decentralized, RAG-based approach to enable agents to evolve and collaborate autonomously within a dynamically structured DAG, thereby enhancing robustness and emergent intelligence while preserving privacy across organizations [55]. The primary challenge for decentralized systems is achieving coherent global behavior without centralized oversight, often relying on local interactions and market-driven or consensus-based mechanisms to ensure alignment, which in turn requires richer peer-to-peer communication protocols of the kind discussed in the next subsection.

Hierarchical Architectures organize agents into multiple layers or levels, with higher-level agents managing and coordinating the activities of lower-level agents. This structure is designed to manage complexity and scale by decomposing tasks and delegating responsibilities. Hierarchical systems can be characterized along multiple dimensions, including control hierarchy, information flow, role delegation, temporal layering, and communication structure, as formalized in a multi-dimensional taxonomy for Hierarchical Multi-Agent Systems [46]. The Hierarchical Adaptive Consensus Network provides another example, using a three-tier architecture to first collect confidence-based voting

from local clusters, then facilitate inter-cluster communication, and finally provide system-wide coordination, achieving linear communication complexity rather than quadratic [51]. The CTHA framework argues for a constrained temporal hierarchical architecture that projects inter-layer communication onto structured manifolds to restore coordination stability, using message-contract constraints and authority manifolds to prevent error propagation and ensure coherent decision-making [56]. Furthermore, hierarchical task abstraction mechanisms, such as those used in EarthAgent, align the multi-agent architecture with the intrinsic task-dependency graph of a domain, enforcing procedural correctness and decomposing complex problems into sequential layers of specialized agents [57]. The trade-off in hierarchical systems lies between the global efficiency gained through structured delegation and the potential loss of local autonomy and the risk of error propagation between layers, a challenge that CTHA directly addresses by bounding decision spaces [56]. Notably, hierarchical structures often require well-defined interfaces between layers, motivating the adoption of structured communication protocols that can enforce message contracts and semantic alignment.

Flat Architectures consist of homogenous agents that interact directly with each other without a designated leader. These systems rely on emergent coordination and collective intelligence to solve tasks. While they can be simpler to design and more robust to individual failures, they often suffer from significant coordination overhead and may lack the strategic coherence of more structured systems. Research on the scaling behavior of such systems shows that performance does not scale monotonically with agent count but follows a pattern of diminishing returns, governed by a trade-off between collaborative synergy and coordination overhead [28]. This finding suggests that simply adding more agents to a flat system is not a guaranteed path to improved performance and that strategic interaction design is critical [28]. Furthermore, an information bottleneck perspective reveals that flat systems with bounded relay messages may lose task-relevant information, providing a theoretical limit to their advantage over single-agent systems [58]. To mitigate some of these issues, researchers have explored structural priors like small-world networks, which balance local clustering and long-range integration to stabilize consensus trajectories and improve robustness in flat, fully-connected debate systems [59]. These findings highlight the tight coupling between the architectural topology and the communication medium, as the information loss in flat systems is often a direct consequence of textual message discretization, a limitation that motivates the latent communication paradigms explored in the following subsection.

Finally, **Dynamic Graph-based Topologies** represent an emerging class of architectures where the communication structure is not fixed but can adapt over time based on the task or the state of the system. This flexibility allows for more efficient resource allocation and improved fault tolerance. For example, GoAgent explicitly treats collaborative groups as atomic units, enumerating task-relevant candidate groups and autoregressively connecting them to construct the final communication graph, thereby capturing intra-group cohesion and inter-group coordination [60]. The agent architecture dynamically adjusts its topology, compressing inter-group communication to reduce redundancy [60]. Similarly, the AgentConductor framework generates a dynamic, density-aware layered DAG topology for each query, adapting the structure to task difficulty and demonstrating significant token cost reduction [47]. The Self-Evolving Concierge System also utilizes a dynamic approach, “hiring” specialized sub-agents based on real-time conversation analysis and employing an LRU eviction policy to manage resources [61]. These dynamic systems aim to overcome the limitations of static topologies by offering a better balance between control and adaptability, though they introduce new challenges in terms of stability and the overhead of deciding when and how to restructure the network. The adaptive nature of these architectures often goes hand-in-hand with the semantic and temporal communication structures discussed next, as agents must continuously negotiate and align on the evolving communication graph.

In summary, the choice of architectural topology is a fundamental design decision for LLM-MAS,

involving a complex interplay between control, autonomy, efficiency, and robustness. Centralized architectures offer simplicity and global control but face scalability bottlenecks. Decentralized systems provide robustness and scalability but struggle with coherent coordination. Hierarchical structures manage complexity through delegation but introduce new fragility points. Flat systems promote emergent behavior but are prone to overhead. Dynamic topologies adapt to the task but require sophisticated mechanisms to decide on restructuring. As the field evolves, future architectures are likely to be hybrid, combining elements of these categories to strike a context-dependent optimal balance, driven by findings on design principles for scalable architectures [49] and the growing recognition that orchestration topology is a first-class optimization target [62]. Crucially, the realization of these hybrid architectures will depend on the availability of flexible communication infrastructure—spanning natural language, structured protocols, and latent-space channels—that can support the diverse interaction patterns they entail, a topic to which we now turn.

2.2 Communication Protocols and Inter-Agent Interaction Paradigms

Communication is the lifeblood of LLM-based Multi-Agent Systems (LLM-MAS), determining not only what information is exchanged but also the efficiency, interpretability, and security of the entire collaborative process. [7] This subsection examines the spectrum of inter-agent communication mechanisms, from the ubiquitous natural language message passing to structured protocols and emerging latent-space paradigms, and analyzes the architectural and semantic shifts that are reshaping how agents interact.

The architectural taxonomy presented in the previous subsection establishes the structural skeletons of LLM-MAS—whether centralized, decentralized, hierarchical, flat, or dynamically reconfigurable—but these skeletons are animated by communication. Indeed, the topology dictates the pathways along which information must flow, and the choice of communication medium in turn constrains which architectural configurations are feasible or efficient. As we shall see, centralized orchestrators typically rely on mediated natural language routing, decentralized systems demand richer peer-to-peer protocols, and dynamic graph-based topologies often benefit from more compact or structured communication to enable rapid restructuring. At the same time, the coordination strategies examined in the following subsection—voting, negotiation, theory-of-mind reasoning, and adaptive task routing—presuppose specific communication capabilities; the richness of the communication medium directly determines what coordination mechanisms are expressible. This subsection thus serves as the critical bridge between *who* agents are connected to and *how* they jointly decide.

At the most fundamental level, agents can communicate through natural language, where messages are serialized into discrete tokens that convey reasoning, plans, and observations. This protocol is simple and interpretable, facilitating human oversight. However, it suffers from three structural drawbacks: high inference cost due to token generation, irreversible information loss during discretization, and the inherent ambiguity and redundancy of human language. [63] Furthermore, the semantic space of natural language is structurally misaligned with the high-dimensional vector spaces in which LLMs operate, leading to behavioral drift and limiting the depth of information that can be transmitted. [8] This has motivated the development of more structured and efficient communication paradigms.

To address the limitations of free-form natural language, a new generation of structured protocols has emerged to standardize how agents discover, interact, and coordinate. These protocols are the foundational infrastructure for scalable and interoperable agent ecosystems. The Model Context Protocol (MCP) provides a JSON-RPC client-server interface for secure tool invocation and typed data exchange, standardizing how agents access external tools and contextual data. [64] For inter-

agent coordination, the Agent-to-Agent (A2A) protocol enables peer-to-peer task delegation using capability-based Agent Cards, supporting secure collaboration across enterprise workflows. [64] The Agent Communication Protocol (ACP) defines a general-purpose communication protocol over RESTful HTTP, supporting MIME-typed multipart messages and both synchronous and asynchronous interactions, while integrating with role-based and decentralized identifiers. [64] For open network discovery, the Agent Network Protocol (ANP) supports secure collaboration using W3C decentralized identifiers and JSON-LD graphs, enabling a decentralized agent marketplace. [64, 65] Beyond these, the Tool-Environment-Agent (TEA) protocol models agents, tools, and environments as first-class, versioned resources with explicit lifecycles, enabling end-to-end context and version management for traceability and self-evolution. [66] In contrast, the LLM-Agent Communication Protocol (LACP) draws inspiration from telecommunications, proposing a three-layer architecture designed to ensure semantic clarity, transactional integrity, and built-in security for NextG networks. [67] A comparative analysis of these protocols reveals key differences in interaction modes, discovery mechanisms, and security models; while MCP has seen massive adoption as an industry standard, newer protocols like A2A and ANP are pushing toward richer peer-to-peer coordination and decentralized discovery. [64, 68] Interestingly, despite their maturity, most of these protocols lack protocol-level mechanisms for clarification, context alignment, and verification, pushing semantic responsibilities into application-specific orchestration logic and creating hidden interoperability and maintenance costs. [69] This gap is particularly consequential for the architectural paradigms discussed earlier: hierarchical systems, for instance, often require message-contract constraints between layers to prevent error propagation [56], yet current protocols leave such contract enforcement to application-level logic rather than providing it natively. These structured protocols, with their emphasis on typed data and standardized discovery, provide the substrate upon which the negotiation and deliberation strategies of the following subsection can be reliably built.

As an alternative to text-based protocols, a growing body of work explores the paradigm of latent communication, where agents exchange continuous representations such as embeddings, hidden states, or KV-caches, bypassing the bottleneck of text generation. [63] This shift is motivated by the desire to preserve richer task-relevant information and reduce inference cost. However, the design space for such systems is vast. To organize this emerging literature, a unified 3-axis framework has been proposed, analyzing methods along three dimensions: (1) *WHAT* information is communicated (embeddings, hidden states, KV-caches), (2) *WHICH* sender-receiver alignment is used (latent-space and layer alignment), and (3) *HOW* the communicated information is fused into the receiver (concatenation, prepending, cross-attention, or cache restoration). [63] For instance, the Interlat framework leverages the continuous last hidden states of an LLM as a representation of its thought for direct communication, achieving superior performance and up to 24x faster inference via compression. [70] Similarly, KVComm enables efficient communication through selective sharing of KV pairs, using an attention-based selection strategy to identify the most informative layers. [71] This paradigm is further enriched by weight-space communication, where a sender’s hidden states are compiled into transient, receiver-specific LoRA perturbations, effectively using weight updates as an executable communication medium. [72] This move toward continuous channels, however, introduces new security concerns. Latent states can carry attack-associated information that remains effective during clean executions, and KV-caches, which encode contextual inputs and intermediate reasoning states, can become an opaque channel for sensitive content leakage. [73, 74] The relevance of latent communication to the architectural discussion is twofold: first, it alleviates the communication bottleneck that plagues centralized designs by dramatically reducing token overhead; second, it enables the peer-to-peer information sharing that decentralized and flat architectures require, allowing agents to exchange richer information without the coordination cost

of natural language. Moreover, the information bottleneck perspective introduced in the previous subsection—which showed that flat systems with bounded relay messages may lose task-relevant information [58]—finds a direct remedy in latent communication, where continuous representations preserve more task-relevant content than discretized text.

Beyond the medium of communication, the *temporal* and *semantic* structure of interaction plays a crucial role. Phase-Scheduled Multi-Agent Systems (PSMAS) reconceptualize agent activation as a continuous control over a shared attention space, assigning each agent a fixed angular phase and activating agents only within a rotating window, leading to substantial token reduction while maintaining task performance. [75] This highlights that effective coordination is not just about *what* agents say, but *when* they are activated. For hierarchical and centralized architectures, such temporal scheduling provides a mechanism to manage the orchestrator bottleneck by staggering communication loads; for flat and decentralized systems, it introduces a lightweight synchronization primitive that reduces coordination overhead without requiring a central coordinator—a theme that directly connects to the coordination overhead concerns raised in the previous subsection on flat architectures [28]. Furthermore, the field is shifting from simple message passing toward shared semantic understanding. The layered “Internet of Agents” (IoA) architecture proposes new layers—an Agent Communication Layer (L8) and an Agent Semantic Layer (L9)—to formalize communication structure and provide semantic grounding, validation, and coordination primitives for consensus on shared states and collective goals. [76] This vision is complemented by thought communication, a paradigm formalized as a latent variable model that enables agents to interact directly mind-to-mind by extracting and sharing latent thoughts with theoretical guarantees on their identifiability. [77] Ultimately, the goal is to move beyond token-level exchange toward a protocolized action-state communication, where raw agent outputs are projected into compact, action-centered records, improving the performance-cost trade-off in multi-agent systems. [78] This semantic grounding is precisely what the coordination strategies of the following subsection require: structured deliberation, consensus protocols, and negotiation all depend on agents sharing a common semantic frame, whether that frame is established through explicit protocols, latent alignment, or shared temporal schedules.

In summary, communication in LLM-MAS is not a monolithic channel but a multi-dimensional design space spanning natural language, structured protocols, and latent representations, each with distinct trade-offs in cost, fidelity, and interpretability. The choice of communication medium is deeply intertwined with the architectural topology of the system: centralized designs leverage mediated message passing, decentralized systems require robust peer-to-peer protocols, hierarchical structures benefit from contract-enforcing interfaces, and dynamic topologies gain flexibility from compact and rapidly reconfigurable communication. Furthermore, the temporal and semantic structuring of communication—when agents speak and what shared meaning they establish—constitutes an additional layer of design freedom that determines whether the coordination strategies we turn to next can be realized effectively. The coordination mechanisms that follow—from voting and consensus to negotiation and theory-of-mind reasoning—are not agnostic to the communication substrate; rather, their feasibility and efficiency are directly conditioned by the medium’s richness, cost, and semantic alignment. As the field progresses, the convergence of structured protocols, latent channels, and semantic grounding will likely define the next generation of communication infrastructure for LLM-MAS, enabling the hybrid architectures anticipated at the end of the previous subsection and the sophisticated coordination strategies examined in the following one.

2.3 Coordination Strategies, Decision-Making, and Interaction Dynamics

Coordination strategies in LLM-based multi-agent systems (LLM-MAS) define the mechanisms through which agents align their actions, share information, and make joint decisions. These strategies span a spectrum from purely cooperative dynamics, where agents work toward shared goals, to competitive and adversarial interactions, where agents pursue individual objectives that may conflict. The choice and design of coordination mechanisms fundamentally determine whether multi-agent collaboration yields performance gains, incurs prohibitive communication costs, or collapses into dysfunctional behaviors—making this a central design consideration for LLM-MAS. While the previous subsection examined the *medium* of communication—from natural language to latent spaces—and the following subsection will address system-level design choices, this subsection focuses on the *decision-making and interaction protocols* that govern how agents use these communication channels to coordinate their behavior. The temporal and semantic structure of interaction, introduced at the end of the previous subsection (e.g., phase scheduling and thought communication), naturally leads to the question of how agents make joint decisions given the information they exchange, which is the core concern here.

A foundational distinction in coordination research is between cooperative, competitive, and hybrid (coopetitive) interaction paradigms. Pure cooperation presumes agents share aligned objectives and coordinate to maximize collective utility, which is the dominant assumption in frameworks designed for collaborative problem-solving. However, emergent behaviors in LLM-MAS frequently deviate from this assumption. Empirical studies of social dilemmas, such as prisoner’s dilemma and public goods games, reveal that LLMs with stronger reasoning capabilities often behave less cooperatively, with recent models consistently defecting in single-shot settings [79]. This finding underscores that cooperation cannot be taken for granted; it must be actively engineered and incentivized. Even in environments where cooperation is costless and explicitly instructed, capable models may actively withhold information despite gaining nothing from doing so, a phenomenon that points to cooperation failures distinct from competence failures [33]. Conversely, research on super-additive cooperation demonstrates that combining repeated interactions with inter-group rivalry can substantially boost cooperation levels, including in one-shot interactions, suggesting that environmental structures and group dynamics can activate cooperative tendencies in LLM agents [80]. Competitive and adversarial dynamics, while less common in collaborative task-solving, play important roles in scenarios involving negotiation, debate, and strategic interaction. The concept of coopetition formalizes the simultaneous pursuit of cooperation and competition, where agents cooperate to create value but compete to capture it, providing a richer model for multi-stakeholder systems [81]. These interaction paradigms, in turn, shape—and are shaped by—the communication protocols discussed in the previous subsection, as the choice between cooperative and competitive dynamics influences whether agents use transparent message passing or more guarded, strategic communication.

Building on these interaction paradigms, decision protocols constitute a critical component of coordination, specifying how multiple agents aggregate their outputs into a final solution. Voting, consensus, and judge mechanisms are the three dominant protocol families, each with distinct strengths and failure modes. Systematic evaluations across knowledge-intensive and logic-based tasks reveal that consensus protocols excel in knowledge-intensive domains, while voting and judge protocols are more effective for logic-based tasks, with response diversity during independent solution generation improving decision quality [24]. Voting protocols also serve as coordination mechanisms in role-constrained settings, where different voting rules—simple, ranked, cumulative, and approval—produce measurably different coordination behaviors among specialized agents [82]. However, consensus-based approaches have fundamental limitations. Research argues

that consensus is strategically insufficient for value-laden tasks, where disagreement may reflect genuine normative uncertainty rather than agent error, proposing reasoning-trace disagreement as a knowledge-representation signal that distinguishes convergent agreement, divergent agreement, convergent disagreement, and divergent disagreement to support defeasible strategic routing [83]. The consensus-diversity tradeoff further complicates protocol design: retaining partial diversity through implicit consensus mechanisms—where agents exchange information yet independently form decisions—can improve exploration and robustness in dynamic environments compared to explicit coordination methods that risk premature homogenization [84]. This finding challenges the prevailing assumption that consensus is always the desired outcome in multi-agent deliberation, and it resonates with the coordination overhead concerns that the following subsection will address in the context of design trade-offs.

Negotiation and debate strategies represent more sophisticated coordination mechanisms than simple voting or aggregation, requiring richer communication exchanges that build on the structured and semantic protocols introduced earlier. Deliberative collaboration, formalized as cooperative joint decision-making under partial and asymmetric observations, requires agents to exchange information through structured deliberation to reach joint decisions, yet complex deliberative tasks continue to challenge state-of-the-art LLMs, which may fail either in the deliberation process for aligning information or in the complex reasoning process for decision-making [85]. Beyond unstructured debate, structured collective reasoning frameworks introduce typed epistemic acts, shared workspaces, and convergent flow algorithms that guarantee termination with structured decision packets containing residual objections and minority reports, significantly improving performance on non-routine tasks while consuming substantially more tokens [86]. Two-phase negotiation protocols, where agents first propose candidate roles with reasoning and then commit to joint allocations under consensus constraints, enable decentralized symbolic planning without requiring trajectory-level alignment, demonstrating that raising the abstraction level of coordination can mitigate brittleness in cooperative multi-agent planning [87]. These structured approaches to negotiation and debate directly complement the communication infrastructure discussed previously, showing how the choice of communication medium influences—and is influenced by—the sophistication of the coordination protocol.

Theory-of-mind and game-theoretic approaches provide complementary frameworks for modeling strategic reasoning in coordination. Evaluation frameworks grounded in behavioral game theory, rather than Nash equilibrium approximation alone, disentangle reasoning capability from contextual effects and reveal that model scale does not deterministically predict strategic performance, with Chain-of-Thought prompting being only selectively effective [88]. LLM-based experiments in repeated security dilemmas reproduce canonical mechanisms from international relations theory—including the effects of multipolarity on conflict likelihood, finite time horizons on backward-induction unraveling, and communication on reducing conflict through signaling and reciprocity—while also providing access to private reasoning and messages that link choices to underlying strategic logics [89]. Dynamic strategy adaptation frameworks that integrate game-theoretic principles such as belief consistency and Nash equilibrium with real-time feedback mechanisms enable LLM agents to adjust policies based on immediate contextual interactions, achieving substantial improvements over reinforcement learning baselines in high-noise environments [90]. These game-theoretic perspectives are particularly relevant for understanding the competitive and adversarial interaction paradigms mentioned earlier, and they provide a bridge to the resource-constrained design considerations that will be examined in the following subsection.

A crucial dimension of coordination is the effect of lower-layer factors—particularly computational and communication resources—which has been largely overlooked in favor of high-level institutional design, yet directly connects to the communication cost considerations raised in the previous

subsection. The FLCOA framework (Five Layers for Cooperation/Coordination among Autonomous Agents) conceptualizes how cooperation emerges across multiple layers and reveals that communication delay has a non-monotonic effect: as delay increases, agents begin exploiting slower responses, yet excessive delay reduces exploitation cycles, yielding a U-shaped relationship between delay magnitude and mutual cooperation [91]. Communication efficiency can be dramatically improved through lightweight digital twin coordination, where agents report structured action decisions and temporal constraints rather than engaging in multi-round natural-language conversations, achieving comparable task success rates while reducing communication overhead by more than 70x [92]. Exploration, in the form of engineered randomness, also plays a functional role in sustaining coordination: moderate exploration undermines the stability of defection and catalyzes self-organizing cooperative alliances in communication-based multi-agent systems [93]. These findings on resource sensitivity and communication overhead directly inform the cost-performance trade-off analysis that the following subsection will develop more systematically.

Adaptive task routing and feedback loops constitute the final class of coordination mechanisms, providing the dynamic adaptability that static protocols lack. Language Agent Teams for Task Evolution (LATTE) coordinates LLM teams through a shared, evolving coordination graph that encodes sub-task dependencies, agent assignments, and progress states, dynamically allocating work and adapting coordination while reducing token usage, wall-clock time, communication, and coordination failures relative to fixed-role designs [94]. Cooperative verification signals can guide adaptive “coopetition” mechanisms, where agents use UCB-based strategies to determine whether to collaborate or compete at each round based on coarse verifier feedback, achieving significant performance gains on reasoning benchmarks without relying on high-performance verifiers [95]. At the modeling level, coordination can be understood as a closed-loop interaction between agents, incentive fields, and persistent environmental memory, with dynamical analysis revealing critical coupling thresholds beyond which coordination breakdown occurs, providing computable early warning signatures in observable time series [96]. Coalition formation, as a higher-order coordination strategy, grounds LLM agent cooperation in hedonic game theory with formal stability guarantees, demonstrating that LLM agents exhibit bounded rationality characterized by epsilon-rational preferences, with specific protocols significantly improving Nash stability achievement [97]. Self-evolving coordination protocols, which permit limited, externally validated self-modification while preserving formal invariants, further extend the design space by treating coordination logic as a governance layer rather than an optimization heuristic [98]. Together, these mechanisms—ranging from voting and debate to game-theoretic reasoning and adaptive routing—collectively shape the coordination landscape of LLM-MAS, with the choice among them governed by task characteristics, resource constraints, and the degree of strategic alignment among agents. These coordination choices, in turn, feed directly into the design decisions and system-level trade-offs that will be examined in the following subsection, where the interplay between coordination complexity, cost, and performance becomes the central focus.

2.4 Design Choices, Heterogeneity, and System-Level Trade-offs

The coordination strategies discussed in the previous subsection—from voting and debate to adaptive routing—do not operate in a vacuum; they are instantiated within concrete system architectures that require a series of design decisions. These decisions, spanning context management, task allocation, and agent heterogeneity, significantly influence not only task performance but also operational costs, latency, and scalability. This subsection examines the practical design choices that system architects must navigate, while also scrutinizing the systemic trade-offs between performance, cost, and coordination overhead—the very overhead that the coordination mechanisms examined

previously must justify. These choices often determine whether the system achieves emergent collective intelligence or suffers from diminishing returns and inefficiency, and they directly shape the system-level trade-offs that the remainder of this survey will explore.

A fundamental design decision concerns the management of context, which can be broadly categorized into shared versus separate (or local) paradigms. Research on the “Modeling Response Consistency in Multi-Agent LLM Systems” [99] introduces a probabilistic framework to analyze the impact of these configurations, highlighting that shared contexts promote consistency but may introduce noise and scalability bottlenecks, while separate contexts offer isolation but risk inter-agent misalignment. This trade-off is further explored by “Decentralized Multi-Agent Systems with Shared Context” [100], which proposes a decentralized architecture where agents operate with a shared, verified context and a task queue. This design mitigates the central controller bottleneck but leverages a common substrate for asynchronous coordination, demonstrating that shared context does not inherently require centralized orchestration. The choice between these paradigms is often a trade-off between response consistency, which favors shared memory, and system scalability, which benefits from decentralized, separate contexts, as shown by the RCI metric in “Modeling Response Consistency in Multi-Agent LLM Systems” [99]. Notably, this design axis tightly couples with the communication medium and latency considerations raised in the previous subsections—shared contexts effectively serve as a high-bandwidth, low-latency communication channel, while separate contexts push coordination costs onto the explicit protocols discussed earlier.

Another critical axis of design is the degree of heterogeneity among agents, which spans both agent roles and the underlying LLM backbones. A common starting point is the homogeneous system, where all agents share the same base LLM and differ only through prompts and roles. However, research increasingly challenges the efficacy of this approach. “Rethinking the Value of Multi-Agent Workflow” [101] demonstrates that many homogeneous workflows can be simulated by a single, well-instructed agent with a KV cache reuse advantage, questioning the inherent value of agent plurality in such settings. This perspective is reinforced by information-theoretic analyses: “Understanding Agent Scaling in LLM-Based Multi-Agent Systems via Diversity” [31] argues that homogeneous agents saturate early because their outputs are strongly correlated, providing diminishing returns as the agent count grows. In contrast, introducing heterogeneity—through diverse models, prompts, or tools—provides complementary evidence that can significantly boost performance, as heterogeneous agents access more effective “information channels.” This finding is central to architectures like “SC-MAS” [102], which, motivated by Social Capital Theory, argues that different agent roles benefit from different forms of collaboration. SC-MAS models the system as a directed graph where edges represent edge-level collaboration strategies, and a controller selects task-relevant agent roles, assigns these strategies, and allocates appropriate LLM backbones per agent, thus achieving heterogeneous collaboration at a fine-grained level. This notion of heterogeneity directly complements the adaptive coordination strategies from the previous subsection, as diverse agent capabilities and communication patterns require more sophisticated coordination protocols to be effective.

The choice of heterogeneous backbones also intersects with explicit resource constraints. “AgentBalance” [103] introduces a backbone-then-topology design philosophy, first constructing agents with heterogeneous backbones based on an LLM pool and then generating an adaptive topology. This prioritizes cost-effectiveness under explicit token-cost and latency budgets, suggesting that performance gains are often contingent on strategic backbone selection rather than merely increasing the number of agents. Similarly, “HieraMAS” [52] proposes a hierarchical framework that combines intra-node LLM mixtures (using a propose-synthesis structure within a role) with inter-node topology optimization, demonstrating that mixing LLMs within individual agents can strengthen role-specific abilities and enhance cost-performance trade-offs. However, heterogeneity alone is not a silver bullet; “Too Many Specialists” [104] identifies a “specialist’s dilemma” where rigid role assertion among heterogeneous

agents can generate system-level bottlenecks and workload inequality, emphasizing that the benefits of heterogeneity depend heavily on task structure and coordination protocols. This observation echoes the coordination-overhead concerns raised in the context of communication efficiency in the previous subsection, where excessive inter-agent negotiation can offset the benefits of specialization.

The scaling behavior of LLM-MAS is governed by a delicate balance between synergy and overhead. “Scaling Behavior of Single LLM-Driven Multi-Agent Systems” [28] systematically shows that MAS performance does not scale monotonically with agent count but instead follows a pattern of diminishing returns, moderated by the capability of the base LLM and the task type. The degradation is attributed to coordination overhead—a direct cost of inter-agent communication. This finding aligns with the “Communication to Completion” [105] framework, which models communication as a constrained resource with temporal costs, showing that cost-aware strategies significantly outperform unconstrained interaction. Interestingly, the performance ceiling is also task-dependent: “Towards Multi-Agent Reasoning Systems for Collaborative Expertise Delegation” [106] finds that collaboration integrating diverse knowledge consistently outperforms rigid task decomposition, and that scaling with expertise specialization involves significant computational trade-offs. These observations suggest that the optimal number of agents and the ideal communication topology are not fixed but must be dynamically aligned with task characteristics—a principle operationalized in frameworks like “AgentGroupChat-V2” [107], which employs a divide-and-conquer fully parallel architecture to decompose queries and dynamically select interaction modes. This dynamic alignment directly connects to the adaptive coordination strategies and feedback loops examined in the previous subsection, suggesting that system design and coordination mechanisms must co-evolve.

Critically, the value of multi-agent collaboration must be evaluated against strong single-agent baselines, a consideration that places the coordination mechanisms and design choices examined thus far in sharp relief. “The Illusion of Multi-Agent Advantage” [108] and “Single-Agent LLMs Outperform Multi-Agent Systems on Multi-Hop Reasoning Under Equal Thinking Token Budgets” [109] both present compelling evidence that, under controlled computation budgets, single-agent systems with sophisticated reasoning (e.g., Chain-of-Thought with Self-Consistency) can match or outperform automatically generated MAS. “The Illusion of Multi-Agent Advantage” [108] specifically argues that automated MAS design often produces “architectural bloat” that prioritizes superficial complexity over functional utility. Furthermore, “Multi-Agent Teams Hold Experts Back” [110] demonstrates that even when experts are present, self-organizing teams fail to leverage their expertise effectively, incurring performance losses due to an integrative compromise behavior. These studies collectively suggest that the benefits of multi-agent systems emerge under specific conditions—such as when relays are near-sufficient and weaker models benefit from isolated context, as formalized in “When Do Multi-Agent Systems Help” [58]—and are not an intrinsic property of plurality. This perspective is further nuanced by “Towards a Science of Scaling Agent Systems” [111], which suggests that architecture-task alignment is the most critical factor, where mismatched coordination can degrade performance by up to 70% relative to a single-agent baseline. Importantly, these findings imply that the coordination mechanisms surveyed in the previous subsection—however sophisticated—must justify their overhead against simpler baselines, and that the design choices examined here—context, heterogeneity, and scaling—are the levers through which such justification is achieved. In summary, the design of effective LLM-MAS requires a holistic consideration of context management, agent diversity, backbone selection, and resource budgets, with a clear-eyed view of the costs of coordination and the conditions under which “more” truly becomes “better”—a perspective that sets the stage for the broader system-level considerations and future directions that will be discussed in the remainder of this survey.

3 Memory, Learning, and Self-Evolution in Multi-Agent Systems

3.1 Memory Taxonomies and Hierarchical Architectures for Multi-Agent Systems

Memory serves as the foundational cognitive infrastructure that distinguishes stateless language models from genuinely adaptive agents capable of sustained reasoning, personalization, and collaborative problem-solving across extended interactions [112, 113]. Within the context of LLM-based Multi-Agent Systems (LLM-MAS), memory becomes an even more complex construct, as it must orchestrate not only individual agent cognition but also collective team knowledge, cross-agent coordination, and the accumulation of shared experience over time. This layered foundation—spanning individual cognition, collective knowledge, and shared experience—sets the stage for the next critical capability examined in this survey: how accumulated memories are transformed into durable, reusable skills. Drawing on cognitive science foundations and organizational memory theory, this subsection establishes a comprehensive taxonomy of memory types in LLM-MAS, examines the hierarchical architectures that manage the interplay between individual experiences and collective knowledge, and thereby provides the conceptual groundwork for understanding how raw episodic traces evolve into the procedural skills discussed in the following subsection.

Memory Taxonomies in LLM-MAS

The taxonomy of memory in LLM-MAS can be organized along multiple orthogonal dimensions. From a temporal perspective, memory is commonly divided into short-term (working) memory, which holds information relevant to the current task or session, and long-term memory, which persists across sessions and enables cumulative learning [113, 114]. Short-term memory in multi-agent settings is particularly challenging because agents must maintain not only their own task-relevant state but also awareness of collaborators’ actions and evolving team context. From a functional perspective, long-term memory is further subdivided into episodic memory (records of specific past experiences and interactions), semantic memory (generalized factual knowledge abstracted from experiences), and procedural memory (knowledge of how to perform tasks, encoded as skills or policies) [112, 115]. This episodic-semantic distinction mirrors human cognition, where episodic traces are progressively consolidated into semantic knowledge through reflective processes [116, 117]. Notably, the procedural memory category provides the direct conceptual bridge to the distillation mechanisms discussed in the following subsection, where interaction trajectories are transformed into executable skills. Recent work has also proposed a finer-grained taxonomy that distinguishes factual memory (what is known), experiential memory (what has been encountered), and working memory (what is currently active), arguing that traditional long/short-term dichotomies are insufficient to capture the diversity of contemporary agent memory systems [113]. Additional perspectives introduce memory forms such as token-level, parametric, and latent memory, where parametric memory resides in model weights and latent memory operates in embedding or hidden-state spaces [113, 114]. This representational substrate is a crucial design dimension, as different memory forms trade off interpretability, capacity, and retrieval efficiency.

Hierarchical Memory Architectures

Hierarchical organization has emerged as a dominant design principle for structuring memory in LLM-MAS, inspired by both cognitive neuroscience and organizational theory. This structural organization directly influences how effectively accumulated experiences can later be distilled into reusable knowledge, as hierarchical abstraction layers naturally correspond to different levels of skill granularity. Drawing on organizational memory theory, G-Memory introduces a three-tier graph hierarchy comprising insight, query, and interaction graphs to manage lengthy multi-agent interactions [118]. This architecture enables bi-directional memory traversal that retrieves both high-level, generalizable insights for cross-trial knowledge and fine-grained, condensed interaction

trajectories that compactly encode prior collaboration experiences, with the entire hierarchy evolving by assimilating new collaborative trajectories after each task execution [118]. Similarly, H-MEM organizes memory in a multi-level fashion based on the degree of semantic abstraction, where each memory vector carries a positional index encoding that points to semantically related sub-memories in the next layer, enabling efficient layer-by-layer retrieval without exhaustive similarity computations [119]. This approach addresses the limitations of flat, similarity-based retrieval by providing structured organization and efficient indexing. Another hierarchical design, BMAM, decomposes agent memory into functionally specialized subsystems—episodic, semantic, salience-aware, and control-oriented components—that operate at complementary time scales, with episodic memories organized along explicit timelines to support temporal reasoning [120]. The system’s hippocampus-inspired episodic subsystem is critical for maintaining behavioral consistency across sessions, addressing what the authors term “soul erosion” [120].

The hierarchical memory design space also encompasses multi-level semantic abstraction structures that separate planning-level knowledge from execution-level details. H²R, for instance, decouples high-level planning memory from low-level execution memory within a hierarchical architecture, enabling fine-grained knowledge transfer by retrieving these layers separately at test time [121]. This separation recognizes that planning knowledge (strategies, task decompositions) and execution knowledge (action sequences, tool usage) exhibit different abstraction levels and reuse patterns—a distinction that directly parallels the episodic-to-procedural transformation elaborated in the following subsection, where high-level strategies and low-level action sequences require different distillation mechanisms. HyperMem takes a hypergraph-based approach, structuring memory into three levels—topics, episodes, and facts—and grouping related episodes and their facts via hyperedges to capture high-order associations that pairwise graph structures miss [122]. The coarse-to-fine retrieval strategy over this hierarchy supports accurate and efficient retrieval, achieving state-of-the-art performance on long-term conversation benchmarks [122].

For multi-agent systems specifically, the hierarchy must additionally manage the tension between individual agent experience and collective team knowledge. G-Memory’s tri-level hierarchy addresses this by maintaining fine-grained interaction trajectories that encode collaboration paths among agents, alongside higher-level insights that generalize across trials [118]. Similarly, the Hierarchical Long-Term Semantic Memory (HLTM) framework deployed in LinkedIn’s Hiring Agent organizes textual data into a schema-aligned memory tree that captures semantic knowledge across multiple levels of granularity, supporting scalable ingestion, privacy-aware storage, and low-latency retrieval [123]. This industrial deployment demonstrates the practical viability of hierarchical memory architectures in production multi-agent systems.

Design Choices: Shared vs. Per-Agent Memory and Memory Topologies

A critical design dimension in LLM-MAS memory architectures is the choice between shared and per-agent memory, and the associated memory topology. This design choice fundamentally shapes what kinds of collective experiences can be accumulated and subsequently distilled into shared procedural knowledge, as the granularity and accessibility of stored information directly constrain the distillation mechanisms described in the following subsection. From a computer architecture perspective, multi-agent memory can be framed as a problem of shared versus distributed memory paradigms, with a proposed three-layer memory hierarchy (I/O, cache, and memory) and open challenges in cache sharing across agents and structured memory access control [124]. The authors identify multi-agent memory consistency as the most pressing open challenge, highlighting the difficulty of maintaining coherent shared state in distributed settings [124]. This architectural framing draws useful analogies between multi-agent memory management and classical distributed systems problems.

The shared versus distributed distinction has significant implications for coordination efficiency

and consistency. The concept of governed collaborative memory extends this by proposing that memory in multi-agent systems be treated as institutional state, governed by explicit selection regimes that determine which candidate memories become shared, which remain private, and which are rejected [125]. This perspective introduces a layered architecture separating agent-local memory, shared institutional memory, archive memory, and project-continuity memory, with provenance and version lineage making selection decisions inspectable [125]. The notion of memory governance as artificial selection emphasizes that persistent multi-agent systems should evaluate memory not only for recall and performance but also for provenance fidelity, selection traceability, epistemic quality, and role preservation [125]. Crucially, these governance mechanisms determine which memories are deemed worthy of preservation and refinement—a selection process that directly conditions the quality of the experience pool available for the distillation and skill-formation mechanisms examined next.

Collaborative memory frameworks further address multi-user, multi-agent environments with asymmetric and time-evolving access controls [126]. In this design, two memory tiers are maintained—private memory visible only to originating users and shared memory selectively shared across users and agents—with each memory fragment carrying immutable provenance attributes (contributing agents, accessed resources, timestamps) to support retrospective permission checks [126]. This provenance-rich approach enables safe and interpretable cross-user knowledge sharing with provable adherence to access policies and full auditability [126].

The empirical evaluation of memory topologies in multi-agent systems reveals non-trivial interactions between team size, memory design, and long-term performance. The LLMA-Mem framework studies how memory design shapes the scaling landscape across team size and lifelong learning ability, revealing a non-monotonic scaling landscape where larger teams do not always produce better long-term performance [127]. Smaller teams can outperform larger ones when memory better supports the reuse of experience, positioning memory design as a practical path for scaling multi-agent systems more effectively over time [127]. This finding underscores that the value of memory lies not merely in storage capacity but in how effectively accumulated experiences can be retrieved and repurposed—directly motivating the retrieval and distillation mechanisms that constitute the focus of the following subsection.

The Write-Manage-Read Loop

A unifying abstraction for understanding memory operations in LLM-MAS is the write–manage–read loop, which formalizes memory as a continuous cycle of storing new information, managing existing memories, and retrieving relevant content when needed [112]. This loop is tightly coupled with perception and action, and the control policy governing when and how memories are written, consolidated, and retrieved is itself a critical design dimension. The write and manage phases of this loop determine the quality and organization of stored experiences, while the read phase directly feeds into the retrieval mechanisms that the following subsection examines in depth—highlighting that effective distillation presupposes well-managed memory infrastructure. Recent work has proposed conceptualizing memory management as a trainable metamemory skill—a framework instantiated in AutoMem, which promotes file-system operations to first-class memory actions and learns both the memory structure and the model’s proficiency in exercising it [128]. The results demonstrate that optimizing memory management alone can improve base agent performance by 2-4x on long-horizon tasks, establishing memory management as an independently learnable and high-leverage skill [128]. Extending this direction further, AgeMem integrates long-term and short-term memory management directly into the agent’s policy by exposing memory operations as tool-based actions, enabling the LLM agent to autonomously decide what and when to store, retrieve, update, summarize, or discard information, trained via three-stage progressive reinforcement learning with step-wise GRPO [129]. This policy-learned approach represents a significant departure from heuristic-based memory

management, enabling adaptation and end-to-end optimization of memory strategies.

The write–manage–read abstraction, combined with the taxonomy of memory types and hierarchical architectures, provides a principled framework for comparing diverse LLM-MAS memory designs. The cognitive science foundations—particularly the episodic-semantic distinction and the notion of hierarchical consolidation—and organizational memory theory jointly inform the design of memory systems that balance individual agent autonomy with collective intelligence [115, 118]. The emerging emphasis on memory governance and policy-learned management signals a trend toward treating memory not as a passive storage layer but as an active, adaptive component of agent cognition. This active perspective directly sets up the central question of the following subsection: given well-structured and well-governed memory, how do multi-agent systems transform the accumulated episodic traces and semantic knowledge into durable procedural skills and meta-cognitive strategies that enable genuine self-evolution? The memory architectures surveyed here thus constitute the essential substrate upon which the distillation, retrieval, and skill-formation mechanisms examined next operate, with far-reaching implications for multi-agent coordination, personalization, and continual learning in complex task environments.

3.2 Experience Accumulation, Retrieval, and Procedural Skill Formation

The transformation of raw interaction trajectories into reusable knowledge constitutes the cognitive core of self-evolving LLM-based multi-agent systems. Building directly upon the memory taxonomies and hierarchical architectures established in the preceding subsection, this process—spanning experience accumulation, adaptive retrieval, and procedural skill formation—determines whether an agent population merely executes tasks or genuinely learns from them. The challenges are multifaceted: trajectories are noisy, context-specific, and costly to store, while skills must be generalizable, executable, and verifiable. This subsection examines the mechanisms by which multi-agent systems bridge the gap between episodic traces and durable procedural knowledge, addressing the critical tension between passive accumulation and active refinement. In doing so, it provides the operational link between the structured memory substrate described earlier and the learning frameworks for self-evolution that follow.

Multi-Faceted Distillation: From Raw Trajectories to Reusable Knowledge. The first step in experience accumulation is distilling raw trajectories into structured, reusable forms—a process that directly operationalizes the episodic-to-semantic consolidation and proceduralization introduced in the memory taxonomy. A central finding is that naive trajectory storage is often suboptimal: full trajectories are too context-specific to transfer across tasks, while tool-level reuse ignores surrounding context and environmental constraints. The H-EPM framework addresses this by constructing a tool graph from accumulated trajectories, where recurring tool-to-tool dependencies capture procedural routines, and each edge is augmented with compact episodic summaries of relevant context [130]. This hybrid approach dynamically balances episodic recall for contextual reasoning with procedural execution for routine steps, demonstrating that distillation must preserve both the “what” (procedural dependencies) and the “when/why” (episodic context)—a duality that mirrors the hierarchical memory architectures’ separation of planning-level and execution-level knowledge. Similarly, the ReMe framework emphasizes multi-faceted distillation, which extracts fine-grained experiences by recognizing success patterns, analyzing failure triggers, and generating comparative insights [131]. This goes beyond simple summarization by explicitly modeling the contrast between successful and unsuccessful trajectories, enabling the system to learn not only what works but also what does not and why. The SkillDisCo framework formalizes this distillation as a compilation problem: in FSM-defined scenarios, successful traces are viewed as paths in an unknown transition graph, and procedural skills are formulated as reusable parameterized control-flow subgraphs distilled

from these paths [132]. This control-flow perspective enables skills to be callable, executable, and verifiable, moving beyond textual descriptions toward operational artifacts—a transformation that directly feeds the skill-based optimization and decoupling-from-parameters paradigms examined in the following subsection.

A critical insight emerging from recent work is the distinction between passive and active distillation. The Experience Compression Spectrum framework positions memory, skills, and rules as points along a single axis of increasing compression, revealing that most systems operate at a fixed, predetermined compression level without supporting adaptive cross-level compression [133]. This “missing diagonal” suggests that systems rarely adapt their distillation granularity based on task demands—a limitation that the hierarchical memory architectures of the preceding subsection partially address through multi-level abstraction but which requires explicit distillation policies to fully resolve. The ReMe framework challenges the “passive accumulation” paradigm that treats memory as a static append-only archive, instead proposing utility-based refinement that autonomously adds valid memories and prunes outdated ones to maintain a compact, high-quality experience pool [131]. This active refinement is crucial: without it, memory systems accumulate noise, redundancy, and even harmful artifacts. Indeed, research shows that consolidated memories produced by LLMs can become faulty even when derived from useful experiences, with memory utility rising then degrading as consolidation proceeds [134]. This finding underscores that distillation is not a one-time transformation but an ongoing process requiring careful governance—a principle that resonates with the write–manage–read loop and the adaptive memory management policies discussed as future directions in the preceding subsection.

Context-Adaptive Retrieval: Similarity, Graph Navigation, and Policy Learning.

Once experience is distilled, the retrieval mechanism determines whether and how it is applied to new tasks. This retrieval stage operationalizes the “read” phase of the write–manage–read loop and determines how effectively the hierarchical memory structures described earlier are exploited. Retrieval strategies span a spectrum from simple similarity matching to sophisticated policy-learned selection. Static methods relying on fixed similarity heuristics are limited because they do not optimize downstream utility [135]. Context-adaptive retrieval addresses this by tailoring historical insights to new contexts via scenario-aware indexing [131]. Graph-based retrieval has emerged as a powerful paradigm: the HiSkill framework organizes interaction trajectories into a hierarchical skill graph with skill nodes, AtomicOp nodes, and typed edges capturing decomposition, temporal transition, compatibility, support, and recovery relations [136]. At inference time, HiSkill retrieves a compact task-relevant subgraph and performs subgraph-guided task execution, demonstrating that structural relations among skills provide richer retrieval signals than flat collections. This graph-based organization directly extends the hierarchical memory architectures discussed earlier, translating the insight-query-interaction or planning-execution hierarchies into operational retrieval structures. Similarly, the SkillLens framework organizes skills into a four-layer graph of policies, strategies, procedures, and primitives, retrieving them at mixed granularity through degree-corrected random walk expansion [137]. This multi-granularity approach addresses the tension between relevance and cost: injecting coarse skills can introduce irrelevant context, while rewriting entire skills is expensive—a trade-off that parallels the compression-specificity trade-off along the Experience Compression Spectrum.

Policy-learned retrieval represents the frontier of adaptive selection. The MERIT framework maintains episode-level memory for global strategic guidance and turn-level memory for local decision support, with both levels using learned retrieval policies optimized with reinforcement learning [135]. This dual-horizon approach recognizes that memory usefulness changes across interaction stages, with memories useful for initial planning differing from those needed for state-conditioned execution—a distinction that mirrors the hierarchical separation of planning-level and execution-level knowledge

in H^2R described earlier. The SkillReranker framework performs semantic decomposition on both task and skill sides, constructing a directed acyclic execution graph where intermediate task states are nodes and candidate skills are edges, then employing a cross-encoder to score candidate skills for each task interval [138]. This structured task-skill correspondence enables more accurate selection than purely semantic matching. Furthermore, the MemCon framework models memory operations as a Markov Decision Process, learning an online policy that adaptively decides when, what, and how much to retrieve, when to inject a distilled plan, and when to consolidate or forget [139]. This controlled-process view treats retrieval not as a fixed function but as a learnable behavior that optimizes the trade-off between information sufficiency and context efficiency—directly extending the trainable metamemory skill concept introduced in the preceding subsection’s discussion of the write–manage–read loop, and anticipating the multi-agent reinforcement learning approaches for memory construction that the following subsection examines in depth.

From Episodic Traces to Procedural Skills and Meta-Cognitive Strategies. The ultimate goal of experience accumulation is the formation of procedural skills—reusable behavioral patterns that generalize across tasks. This evolution from episodic traces to procedural skills represents the culmination of the hierarchical consolidation processes introduced in the memory taxonomy, and it constitutes the target output that the self-evolution frameworks of the following subsection aim to optimize. However, this transformation is not automatic but requires explicit mechanisms. The SkillEvolBench benchmark systematically evaluates this step, containing tasks organized into role-conditioned task families with shared latent procedures, where agents learn from acquisition tasks, update an external skill library, and face frozen deployment tasks [140]. Its findings are sobering: current agents often adapt locally but rarely form robust reusable skills, and raw-trajectory reuse frequently outperforms distilled skills, suggesting that current abstraction procedures discard contextual and procedural cues that remain useful for future tasks. This challenges the assumption that distillation always improves generalization, highlighting the need for more sophisticated abstraction mechanisms—a challenge that motivates the experience-driven self-evolution approaches with hierarchical memory discussed in the following subsection. The Skill-Pro framework addresses this by formalizing a Skill-MDP that transforms passive episodic narratives into executable Skills defined by activation, execution, and termination conditions [141]. Non-Parametric PPO leverages semantic gradients for high-quality candidate generation and a PPO Gate for robust Skill verification, achieving extreme memory compression while sustaining high reuse rates.

The transition to meta-cognitive strategies represents a higher level of abstraction beyond individual skills. The MAGE framework externalizes self-knowledge into a four-subgraph co-evolutionary knowledge graph, with an experience subgraph storing both teacher-written corrections and the learner’s own correct traces, retrieved as task-conditioned guidance for a frozen execution model [142]. During evolution, the graph, a task-level search bandit, and a skill-level routing bandit are updated from the same reward stream, enabling stable improvement of the retrieval substrate. This co-evolution of knowledge content and retrieval policy exemplifies the shift from passive storage to active cognitive infrastructure—a shift that directly parallels the meta-evolutionary and joint optimization frameworks presented in the following subsection, where both knowledge and learning mechanisms are refined. The Skill-MAS framework conceptualizes high-level orchestration capability as an evolvable Meta-Skill, refined through multi-trajectory rollout and selective reflection with hierarchical contrastive analysis [143]. These meta-skills capture systemic, strategy-level principles that transfer across unseen tasks and different LLMs, demonstrating that the highest-value knowledge is not task-specific but architectural and strategic. This aligns with the observation from the Experience Compression Spectrum that transferability increases with compression at the cost of specificity [133], suggesting that meta-cognitive strategies, while less detailed, provide the

most generalizable guidance—a property that becomes essential for the agent co-adaptation and collective improvement mechanisms examined in the following subsection.

Lifecycle Governance and the Passive–Active Spectrum. A persistent challenge across experience accumulation is lifecycle management—a concern that extends the governance and selection mechanisms introduced in the preceding subsection’s discussion of governed collaborative memory. The SkillsVote framework addresses this through collection, recommendation, attribution, and evolution phases, profiling a million-scale open-source corpus for environment requirements, quality, and verifiability, and decomposing trajectories into skill-linked subtasks with evidence-gated updates [144]. Similarly, the SkillOS framework trains a skill curator that updates an external SkillRepo from accumulated experience, with composite rewards and grouped task streams enabling learning of complex long-term curation policies [145]. The ReSkill framework reconciles skill creation with policy optimization by exploiting the group-wise structure of GRPO to embed assertion-driven skill creation, within-group rollout sampling, and Thompson Sampling for skill version selection [146]. These frameworks collectively address the challenge of passive accumulation versus active refinement: durable systems must not only store knowledge but also curate, prune, and evolve it in response to task outcomes—a principle that aligns with the memory governance and artificial selection perspective introduced in the preceding subsection, and that anticipates the reinforcement learning approaches for memory construction and the joint optimization frameworks for credit assignment in the following subsection.

The trade-off between raw-trajectory replay and distilled skills remains a central design question. While distillation offers compression, efficiency, and generalization benefits, it risks information loss and procedural clutter. The finding that raw-trajectory reuse frequently outperforms distilled skills in SkillEvolBench [140] suggests that episodic evidence retains contextual cues that are valuable yet difficult to abstract—echoing the preceding subsection’s emphasis on the importance of preserving fine-grained interaction trajectories alongside high-level insights. Conversely, the success of skill-based methods with sophisticated distillation and verification mechanisms [141, 146] demonstrates that high-quality skills, when properly formed and governed, can surpass raw experience. The resolution likely lies in hybrid approaches that combine episodic memory for contextual reasoning with procedural skills for routine execution [130], along with adaptive mechanisms that dynamically determine the appropriate compression level for each situation—capabilities that the self-evolution frameworks in the following subsection aim to learn and optimize. Future systems must navigate this spectrum intelligently, recognizing that the optimal representation of experience is not fixed but depends on the task, the learner’s current capability, and the cost-benefit structure of the deployment context. The distillation and retrieval mechanisms surveyed here thus constitute the essential bridge between memory infrastructure and self-evolution capability, providing the raw material and operational substrate upon which the reinforcement learning, skill-based optimization, and meta-evolutionary frameworks examined next operate.

3.3 Self-Evolution Frameworks and Multi-Agent Learning Mechanisms

The capacity for self-evolution distinguishes static multi-agent systems from truly adaptive ones, and it constitutes the learning engine that powers the experience accumulation mechanisms described in the preceding subsection. While the previous discussion established *what* knowledge should be distilled, retrieved, and governed, this subsection investigates the *how*—the frameworks and learning mechanisms that enable LLM-based Multi-Agent Systems (LLM-MAS) to improve over time through both internal and external feedback loops. We organize the discussion around four complementary threads: multi-agent reinforcement learning for memory construction, experience-driven self-evolution with hierarchical memory modules, skill-based optimization that decouples

experience retention from parametric updates, and meta-evolutionary approaches that jointly refine experiential knowledge and the memory architecture itself, along with joint optimization frameworks for credit assignment and agent co-adaptation.

Multi-Agent Reinforcement Learning for Memory Construction. A central challenge in LLM-MAS is learning how agents should construct, update, and retrieve memories to support downstream objectives—a challenge that directly extends the lifecycle governance concerns raised earlier. Early approaches relied on hand-crafted rules, but recent work frames memory management as a trainable policy. For instance, **Mem- α** trains agents via reinforcement learning (RL) to manage complex memory systems, where the reward is derived from downstream question-answering accuracy, enabling agents to learn what to store, how to structure it, and when to update it [147]. This RL-based paradigm is extended by **MemCoE**, which draws on memory schema theory to design a two-stage optimization: first, a global memory guideline is induced via contrastive feedback interpreted as textual gradients; second, guideline-aligned multi-turn RL learns a memory evolution policy, providing structured process rewards that stabilize long-horizon optimization [148]. In a similar vein, **Memory-R2** addresses the credit assignment challenge in long-horizon memory-augmented agents by proposing LoGo-GRPO, which combines local and global group-relative optimization to ensure fairer comparisons of memory-operation outcomes from the same intermediate memory state, alongside a progressive curriculum to stabilize multi-step RL [149].

However, optimizing each agent independently under a shared global objective often fails to capture inter-agent dependencies—a limitation that becomes particularly salient given the non-monotonic scaling landscape discussed in the following subsection. To address this, **CoMAM** proposes a joint optimization framework that models the multi-agent memory pipeline as a Markov decision process (MDP), enabling end-to-end RL with an adaptive credit assignment mechanism that combines local task rewards with adaptively weighted global rewards [150]. This joint training fosters co-adaptation among agents with specialized roles. A related but distinct approach is **LERO**, which leverages LLMs within an evolutionary framework to generate hybrid reward functions and observation enhancement functions for multi-agent reinforcement learning, addressing the dual bottlenecks of credit assignment and partial observability [151]. Furthermore, **MAGE** (Meta-Reinforcement Learning for Language Agents) presents a meta-RL framework that integrates interaction histories and reflections into the context window, using the final episode reward to incentivize strategic refinement, thereby enhancing both exploration and exploitation in multi-agent settings [152].

Experience-Driven Self-Evolution with Hierarchical Memory. Beyond RL-based training, many frameworks enable self-evolution at test time by structuring memories hierarchically—a design choice that resonates with the multi-faceted distillation and context-adaptive retrieval mechanisms examined previously. **G-Memory** introduces a three-tier graph hierarchy (insight, query, and interaction graphs) inspired by organizational memory theory, allowing the system to retrieve both high-level generalizable insights and fine-grained interaction trajectories [118]. This design enables cross-trial learning and progressive team evolution. **LLMA-Mem** offers a lifelong memory framework for multi-agent systems under flexible memory topologies, demonstrating that memory design can enable smaller teams to outperform larger ones by better reusing experience, revealing a non-monotonic scaling landscape [127]. **SEDM** (Self-Evolving Distributed Memory) transforms memory into an active component with verifiable write admission, a self-scheduling controller that consolidates entries based on empirical utility, and cross-domain knowledge diffusion for robust generalization [153]. For multi-agent systems specifically, **DecentMem** proposes a decentralized memory framework where each agent maintains its own dual-pool memory—an exploitation pool for consolidated past trajectories and an exploration pool for candidate strategies—theoretically guaranteeing global reachability and near-optimal regret bounds while reducing communication

overhead [154]. Similarly, **LatentMem** addresses memory homogenization and information overload by customizing agent-specific memories in a token-efficient manner via a learnable memory composer, optimized through Latent Memory Policy Optimization (LMPO) [155].

Skill-Based Optimization and Decoupling Experience from Parameters. A significant trend in self-evolution is decoupling experience retention from parametric updates, enabling agents to accumulate knowledge without the high cost of fine-tuning large models. This paradigm directly complements the procedural skill formation discussed in the previous subsection, providing the learning mechanisms by which such skills are discovered and refined. **Skill-MAS** conceptualizes the high-level orchestration capability of automatic multi-agent systems as an evolvable Meta-Skill, refined through a closed optimization loop of multi-trajectory rollout and selective reflection, thereby bridging the gap between frozen frontier LLMs and training-time internalization [143]. **SkillMAS** couples skill evolution with multi-agent restructuring, using utility learning for credit assignment and evidence-gated restructuring to adapt system organization [156]. **MemSkill** reframes memory operations themselves as learnable and evolvable skills, where a controller learns to select relevant skills, an LLM-based executor produces skill-guided memories, and a designer periodically reviews hard cases to propose skill refinements [157]. **MSCE** (Memory–Skill Co-Evolution) similarly organizes experience into grounded step traces, procedural policies, and declarative environment cognition, crystallizing evidence-backed policies into callable skills and governing evolution through reflection-weighted value backfilling [158]. **ExpGraph** builds a model-agnostic experience learning framework that summarizes trajectories into reusable skills and failure lessons organized as a graph, retrieved via graph diffusion and utility-aware ranking, allowing frozen executors to improve without parameter updates [159].

Meta-Evolution, Joint Optimization, and Agent Co-Adaptation. The frontier of self-evolution research involves meta-level optimization that refines not just the knowledge but also the mechanisms through which agents learn—an ambition that anticipates the lifelong learning and continuous improvement concerns of the following subsection. **MemEvolve** introduces a meta-evolutionary framework that jointly evolves agents’ experiential knowledge and their memory architecture, allowing the system to progressively refine how it learns from experience; it is grounded in EvolveLab, a unified codebase distilling twelve memory systems into a modular design space [160]. **EvolveMem** exposes the full retrieval configuration of a memory system as a structured action space optimized by an LLM-powered diagnosis module, enabling closed-loop self-evolution of retrieval strategies with revert-on-regression safeguards [161]. **RoMeRL** addresses the memory-reward trap by representing trajectory-indexed utility with fixed-dimensional per-task memory states, improving feedback density and reducing reward contamination [162]. **MetaEvo** focuses on improving how the model learns from experience through preference-based optimization of principle abstraction [163], while **MemEvolve** and **EvolveMem** demonstrate the power of co-evolving the memory substrate itself [160, 161].

Joint optimization and co-adaptation are critical for multi-agent collaboration, serving as the mechanism by which individual learning translates into collective improvement. **Meta-Team** enables experience-driven MAS evolution by preserving execution context and coordinating post-task communication, allowing agents to exchange distributed evidence and conduct multi-scale self-evolution that improves behaviors, coordination, and team organization [29]. **CoMAS** (Co-Evolving Multi-Agent Systems) generates intrinsic rewards from inter-agent discussion dynamics, optimized via RL without external supervision, enabling decentralized and scalable co-evolution [164]. **Multi-Agent Evolve** instantiates a triplet of interacting agents (Proposer, Solver, Judge) from a single LLM and applies RL to co-optimize their behaviors, achieving data-efficient self-improvement on diverse tasks [165]. **RetroAgent** augments extrinsic rewards with hindsight-generated dual intrinsic feedback—numerical feedback for exploration and language feedback for reusable lessons—

updating the system through a Similarity & Utility-Aware UCB retrieval strategy [166]. Finally, **CoEvolve** addresses the challenge of static data distributions by enabling closed-loop, interaction-driven training where feedback from rollouts guides task synthesis, jointly adapting the agent and its data distribution [167]. These approaches collectively illustrate that robust self-evolution in LLM-MAS requires not only sophisticated memory architectures and learning algorithms but also principled mechanisms for credit assignment, agent co-adaptation, and the continuous refinement of the learning process itself—foundational capabilities that must be sustained over extended time horizons, as the next subsection examines in detail.

3.4 Lifelong Learning, Test-Time Adaptation, and Continuous Improvement

The temporal dimension of LLM-based Multi-Agent Systems (LLM-MAS) represents one of the most critical yet underexplored frontiers in the field. While static multi-agent configurations can address complex tasks at a fixed point in time, real-world deployments demand that systems sustain performance over extended periods, adapt to evolving task distributions, and continuously accumulate and refine knowledge. This subsection examines the landscape of lifelong learning, test-time adaptation, and continuous improvement mechanisms for LLM-MAS, focusing on strategies for mitigating catastrophic forgetting, extending learning horizons, and governing the evolution of both memory and skills. In doing so, it extends the preceding discussion of self-evolution frameworks—which established *how* agents improve through internal and external feedback loops—by asking a more demanding question: *how can these improvements be sustained, consolidated, and protected over extended operational horizons?* While the previous subsection focused on the mechanisms of learning and co-adaptation, here we shift attention to the temporal dynamics that determine whether such learning translates into durable capability, and we lay the groundwork for the architectural and governance considerations examined next.

Lifelong Learning Strategies and the Challenge of Catastrophic Forgetting. The central obstacle to lifelong learning in LLM-MAS is catastrophic forgetting, the phenomenon where the acquisition of new knowledge degrades previously learned capabilities [168, 169]. In multi-agent settings, this challenge is compounded by the fact that both individual agents and the collective system must preserve knowledge across tasks and sessions. Research in this area has revealed several critical insights. For instance, the fine-tuning regime—defined by the trainable parameter subspace—itself constitutes a key evaluation variable, and comparative conclusions about continual learning methods can vary substantially across different trainable depth regimes [170]. This suggests that no universal solution exists; rather, the choice of adaptation strategy must be carefully aligned with the specific architectural configuration of the multi-agent system.

Memory-replay approaches have emerged as a dominant paradigm for mitigating forgetting. However, conventional experience replay often exhibits limited effectiveness for LLM agents due to the retention of irrelevant information and context length constraints [171]. More sophisticated approaches have been developed to address these limitations. For example, General Sample Replay (GeRe) demonstrates that a small, fixed set of pre-collected general replay samples can simultaneously retain general capabilities while promoting performance across sequential tasks [172]. Similarly, the MSSR framework estimates sample-level memory strength and schedules rehearsal at adaptive intervals, showing particular gains on reasoning-intensive benchmarks [173]. The FOREVER framework aligns replay schedules with a model-centric notion of time based on the magnitude of optimizer updates, rather than raw training steps, thereby better aligning with the model’s internal evolution [174]. These advances highlight a crucial shift: effective lifelong learning requires moving beyond fixed, step-based heuristics toward more adaptive, model-aware mechanisms. This theme of adaptive, model-aware scheduling directly parallels the self-scheduling consolidation mechanisms

introduced in the previous subsection’s discussion of SEDM, where memory entries are consolidated based on empirical utility rather than fixed rules.

Curriculum-Based Training Horizons and Progressive Extension. Long-horizon training in multi-agent systems faces the fundamental challenge that memory turns the agent’s past actions into part of its future environment, making trajectory-level comparisons across rollouts fundamentally unfair due to divergent intermediate memory states [149]. This challenge is particularly acute when considered alongside the skill-based optimization frameworks from the previous subsection, where experience is decoupled from parameters but must still be integrated into long-horizon training regimes. To address this, a progressive curriculum that increases the training horizon from 8 to 16 to 32 sessions has been proposed, enabling more stable multi-step reinforcement learning over long memory horizons while providing fairer group comparisons for memory construction operations [149]. This progressive horizon extension addresses the instability that arises when agents must simultaneously learn task-solving policies and memory-management strategies over increasingly long interaction sequences, and it complements the progressive curriculum strategies already discussed in the context of Memory-R2’s RL framework.

Test-Time Adaptation and Self-Evolution. Beyond static lifelong learning, test-time adaptation has emerged as a critical capability for LLM-MAS deployed in dynamic environments, and it represents a natural extension of the self-evolution mechanisms detailed in the previous subsection. The Self-Optimizing Lifelong Autonomous Reasoner (SOLAR) exemplifies this paradigm, employing parameter-level meta-learning to treat model weights as an environment for exploration, and enabling efficient test-time adaptation to unseen domains without gradient-based fine-tuning [175]. The Skill-enhanced Test-Time Co-Evolution framework (LifeSkill) goes further by proposing a two-stage reinforcement learning approach for online lifelong learning agents: Verifier-Guided Skill Learning addresses the lack of direct supervision for skill extraction by rewarding candidate skills based on the average verifier success of multiple skill-conditioned policy rollouts, while Online Skill Internalization continuously improves the policy model during test-time interaction by transforming skill-conditioned trajectories into reward signals, thereby enabling the agent to internalize reasoning capabilities directly into its parameters [176]. This two-stage design mirrors the skill-based optimization paradigm from the previous subsection, where skills are first discovered or proposed, then selectively refined based on evidence of utility.

A particularly important insight is that lifelong learning is not a single capability; different patterns of environmental change—whether new domains arriving over time, facts drifting past their training cutoff, or agentic interactions accumulating state across episodes—require fundamentally different update behaviors [177]. Prompt-based methods fit each new stage quickly but degrade on future tasks; distillation-based methods accumulate knowledge stably but struggle to update outdated facts; and online reinforcement learning adapts most effectively to knowledge updates but remains sensitive to noisy reward signals [177]. This finding underscores the need for mechanism-agnostic protocols that can systematically compare divergent adaptation strategies under realistic sequential conditions, and it reinforces the meta-evolutionary perspective from the previous subsection—where the learning mechanism itself must be subject to refinement.

Benchmarking Frameworks for Lifelong Learning. Rigorous evaluation is essential for advancing lifelong learning in LLM-MAS, and this is where the field overlaps most directly with the governance and verification concerns that will be examined in the following subsection. Multiple benchmarking frameworks have been developed to assess different dimensions of lifelong learning. LifelongAgentBench provides the first unified benchmark designed to systematically assess the lifelong learning ability of LLM agents, offering skill-grounded, interdependent tasks across Database, Operating System, and Knowledge Graph environments with automatic label verification and modular extensibility [171]. PATH-Bench focuses on path-dependent evaluation, estimating

directed task relationships and constructing probe-centered sequences with controlled helpful and interfering histories; it measures average performance, forward transfer, backward transfer, and forgetting, revealing that experience utility depends jointly on how experience is represented and on the task’s interaction structure [178]. EvoAgentBench shifts the focus to agent self-evolution via ability-guided transfer across web research, algorithmic reasoning, software engineering, and knowledge work domains, extracting trace-grounded abilities and building domain-specific ability graphs to diagnose experience encoding, routing, and uptake rather than merely comparing aggregate accuracy [179]. CL-Bench, another significant contribution, spans six diverse domains validated by domain experts and isolates online learning from underlying model capability, finding that dedicated memory systems frequently underperform naive in-context learning, indicating substantial headroom for improved continual learning systems [180]. These benchmarks provide the empirical foundation upon which the governance frameworks discussed next must operate.

Scaling, Memory Governance, and Sustainable Improvement. The interaction between team size and lifelong learning ability under realistic cost constraints reveals a non-monotonic scaling landscape: larger teams do not always produce better long-term performance, and smaller teams can outperform larger ones when memory better supports the reuse of experience [127]. This finding directly echoes the non-monotonic scaling landscape introduced in the previous subsection through LLMA-Mem, and it positions memory design as a practical path for scaling multi-agent systems more effectively and efficiently over time. Governance of memory evolution is essential for sustainable lifelong improvement, and it provides the bridge to the following subsection’s examination of systemic governance. Live-Evo decouples “what happened” from “how to use it” via an Experience Bank and a Meta-Guideline Bank, maintaining experience weights that are updated from feedback—experiences that consistently help are reinforced and retrieved more often, while misleading or stale experiences are down-weighted and gradually forgotten [181]. The EvoMem paradigm records memory evolution as structured update histories, enabling agents to reason about environmental evolution through changes in their memory, achieving consistent improvements on both evolving and standard benchmarks [182]. EvolveMem takes this further by making the full retrieval configuration a structured action space optimized by an LLM-powered diagnosis module, enabling closed-loop self-evolution where the system autonomously conducts iterative research cycles on its own architecture [161]. Together, these frameworks underscore that lifelong learning in LLM-MAS requires not merely accumulating experience but actively governing how that experience is stored, retrieved, and refined to ensure sustained, positive transfer across an ever-evolving task landscape. This governance dimension—the policies, safeguards, and oversight mechanisms that ensure continued positive transfer—lays the foundation for the systemic governance frameworks examined in the next subsection.

4 Planning, Reasoning, and Decision-Making in Multi-Agent Collaboration

4.1 Advanced Planning Methodologies in Multi-Agent Collaboration

Building on the neuro-symbolic and causality-driven foundations established earlier, advanced planning in multi-agent systems has evolved from purely symbolic, centralized paradigms toward integrated frameworks that leverage the reasoning strengths of large language models (LLMs) while retaining formal guarantees. A dominant theme is the decomposition of complex tasks into hierarchical structures, which reduces the search space and enables agents to manage long-horizon objectives. The [183] exemplifies this by coupling a classical symbolic planner with a transformer-based policy; the planner assembles interpretable operator sequences that guarantee logical coherence, and these operators condition a decision transformer to generate fine-grained

actions. This bidirectional interface preserves the explainability of symbolic reasoning while maintaining the adaptability of deep sequence models. Similarly, [184] introduces a framework that uses simple temporal networks to reason about both precedence constraints from task-level coordination and temporal constraints from kinematic limitations. This integration allows for scalable plan generation and robust execution that can absorb plan deviations without frequent replanning, which is critical for long-term robot autonomy.

The integration of LLMs with symbolic planners has become a powerful approach to bridge the gap between natural language understanding and formal, verifiable planning. [185] proposes the Onto-LLM-TAMP framework, which refines user prompts by integrating domain-specific knowledge and environment state descriptions. This knowledge-based prompting resolves semantic errors—such as maintaining temporal goal ordering in hierarchical object placement—that commonly arise in purely LLM-generated plans. Similarly, [186] enhances LLM planners with a knowledge-graph-based retrieval-augmented generation (RAG) system for hierarchical plan generation. This method decomposes complex tasks into subtasks and atomic actions, and a symbolic validator ensures formal correctness while detecting failures by aligning expected and observed state. Building on this, [187] treats planning as an iterative conversation between neural and symbolic components rather than a one-shot translation, jointly refining a plan based on symbolic feedback from the environment. LOOP generates PDDL specifications and iteratively improves them, achieving substantially higher success rates on IPC benchmark domains by enabling neural and symbolic components to “talk” to each other throughout the entire planning process. A complementary perspective is provided by [188], which equips LLMs with a symbolic state space serving as an explicit world model; a policy model proposes actions that the symbolic environment deterministically verifies, and iterative correction or contrastive ranking of candidate plans enables more coherent, verifiable plan generation.

Causality-driven planning represents another important frontier for grounding multi-agent decisions in an explicit model of how actions and environment states influence future outcomes. [189] introduces a Structural Causal Action (SCA) model that learns a causal graph from agent trajectories, and then uses this structure to assign causal scores to LLM-generated proposals, reweighting or replacing them when they are causally inconsistent. This constrains planning to intervention-consistent behaviors, which reduces invalid actions and improves collaboration in AI-AI and human-AI settings. Taking a holistic approach, [190] introduces a causality planning framework that constructs an overarching task graph for global planning and a causality module for managing subtask dependencies, using inherent rules to perform causal intervention. This causality-aware decomposition enables multi-agent systems to handle complex, long-horizon cooperative tasks in open-world environments like Minecraft. For planning under unknown dynamics, [191] learns a symbolic transition model via inductive logic programming from interactions, which captures the logical rules of state transitions. By planning over the product of this transition model and an automaton derived from a Linear Temporal Logic (LTL) formula, the agent can resolve unknown causal dependencies and break a causally complex problem into simpler sub-tasks. At a higher level of abstraction, [192] learns symbolic state representations and causal processes for both endogenous actions and exogenous mechanisms, enabling a planning agent to generalize to held-out tasks involving more objects and goals than seen in training. This capability is especially important in embodied domains where the environment state changes concurrently with agent actions.

Dynamic world models that are updated as agents act are also critical for synchronized collective progress in partially observable environments. [193] presents a decentralized neuro-symbolic framework for cooperative multi-agent planning that employs a two-phase negotiation protocol. Agents first propose candidate roles with reasoning, then commit to a joint allocation; each agent generates and executes a symbolic plan independently without revealing detailed trajectories. Plans are grounded via a shared world model that encodes the current state, which avoids brittle step-level

alignment and enables higher-level operations that are both interpretable and reusable. A related design is proposed in [194], which introduces a hierarchical agent architecture in which an LLM plans high-level subgoals, and a learning mechanism continuously incorporates experiences into a persistent neurosymbolic belief state composed of textual inferences and code-based symbolic memory. This dynamic co-evolution of the world model with planning enables the agent to adapt to new observations and avoid the misalignment that plagues static internal models. Similarly, [195] proposes the use of “jumpy” models—predictive models of multi-step dynamics that capture state occupancies induced by pre-trained policies across multiple timescales. This approach composes pre-trained policies to solve tasks no constituent policy can solve alone, yielding substantial improvements in long-horizon manipulation and navigation tasks. These dynamic frameworks converge on a critical insight: the world model is not a static input to planning but a living artifact that must co-evolve with the agent’s accumulating experience.

Beyond these core mechanisms, several complementary strategies contribute to effective planning in multi-agent collaboration. [196] introduces “Action Chains”—sequences of actions explicitly bound to sub-goal intentions—as a planning primitive. This cyclical process of constructing intention-bound sequences, validating for conflicts, refining issues, and executing validated actions balances adaptability and efficiency by providing longer lookahead without incurring the cost of full re-planning. The concept of negotiation is formalized in [193] and also in [197], which emphasizes a social continual learning framework with a natural language oracle for question-asking, enabling agents to identify knowledge gaps and refine causal understanding. This interaction-driven planning is complemented by the strategy of adaptive compute allocation, which is central to [198]. This work shows through systematic compute-allocation analysis that the planner is the dominant factor influencing task performance, and that concentrating model capacity on planning—while using smaller, frozen components for execution—yields robust gains. This insight points to a generalizable design principle: prioritizing a capable planner with demonstrated efficacy in long-horizon tasks. Meanwhile, [199] introduces a framework for planning test-time compute in multi-agent systems under explicit budget constraints, using collaboration modules that are automatically induced from recurring interaction patterns and optimized via a dual-level planning architecture that performs both short-horizon action selection and long-horizon abstract lookahead. [200] tackles the long-context interference problem in long-horizon agentic tasks by training agents to organize execution around explicit subgoals while folding completed subgoal histories, reducing long-context interference and improving decision-making. Finally, the challenge of temporal reasoning is further addressed by [201], which formulates planning as a constraint programming problem over Allen’s Interval Logic, and by [202], a hierarchical framework for coordination under continual and uncertain temporal tasks in large-scale heterogeneous systems. These diverse methodologies collectively highlight that successful multi-agent collaboration requires not only the ability to decompose tasks and reason about causality, but also a dynamic, adaptive mechanism for resource allocation and world-model refinement that is tightly interwoven with the planning process itself. This adaptive capacity, in turn, establishes the foundation for the structured deliberation and collective reasoning mechanisms explored in the following section, where the focus shifts from how agents plan to how they reason together.

4.2 Reasoning Enhancement Through Debate, Consensus, and Critique Refinement

The pursuit of enhanced collective reasoning in LLM-based multi-agent systems builds directly upon the strategic interaction foundations established in the preceding section. While game-theoretic frameworks and Theory of Mind mechanisms provide the micro-foundations for how individual agents reason, model others, and make strategic choices, the techniques examined

here address a complementary question: how can multiple agents pool their individual reasoning capabilities to achieve collective outcomes that surpass what any single agent could accomplish alone? This subsection explores the structured deliberation, consensus-building, and critique-refinement mechanisms that define this domain, examining the cognitive underpinnings, methodological innovations, and critical trade-offs that shape their effectiveness.

Structured Deliberation and the Limits of Simple Aggregation

Early approaches to multi-agent reasoning often relied on unstructured debate or simple majority voting, operating under the implicit assumption that interaction alone would yield collective improvement. However, research has progressively revealed the limitations of such methods and, in doing so, has illuminated the need for the kind of principled strategic reasoning discussed in the previous section. For instance, it has been demonstrated that majority voting alone accounts for a significant portion of the performance gains attributed to multi-agent debate, with debate modeled as a stochastic process inducing a martingale over belief trajectories, implying no inherent improvement in expected correctness [203]. This finding challenges the assumption that mere interaction is beneficial and motivates the need for more structured deliberation. In response, frameworks like Deliberative Collective Intelligence (DCI) have been proposed, which move beyond unstructured exchange by specifying a phased process with four reasoning archetypes and 14 typed epistemic acts, enabling differentiated participants to engage in structured reasoning moves and preserve disagreements. DCI’s convergent flow algorithm guarantees termination and produces structured decision packets containing the selected option, residual objections, and minority reports, demonstrating the value of process accountability in consequential decisions [86]. Similarly, the Council of Hierarchical Agentic Language (CHAL) treats multi-agent debate as a structured belief optimization process, where each agent maintains a graph-structured belief schema and uses a gradient-informed dynamic mechanism for belief revision. By elevating meta-cognitive value systems to configurable hyperparameters, CHAL demonstrates that the adjudicator’s value system determines the debate’s trajectory, highlighting the importance of explicit structure over implicit convergence [204].

Cognitive Synergy and Hypothesis-Driven Evaluation

Beyond structural protocols, enhancing collective reasoning requires emulating human-like cognitive processes, particularly those that enable agents to model and anticipate one another’s reasoning—capabilities foregrounded in the previous section’s discussion of Theory of Mind. The concept of cognitive synergy suggests that the collective intelligence of a system can exceed the sum of its parts. This can be achieved by integrating adaptive Theory of Mind (ToM) and structured critical evaluation, where agents model others’ perspectives and systematically critique reasoning to identify logical gaps and mitigate biases [25]. Drawing inspiration from the Analysis of Competing Hypotheses (ACH) in cognitive science, the AgentCDM framework introduces a structured reasoning paradigm that shifts decision-making from passive answer selection to active hypothesis evaluation and construction. By systematically mitigating cognitive biases and internalizing this reasoning process through a two-stage training paradigm, AgentCDM demonstrates state-of-the-art performance, validating the effectiveness of hypothesis-driven approaches in collaborative decision-making [38]. This move toward active evaluation is further underscored by frameworks that emphasize process-centric debate, where the focus shifts from outcome-level voting to rigorous step-by-step logic critique to ensure process correctness, as seen in DynaDebate [205].

The Role of Disagreement and Strategic Routing

A critical insight in multi-agent reasoning is that disagreement can be a valuable signal, not merely a problem to be eliminated—a perspective that resonates with the game-theoretic treatment of strategic interaction in the preceding section, where divergent perceptions and beliefs are recognized as inherent features of multi-agent systems rather than anomalies. Research has shown that general,

partially overlapping disagreements can prevent premature consensus and expand the solution space, leading to improved success in collaborative tasks by encouraging complementary exploration. However, disagreements on task-critical steps can derail collaboration, with the impact depending on the topology of solution paths [206]. This has led to the proposal of disagreement-aware routing strategies. For instance, a knowledge-representation layer can abstract reasoning traces and decisions into symbolic disagreement states, distinguishing between convergent/divergent agreement and disagreement. These states support defeasible strategic routing rules, providing a bridge between sub-symbolic LLM deliberation and symbolic knowledge representation for strategic reasoning [83]. Furthermore, uncertainty-driven policy optimization has been proposed to mitigate “debate collapse,” a failure mode where agents converge on erroneous reasoning. By quantifying behavioral uncertainty at intra-agent, inter-agent, and system levels, and formulating an optimization to penalize self-contradiction and low-confidence outputs, this approach reliably calibrates the multi-agent system, improving decision accuracy while reducing system disagreement [207].

Trade-offs in Decision-Making Protocols: Dictatorial, Voting, and Consensus

The design of the final decision-making protocol is a pivotal choice in multi-agent reasoning, significantly impacting performance. Decision-making protocols exhibit a fundamental trade-off between dictatorial and voting-based approaches. Dictatorial strategies, where a single agent’s judgment prevails, are vulnerable to the cognitive biases of that individual, while voting-based methods may fail to fully harness collective intelligence [38]. Systematic evaluations of decision protocols reveal task-dependent performance variations. For instance, voting protocols have been shown to improve performance in reasoning tasks, while consensus protocols excel in knowledge-intensive domains, with judge-based protocols proving effective for logic-based tasks [208, 24]. The choice of protocol also has efficiency implications. In decentralized multi-agent settings, majority voting can cause inefficient collaboration due to strict acceptance criteria, and unanimous voting can severely degrade initial performance, leading to longer and more repetitive communication [209]. To address this, adaptive mechanisms like the Hierarchical Adaptive Consensus Network propose dynamic consensus policies based on task characterization and agent performance, achieving $O(n)$ communication complexity compared to $O(n^2)$ for fully connected systems [51]. Similarly, HCP-MAD leverages consensus as a dynamic signal, using heterogeneous pair-agent debates for lightweight verification and escalating to collective voting only for unresolved, complex tasks, thereby enhancing accuracy while substantially reducing token costs [210].

Critique-Refinement and Oversight

Finally, critique-refinement loops form the core of many reasoning enhancement techniques. This paradigm moves beyond debate toward collaborative truth-seeking. For instance, an automated pipeline inspired by human mediation directs models to collaboratively identify points of disagreement and examine the evidence for conflicting claims, converging toward consensus or isolating the “crux” of their disagreement. This approach significantly outperforms standard adversarial debate in helping non-expert models identify truth [211]. Furthermore, the integration of critique and revision has been shown to provide a richer training signal than majority voting alone. The Multi-Agent Consensus Alignment (MACA) approach uses reinforcement learning to post-train models on multi-agent debate traces, where preference learning over full reasoning traces—learning to differentiate between majority and minority reasoning—is more effective than binary consensus rewards, leading to improved individual accuracy and self-consistency [212]. The effectiveness of these mechanisms, however, is not guaranteed. Studies have shown that homogeneous teams without structured roles may not benefit from unguided peer exchange, and debate can sometimes be harmful, leading to decreases in accuracy due to sycophantic conformity and consensus collapse [213, 214]. This highlights that the benefits of critique-refinement are highly dependent on diversity, structure, and careful calibration, making it a powerful yet delicate instrument in the multi-agent reasoning toolkit.

Taken together, the structured deliberation, cognitive synergy mechanisms, disagreement-aware routing, decision protocol design, and critique-refinement loops examined in this subsection constitute the operational layer of collective reasoning in LLM-based multi-agent systems. These techniques translate the strategic reasoning capabilities—game-theoretic analysis, cognitive hierarchy modeling, and Theory of Mind—discussed in the preceding section into practical protocols for pooling diverse perspectives and synthesizing collective judgment. The findings reveal a consistent theme: the success of collective reasoning depends not on the mere presence of multiple agents, but on the careful design of interaction structures that preserve productive disagreement, mitigate cognitive biases, and adaptively allocate decision authority based on task demands and agent capabilities.

4.3 Game-Theoretic Decision-Making, Strategic Reasoning, and Theory of Mind

Game-theoretic modeling provides a rigorous mathematical foundation for understanding strategic interactions among LLM-based agents, complementing the emergent coordination phenomena discussed above with principled analyses of cooperation, competition, and negotiation. While the previous section examined how collective outcomes arise from interaction dynamics, network structures, and population composition, game theory offers a complementary lens: rather than asking how agents coordinate, it asks how rational (or boundedly rational) agents *should* and *do* make strategic choices when outcomes depend on the actions of others. This perspective is essential for understanding not only the behavioral regularities observed in multi-agent systems but also for designing mechanisms that promote desirable strategic outcomes. Classical game theory assumes rational agents with common knowledge of payoffs, but real-world multi-agent systems—particularly those composed of LLM agents—are characterized by uncertainty, misaligned perceptions, and bounded rationality. To address these limitations, hypergame theory extends the classical paradigm by explicitly modeling agents’ subjective perceptions of the strategic scenario—known as perceptual games—in which agents may hold divergent beliefs about the structure, payoffs, or available actions [215]. Hypergame rationalisability further provides a mechanism for finding belief structures that justify seemingly irrational outcomes, offering a declarative, logic-based domain-specific language for encoding hypergame structures and solution concepts via answer-set programming [216]. This formal treatment of misaligned perceptions is particularly relevant for LLM-based agents, whose internal representations of tasks and partners may diverge significantly from ground-truth scenarios—a phenomenon directly connected to the Theory of Mind considerations that will be examined in depth later in this section.

Cognitive hierarchy models from behavioral economics offer a complementary lens for analyzing strategic reasoning in LLMs, bridging the gap between normative game-theoretic solutions and empirically observed behavior. These models posit that agents have bounded rationality and behave at varying reasoning depths or levels, rather than assuming full rationality and equilibrium play [217]. The quantal cognitive hierarchy (QCH) model, for instance, assumes that agents can be both non-strategic (level-0) and strategic (level- k), with level- k agents best responding against lower-level opponents. However, the standard QCH model relies on a Poisson distribution assumption for the population distribution of reasoning levels, which may not hold empirically. Iterative population learning methods relax this assumption by estimating the empirical distribution of reasoning levels from data, yielding more accurate behavioral predictions in real-world games [217]. Building on these foundations, the Cognitive Hierarchy Benchmark (CHBench) evaluates LLMs’ strategic reasoning through a three-phase systematic framework, hypothesizing that different agents behave at varying reasoning depths and measuring models’ consistency across diverse opponents [218]. Results from CHBench indicate that LLMs exhibit consistent strategic reasoning levels across diverse opponents, confirming the framework’s robustness, while also revealing that chat mechanisms can degrade

strategic reasoning performance whereas memory mechanisms enhance it [218]—a finding that resonates with the emergent Theory of Mind behavior observed in agents with persistent memory discussed later in this section.

Behavioral game theory provides another important perspective, disentangling reasoning capability from contextual effects in LLM evaluations. Unlike traditional evaluations that emphasize Nash Equilibrium approximation, behavioral game theory frameworks assess the mechanisms driving strategic choices by jointly analyzing models’ revealed choices and reasoning traces [88, 219]. Studies testing 22 state-of-the-art LLMs find that while models like GPT-o3-mini, GPT-o1, and DeepSeek-R1 dominate most games, model scale alone does not determine performance [88]. Chain-of-Thought prompting is not universally effective for enhancing strategic reasoning—it benefits only models at certain levels while providing limited gains elsewhere [88]. Furthermore, encoded demographic features can bias decision-making patterns; for example, GPT-4o shows stronger strategic reasoning with female traits than male traits, while Gemma assigns higher reasoning levels to heterosexual identities compared to other sexual orientations [88]. Quantal response equilibrium (QRE) offers a theoretically grounded framework for placing model behavior on a continuous rationality scale calibrated against human data, with rationality parameters estimated in the range of $\lambda_{\text{human}} \in [220]$ [221]. Validation across 1,855 games with seven frontier models reveals that bluff rates converge to within 4% of equilibrium predictions, with substantial cross-model variation in rationality parameters, while robustness analyses expose high sensitivity to prompt framing and version instability in QRE rankings [221]. For non-strategic baseline behavior, ElementaryNet provides a provably incapable-of-strategic-behavior neural network for predicting human behavior in normal-form games, enabling the derivation of insights about human iterative reasoning and the value of rich level-0 specifications [222]. These behavioral game theory approaches connect naturally to the cognitive hierarchy models above, as both seek to characterize the bounded rationality that distinguishes real agents—whether human or LLM—from the idealized rational agents of classical game theory.

Strategic planning benchmarks systematically evaluate LLMs’ abilities in multi-agent decision-making, intent inference, and counterfactual reasoning across diverse game environments, providing empirical grounding for the theoretical frameworks discussed above. GameBench introduces a cross-domain benchmark covering nine game environments, each targeting at least one axis of key reasoning skill in strategy games, with evaluations revealing that none of the tested models match human performance and that GPT-4 can perform worse than random action in certain settings [223]. SPIN-Bench combines classical PDDL tasks, competitive board games, cooperative card games, and multi-agent negotiation scenarios in one unified framework, revealing that contemporary LLMs handle basic fact retrieval and short-range planning well but encounter significant bottlenecks in tasks requiring deep multi-hop reasoning over large state spaces and socially adept coordination under uncertainty [224]. WGSR-Bench leverages wargames as high-complexity strategic scenarios, integrating environmental uncertainty, adversarial dynamics, and non-unique strategic choices to systematically assess environmental situation awareness, opponent risk modeling, and policy generation [225]. TMGBench incorporates all 144 game types summarized by the Robinson-Goforth topology of 2x2 games, constructing classic games and synthesizing diverse story-based scenarios while organizing games into sequential, parallel, and nested structures for sustainable evaluation of increasingly powerful LLMs [226]. Evaluation results reveal that LLMs still have flaws in the accuracy and consistency of strategic reasoning processes, with varying mastery levels of Theory-of-Mind capabilities across models [226]. The Decrypto benchmark specifically targets multi-agent reasoning and Theory of Mind in a game-based setting, finding that LLM game-playing abilities lag behind humans and simple word-embedding baselines, and that state-of-the-art reasoning models can be significantly worse at classic cognitive science experiment variants than their older counterparts [227]. These benchmarks collectively reveal that while LLMs demonstrate partial strategic competence,

their performance is highly sensitive to task structure, opponent characteristics, and reasoning depth requirements—findings that underscore the importance of the Theory of Mind capabilities examined next.

Theory of Mind (ToM) constitutes a central capability for strategic interaction, enabling agents to reason about others’ beliefs, desires, and intentions, and thereby connecting the strategic reasoning frameworks above to the emergent coordination phenomena discussed in the previous section. Indeed, ToM provides the cognitive mechanism through which agents can anticipate partners’ actions, form adaptive strategies, and navigate the complex social landscapes where coordination and competition intermingle. Empirical studies demonstrate that ToM-equipped agents significantly outperform non-ToM agents in cooperative games, with teams of ToM agents outperforming non-ToM counterparts across all partner types—including non-ToM, ToM, and human players—and the benefit increasing with the number of ToM agents [228]. However, higher ToM abilities do not necessarily translate into better cooperative behavior; agents with higher ToM may exhibit worse cooperation than those with lower ToM, motivating the design of matching coalition mechanisms that leverage strengths of agents with different ToM levels by considering belief alignment and specialized abilities [229]. In LLM-based agents, ToM reasoning has been shown to enhance behavior alignment, decision-making consistency, and negotiation outcomes in ultimatum games, with reasoning models exhibiting limited capability compared to models with ToM reasoning, and different game roles benefiting from different orders of ToM reasoning [230]. From a computational perspective, ToM can be implemented within active inference frameworks, where agents maintain distinct representations of their own and others’ beliefs and goals, using sophisticated inference tree-based planning to systematically explore joint policy spaces through recursive reasoning [231]. ToM agents in this framework cooperate better than non-ToM counterparts in collision avoidance and foraging tasks by inferring others’ beliefs solely from observable behavior [231]. Causal models of ToM in conflict formalize when ToM engagement is warranted, specifying situational and agent-level conditions through three distinct causal pathways: tractability, reasoning-depth, and enabling-cause pathways [232].

The alignment of ToM orders between agents is critical for effective coordination, and this concern bridges directly to the generalization challenges identified in the preceding section regarding collaboration with unseen partners. Misaligned ToM orders—mismatches in the depth of ToM reasoning between agents—can lead to insufficient or excessive reasoning about others, impairing coordination [233]. Adaptive ToM (A-ToM) agents address this by estimating the partner’s likely ToM order based on prior interactions and using this estimation to predict the partner’s action, thereby facilitating behavioral coordination across repeated matrix games, grid navigation tasks, and Overcooked [233]. Higher-order ToM involves recursive reasoning about others’ beliefs, and benchmarks such as HI-TOM reveal a decline in LLM performance on higher-order ToM tasks, demonstrating limitations of current models in recursive mental state reasoning [234]. Decompose-ToM improves LLM performance on complex ToM tasks by recursively simulating user perspectives and decomposing tasks into subject identification, question-reframing, world model updation, and knowledge availability, showing significant improvements across models compared to baseline methods [235]. AutoToM provides an automated agent modeling method for scalable, robust, and interpretable mental inference, first proposing an initial agent model and then performing automated Bayesian inverse planning with iterative refinement guided by inference uncertainty, outperforming existing ToM methods and large reasoning models across five diverse benchmarks [236].

Theory of Mind is also instrumental in negotiation and persuasive dialogue, where strategic reasoning must be coupled with social influence and communication. The Theory of Mind Utility (ToM-U) formalizes epistemic state inference by constructing Local Epistemic World Models (LEWMs)—directed typed graphs representing agents, state nodes, and epistemic relationships—and evaluating candidate LEWMs against observed behavior, specifying a residue function that captures

the structured trace left by failed mentalizing attempts [237]. MIND (Multi-agent Inference for Negotiation Dialogue) introduces a Strategic Appraisal phase that infers opponent willingness from linguistic nuances with 90.2% accuracy, outperforming traditional Multi-Agent Debate frameworks in coordinating heterogeneous traveler preferences, achieving a 20.5% improvement in High-w Hit and a 30.7% increase in Debate Hit-Rate [238]. PersuasiveToM evaluates ToM abilities in persuasive dialogues through two core tasks: ToM Reasoning, which tests tracking of evolving desires, beliefs, and intentions, and ToM Application, which assesses the use of inferred mental states to predict and evaluate persuasion strategies [239]. Think Thrice Before You Speak (TTBYS) introduces a knowledge-enhanced stepwise reasoning framework that leverages both explicit and implicit prior experiences to improve LLMs’ inference of desires, beliefs, and persuasive strategies, with Qwen3-8B equipped with TTBYS outperforming GPT-5 by 1.20%, 22.80%, and 16.97% in predicting desires, beliefs, and persuasive strategies respectively [240]. NegotiationToM provides a benchmark for stress-testing machine ToM in real-world negotiation scenarios, building upon the Belief-Desire-Intention (BDI) agent modeling theory, and demonstrates that state-of-the-art LLMs consistently perform significantly worse than humans even with chain-of-thought prompting [241].

Belief resonance mechanisms, realized through hierarchical active inference, enable efficient coordination by letting the inferred mental states of one agent influence another agent’s predictive beliefs about its own goals and intentions [242]. This approach achieves team performance comparable to state-of-the-art methods while incurring much lower computational costs, and is especially beneficial in settings where agents have asymmetric knowledge about the environment [242]. Brain-inspired Theory of Collective Mind (ToCM) models extend ToM to complex multi-agent environments by predicting observations of all other agents into a unified but plural representation, constructing an imaginative space to simulate multi-agent interaction processes, and improving cooperative efficiency across various cooperative tasks with different numbers of agents [243]. The emergence of ToM-like behavior in LLM agents can also arise from interaction dynamics alone without explicit training or prompting—autonomous LLM agents playing extended sessions of Texas Hold’em poker progressively develop sophisticated opponent models when equipped with persistent memory, reaching ToM Level 3-5 while agents without memory remain at Level 0 [244]. This finding has significant implications for understanding artificial social intelligence and the role of memory in enabling recursive mental state modeling [244], and it echoes the earlier observation from CHBench that memory mechanisms enhance strategic reasoning. More broadly, the emergence of ToM through interaction alone connects this section’s game-theoretic and cognitive frameworks back to the emergent coordination themes of the previous section, illustrating how sophisticated strategic capabilities can arise from simple interaction dynamics when agents are equipped with the right cognitive infrastructure.

In summary, game-theoretic modeling, cognitive hierarchy theory, behavioral game theory, and Theory of Mind collectively provide a multi-layered understanding of strategic interaction in LLM-based multi-agent systems. The theoretical frameworks establish normative foundations and descriptive models of bounded rationality; the benchmarks provide systematic empirical evaluation of current model capabilities and limitations; and the ToM mechanisms offer both cognitive inspiration and practical algorithms for enhancing strategic reasoning. Together, these elements reveal a nuanced picture: LLMs demonstrate partial strategic competence that varies substantially across models, tasks, and opponent types, and their performance is shaped by factors as diverse as memory capacity, prompt framing, demographic encodings, and interaction history. This understanding of how individual agents reason strategically—and how they model others’ reasoning—provides the micro-foundations necessary for the collective reasoning enhancement techniques explored in the preceding section, where structured debate, consensus-building, and critique-refinement mechanisms build upon agents’ ability to model, anticipate, and influence one another’s reasoning processes.

4.4 Emergent Coordination Behaviors, Task Complexity, and Generalization Theory

The study of emergent coordination behaviors in LLM-based multi-agent systems addresses a foundational question: under what conditions does a population of interacting agents produce collective outcomes that transcend the capabilities of any individual member? This inquiry spans theoretical analyses of task complexity, empirical investigations of scaling dynamics, and formal frameworks for understanding collective agency and generalization. While the preceding discussion established the micro-foundations of individual strategic reasoning—how agents model others, reason recursively, and adapt their choices—this subsection shifts the analytical focus to the macro-level: how these individual capacities aggregate, interact, and sometimes fail to produce coherent collective behavior. This pivot from the cognitive to the collective is essential, as the strategic sophistication of individual agents does not guarantee effective coordination, and the emergent properties of agent societies often diverge sharply from the sum of their parts.

Emergent Coordination and Convention Formation. A central phenomenon in multi-agent systems is the spontaneous emergence of conventions—shared behavioral regularities that solve coordination problems without explicit design. Theoretical work on convention formation has traditionally assumed common knowledge and hyper-rational behavior, but computational models demonstrate that complex, large-scale conventions can emerge from model-free reinforcement learning mechanisms alone, suggesting that habit-like cognition rather than explicit reasoning can support societal-scale coordination [245]. This finding is corroborated by empirical studies of LLM populations, where decentralized groups of agents spontaneously develop universally adopted social conventions through natural language interaction, and where committed minority groups of adversarial agents can drive social change by imposing alternative conventions on the larger population [246]. The introduction of arbitrary common signals—such as dates or periodic numbers—can couple otherwise independent learning processes, enabling the emergence of temporal conventions that improve social welfare in common-pool resource dilemmas, even under decentralized learning with low observability [247]. These results suggest that convention formation is a robust property of interacting adaptive agents, whether biological, algorithmic, or hybrid, and that the ToM capabilities discussed in the previous subsection—while valuable for strategic reasoning—are not strictly necessary for achieving basic coordination; simpler mechanisms often suffice.

Scaling Behavior and Collective Intelligence. A critical question is whether collective intelligence emerges spontaneously from scale alone. Empirical evaluations of large-scale autonomous agent societies—including platforms hosting over two million agents—reveal a stark absence of spontaneous collective intelligence: societies fail to outperform individual frontier models on complex reasoning tasks, rarely synthesize distributed information, and often fail even trivial coordination tasks due to extremely sparse and shallow interaction [248]. This finding is reinforced by systematic studies of homogeneous multi-agent systems, which demonstrate that performance does not scale monotonically with agent count but follows a pattern of diminishing returns governed by a trade-off between collaborative synergy and coordination overhead—and that performance degradation stems from coordination overhead rather than merely long-context failure [28]. Quantitative analyses of coordination dynamics in LLM multi-agent systems further reveal three coupled laws: coordination follows heavy-tailed cascades, concentrates via preferential attachment into intellectual elites, and produces increasingly frequent extreme events as system size grows. These effects arise from an integration bottleneck, where coordination expansion scales with system size while consolidation does not [249]. Benchmarking frameworks for large-scale LLM populations confirm pronounced core-periphery structures and severe coordination overhead in decentralized task resolution [45]. These findings collectively challenge the assumption that more agents invariably lead to better performance,

highlighting the importance of strategic interaction design over mere plurality—a conclusion that resonates with the game-theoretic emphasis on mechanism design and payoff structures examined earlier.

Collective Agency and Causal Foundations. Formal frameworks have been developed to determine when a group of agents can be viewed as a unified collective agent. Adopting a behavioral perspective, collective agency is ascribed to a group when viewing the group’s joint actions as rational and goal-directed successfully predicts its behavior. Using causal games—causal models of strategic multi-agent interactions—and causal abstraction, researchers can make quantitative assessments of the degree of collective agency exhibited by different voting mechanisms and solve puzzles regarding multi-agent incentives in actor-critic models [250]. This framework provides a foundation for understanding, predicting, and controlling emergent collective agents, addressing a key safety challenge: the possibility that multiple simpler agents might inadvertently form a collective agent with capabilities and goals distinct from those of any individual. Information-theoretic approaches offer complementary tools for detecting higher-order structure in multi-agent systems, using partial information decomposition of time-delayed mutual information to test whether systems show signs of dynamical emergence, localize it, and distinguish spurious temporal coupling from performance-relevant cross-agent synergy. Experiments demonstrate that prompt design alone can steer multi-agent LLM systems from mere aggregates to higher-order collectives with identity-linked differentiation and goal-directed complementarity [251]. This notion of collective agency connects to the hypergame-theoretic perspective from the previous subsection, where misaligned perceptions among agents were shown to be crucial determinants of strategic outcomes; here, we see that the aggregation of such perceptions can yield emergent structures with their own causal efficacy.

Diversity, Heterogeneity, and System-Level Outcomes. The composition of agent populations—in terms of cognitive abilities, behavioral strategies, and value systems—exerts a profound influence on emergent coordination and collective outcomes. Behavioral diversity in multi-agent reinforcement learning has been shown to yield non-trivial benefits over homogeneous training paradigms: diversity measurement and control paradigms reveal the emergence of unbiased behavioral roles that improve team outcomes, synergy with morphological diversity, more effective cooperative solutions in sparse reward settings, and enhanced latent skill retention to overcome repeated disruptions [252]. However, the relationship between diversity and performance is nuanced and task-dependent. Systematic experiments with optimizing agents reveal that collectives of generalists perform better on tasks involving generating, choosing, and coordinating, while collectives of specialists with a few generalist mediators perform better on tasks involving negotiation; rationality bounds moderate these relationships, with specialists outperforming generalists at loose bounds and generalists outperforming specialists at tight bounds [253]. Cognitive heterogeneity in multi-stage supply chains—where agents with varying reasoning sophistication interact—reveals that myopic and self-interested behaviors exacerbate systemic inefficiencies, but information sharing effectively mitigates these adverse effects [254]. Value diversity in LLM communities enhances value stability, fosters emergent behaviors, and produces more creative principles, yet extreme heterogeneity induces instability, exhibiting diminishing returns [255]. The Principle of Maximum Heterogeneity formalizes these observations: any distributed production system optimizing for performance converges on an increasingly heterogeneous configuration, with environmental demands placing an upper bound on the degree of heterogeneity required [256]. The relevance of these findings to the cognitive hierarchy models discussed previously is striking: the distribution of reasoning levels within a population, which those models treat as an exogenous input, may itself be shaped by the interaction dynamics and diversity pressures described here.

Social Network Structures and Topological Effects. Network topology co-determines collective welfare in multi-agent systems. Simulations of the Iterated Prisoner’s Dilemma with

personality-driven LLM agents demonstrate that macro-level cooperative outcomes are not predictable from dyadic interactions alone; they are co-determined by network connectivity and the spatial distribution of personalities. Small-world networks are detrimental to cooperation, while strategically placing pro-social personalities in hub positions within scale-free networks significantly promotes cooperative behavior [257]. Evolutionary game selection models show that when the game itself can change over time following evolutionary principles, network architecture—particularly heterogeneity and clustered groups—can significantly amplify pro-social behavior [258]. The consensus-diversity tradeoff framework demonstrates that implicit consensus—where agents exchange information yet independently form decisions via in-context learning—can outperform explicit coordination mechanisms in dynamic environments requiring long-horizon adaptability, as partial deviation from group norms boosts exploration, robustness, and performance [84]. This observation echoes the behavioral game theory finding from the previous subsection that chain-of-thought prompting is not universally beneficial; in both cases, intermediate levels of reasoning or consensus yield optimal outcomes. Synchronization theory, adapted from the Kuramoto model, provides a rigorous mathematical foundation for understanding how coupling strength, agent diversity, and network topology impact emergent collective behavior in heterogeneous AI systems [259].

Generalization Across Tasks and Unseen Partners. The ability of multi-agent systems to generalize across tasks and adapt to novel collaborators is a hallmark of robust collective intelligence, and it directly extends the ToM-based partner modeling discussed in the previous subsection to the group level. Partner modeling—the capacity to infer the strengths and weaknesses of new partners—can emerge spontaneously in model-free agents under the right environmental conditions. In the Overcooked-AI environment, simple RNN agents trained to collaborate with a population of diverse partners develop structured internal representations of partners’ task abilities, enabling rapid adaptation and generalization to novel collaborators, provided the environment imposes social pressure that rewards partner modeling [260]. Capability-limited training approaches, such as ConventionPlay, extend cognitive hierarchies to include diverse populations of adaptive followers, enabling agents to probe partners’ repertoires and steer teams toward effective joint strategies in settings where conventions have differentiated payoffs [261]. Theory of Mind capacities—from first-order to higher-order reasoning—play a central role in intent inference and adaptive coordination, and the integration of dynamic Theory of Mind with critical evaluation mechanisms leads to more coherent, adaptive, and rigorous agent interactions where collective intelligence exceeds the sum of its parts [25]. However, generalization failures also abound: benchmark evaluations reveal a “collaboration gap” where models that perform well solo degrade substantially when required to collaborate with other agents, and heterogeneous pairings of independently developed agents can exhibit dramatic coordination breakdowns [262]. This asymmetry between within-population coordination and cross-population generalization underscores that collaboration-aware evaluation and training are essential for realizing the potential of multi-agent systems in real-world deployments where agents interact with unseen partners—a challenge that connects directly to the collective reasoning enhancement techniques explored in the following subsection, where structured mechanisms for inter-agent communication and deliberation are examined as potential remedies for the coordination failures identified here.

5 Optimization, Efficiency, and Resource Management

5.1 Automated Architecture Construction and Search for Cost-Effective MAS

The manual design of LLM-based multi-agent systems (LLM-MAS) is a labor-intensive process that requires significant domain expertise and iterative debugging, creating a critical bottleneck for scaling

these systems across diverse applications. As the complexity of tasks grows, the configuration space of possible agent roles, communication topologies, and interaction protocols expands combinatorially, making hand-crafted designs increasingly impractical. This challenge has motivated a substantial body of research focused on automated architecture construction and search, aiming to discover optimal system configurations that balance performance against computational costs. These methods draw inspiration from neural architecture search (NAS), which has long explored automated model design [263, 264, 265], and adapt these principles to the unique challenges of orchestrating multiple LLM-powered agents.

A central paradigm in automated MAS design is the shift from searching for a single static architecture to optimizing over a distribution of architectures, enabling query-dependent adaptation. This idea, rooted in the concept of supernetworks in NAS [266, 267, 268], is elegantly extended to multi-agent systems by MaAS, which introduces an agentic supernet—a probabilistic and continuous distribution over agentic architectures [269]. Rather than committing to a fixed topology, MaAS samples query-dependent agentic systems from this supernet, dynamically allocating inference resources such as LLM calls, tool invocations, and token budgets based on the difficulty and domain of each incoming query. This approach achieves substantial efficiency gains, requiring only 6% to 45% of the inference costs of existing handcrafted or automated systems while simultaneously improving performance by 0.54% to 11.82% across six benchmarks, and demonstrates strong transferability across datasets and different LLM backbones [269]. This paradigm represents a fundamental departure from monolithic system design, acknowledging that no single architecture is optimal for all queries and that resource allocation should be tailored to task-specific demands.

Complementing the supernet approach, other frameworks focus on the continuous refinement and evolution of system architecture through closed-loop optimization. AutoMaAS, for instance, presents a self-evolving multi-agent architecture search framework that leverages principles from NAS to automatically discover optimal agent configurations through dynamic operator lifecycle management [270]. Its key innovations include automatic operator generation, fusion, and elimination based on performance-cost analysis, dynamic cost-aware optimization with real-time parameter adjustment, and online feedback integration for continuous architecture refinement. This approach yields a 1.0% to 7.1% performance improvement while simultaneously reducing inference costs by 3% to 5%, demonstrating that automated search can discover architectures that are simultaneously more effective and more efficient than static hand-designed alternatives [270]. The concept of treating architectural components as evolvable entities with lifecycles—born, fused, and eliminated based on observed utility—provides a principled mechanism for navigating the large design space.

A fundamental design decision in automated MAS construction concerns the order in which system components are optimized. Many existing approaches adopt a topology-first strategy, first determining how agents should communicate and then selecting appropriate backbones. However, this ordering often leads to suboptimal cost-effectiveness when explicit resource budgets are imposed. AgentBalance challenges this convention by proposing a backbone-then-topology design philosophy [103]. The framework first performs backbone-oriented agent generation, constructing agents with heterogeneous backbones through LLM pool construction, pool selection, and role-backbone matching—recognizing that different roles may require different levels of model capability. Only after this backbone assignment does AgentBalance perform adaptive topology generation, guiding inter-agent communication via agent representation learning, gating mechanisms, and latency-aware topology synthesis. This ordering is motivated by the observation that the choice of backend model fundamentally constrains what an agent can accomplish, and thus should be prioritized when resources are limited. Experiments demonstrate that AgentBalance achieves up to 10% and 22% performance gains under matched token-cost and latency budgets, respectively, and exhibits strong performance-versus-budget curves across multiple benchmarks, while also functioning as a

plug-in enhancement for existing MAS frameworks [103]. This work highlights that the order of architectural decisions carries significant weight in determining final system efficiency.

The co-evolution of multiple architectural dimensions represents another important frontier. TacoMAS posits that effective test-time evolution of multi-agent systems requires jointly adapting both agent capabilities and communication topology, but on different timescales [271]. The framework formulates MAS inference as an online graph adaptation problem, where a fast capability loop rapidly updates agent expertise based on trajectory-level feedback to handle emerging subtasks, while a slow topology loop performs agents’ birth-death operations—including edge edits, agent additions, and agent removals—through a meta-LLM-driven process to preserve coordination stability. This fast-slow design is theoretically shown to drive MAS evolution toward a task-conditioned stable equilibrium, and empirically achieves an average improvement of 13.3% over nearly 20 strong baselines across four benchmarks [271]. The insight that capabilities and topologies evolve at different rates mirrors principles from organizational theory and provides a principled framework for dynamic system adaptation. Similarly, LAMaS introduces a latency-aware orchestration framework that explicitly optimizes the critical execution path in parallel multi-agent systems, enabling the controller to construct execution topology graphs with 38% to 46% lower critical path length compared to the state-of-the-art baseline, while maintaining or even improving task performance [272]. This work underscores that architectural search must account not only for performance and cost but also for latency, a dimension often overlooked in earlier approaches.

Beyond architectural search, a critical challenge is the retention of experiential knowledge across generations of system evolution. Many automated systems start from scratch on each new task or deployment, discarding valuable lessons learned from previous interactions. Skill-MAS addresses this by proposing a third path between inference-time MAS (which leverages frozen frontier LLMs but repeats identical searches without learning) and training-time MAS (which internalizes experience via gradient updates but is constrained by smaller model capabilities) [143]. The key insight is to decouple experience retention from parametric updates by conceptualizing high-level orchestration capability as an evolvable Meta-Skill—a form of architectural knowledge that captures how to compose and coordinate agents effectively. Skill-MAS refines this Meta-Skill through a closed optimization loop: multi-trajectory rollout samples behavioral distributions for each task under the current Meta-Skill, while selective reflection adaptively selects priority tasks and applies hierarchical contrastive analysis to distill systemic experience into generalizable, strategy-level principles. This approach achieves remarkable performance gains across four complex benchmarks while maintaining favorable cost-performance trade-offs, with evolved Meta-Skills demonstrating high robustness and strong transferability across unseen tasks and different LLMs [143]. This represents a significant advance in making automated architecture search cumulative rather than episodic.

The evolution of agent capabilities and architectures can also be viewed through the lens of population-based optimization. HMAE reconceptualizes heuristic search for combinatorial optimization problems as an organizational design problem, decomposing each evolutionary generation into an autonomous, role-specialized loop with four coordinated agents: a Proposer for strategy exploration, a Generator for executable heuristic synthesis, an Evaluator for empirical assessment, and a Reflector for archive-backed memory update [273]. By coupling behavior-aware retrieval, lightweight candidate filtering, and fitness-grounded archive updates, HMAE guides the search toward diverse and promising heuristic behaviors while avoiding redundant evaluations, and achieves the lowest average gaps on TSP (0.464%) and Online BPP (0.223%) with substantially fewer token requirements than competing baselines [273]. Similarly, TPGO introduces a Textual Parameter Graph optimization framework that models the MAS as a graph where agents, tools, and workflows are modular, optimizable nodes, deriving textual gradients—structured natural language feedback from execution traces—to pinpoint failures and suggest granular modifications, while employing

Group Relative Agent Optimization for meta-learning from historical optimization experiences [274]. This structural awareness and experience-driven optimization enable significant enhancements to state-of-the-art agent frameworks on complex benchmarks like GAIA and MCP-Universe [274].

Optimization objectives in automated architecture search are naturally multi-faceted, and frameworks have been developed to address the many-objective problem of balancing effectiveness and efficiency. AgentSwift formalizes a hierarchical search space that jointly models agentic workflows and composable functional components, moving beyond workflows alone to co-optimize functional components, and employs a value model trained on high-quality data generated via a strategy combining combinatorial coverage and balanced Bayesian sampling for low-cost evaluation, guided by a hierarchical MCTS strategy informed by uncertainty [275]. Similarly, EvoRoute addresses the “Agent System Trilemma”—the tension among performance, monetary cost, and latency—through a self-evolving model routing paradigm that dynamically selects Pareto-optimal LLM backbones at each step while continually refining its own selection policy through environment feedback, reducing execution cost by up to 80% and latency by over 70% on challenging agentic benchmarks such as GAIA and BrowseComp+ [276]. MasHost employs reinforcement learning to jointly sample agent roles and interactions through a unified probabilistic sampling mechanism, introducing component rationality as a design principle and proposing Hierarchical Relative Policy Optimization to integrate group-relative advantages and action-wise rewards [277]. Finally, Agent-UCT proposes a tree search algorithm that extends upper confidence bounds with a reuse-aware regularization term derived from a bipartite prefix reuse graph, enabling cost-aware and compositionally efficient agentic workflow optimization [278]. Collectively, these approaches illustrate the breadth of automated architecture construction strategies, from supernet sampling and hierarchical co-evolution to experience-driven meta-skill formation and multi-objective optimization, each contributing unique insights into how to systematically and efficiently discover high-performing, cost-effective multi-agent architectures. The field is converging on the recognition that automated search, guided by explicit resource constraints and enriched by experiential knowledge, is not merely a convenience but a necessity for realizing the full potential of LLM-based multi-agent systems at scale. Importantly, these automated search methods also intersect with the dynamic routing paradigms discussed in the following subsection, as both seek to allocate computational resources adaptively—yet while routing approaches optimize over fixed architectures at inference time, the methods surveyed here aim to discover the architectures themselves, often yielding routing policies as a byproduct of the search process.

5.2 Routing, Model Selection, and Budget-Aware Collaboration Strategies

The escalating costs and latency associated with deploying large language models (LLMs) in multi-agent systems (MAS) have made efficient resource allocation a critical research focus. Static configurations that uniformly assign powerful models to all agents and queries are often wasteful, as many tasks do not require frontier-level capabilities. Consequently, a significant body of research has emerged around dynamic routing and model selection strategies that aim to balance performance, cost, and latency. This subsection examines the key paradigms within this domain, including unified routing frameworks, heterogeneous collaboration strategies, and interpretable routing methods, as well as the theoretical underpinnings that formalize these challenges [279]. The methods surveyed here operate at the architectural level, assuming that a multi-agent system—whether manually designed or automatically discovered—is already in place, and focus on optimizing the allocation of queries and models within that fixed or dynamically adapted structure. In doing so, they complement the automated architecture search methods of the previous subsection, which discover system topologies, by providing the inference-time decision mechanisms that determine which available models and agents are actually invoked per query. At the same time, these routing

decisions serve as the critical input to the infrastructure-aware scheduling and serving systems of the following subsection, which execute the selected configurations efficiently on shared GPU clusters. Understanding routing as an intermediate layer between architecture construction and infrastructure execution clarifies its dual role: it is both the output of architectural search (which choices are available) and the input to serving systems (which choices are made).

A central challenge is that existing routing methods often optimize for single-agent scenarios, overlooking the unique structural decisions in MAS, such as which agents to involve and how they should collaborate. To address this, unified routing frameworks have been proposed to jointly optimize all components of a multi-agent system. For instance, the problem of Multi-Agent System Routing (MASR) integrates collaboration mode determination, role allocation, and LLM backbone selection into a single, cohesive decision-making process. A cascaded controller network can progressively construct an agent system tailored to a specific query, balancing effectiveness and efficiency, with methods like MasRouter demonstrating high performance and cost-effectiveness by dynamically selecting both the system architecture and the models for each role [280]. Similarly, the MoMA (Mixture of Models and Agents) framework provides a generalized routing solution that integrates both model and agent selection. It profiles the capabilities of various LLMs to create a training dataset, enabling a router to intelligently assign queries to the most cost-effective model or a specialized agent, thereby achieving a superior balance between efficiency and cost for diverse and complex queries [281]. Further pushing the boundaries of routing granularity, some works model the multi-agent system as a directed graph where edges represent distinct collaboration strategies. This allows different agent pairs to interact via tailored communication patterns; the SC-MAS framework introduces edge-level heterogeneous collaboration, which proves more flexible than homogeneous collaboration modes, leading to improved accuracy and reduced inference costs [102]. Extending this concept, HieraMAS introduces a hierarchical framework where each “supernode” (or agent role) is implemented by a mixture of heterogeneous LLMs, rather than a single model, and uses a two-stage algorithm to optimize both the intra-node LLM mixture and the inter-node communication topology, addressing the credit-assignment challenges that arise in such complex configurations [52]. These unified approaches are particularly relevant when the optimal architecture is not known a priori—for example, when the architectural search methods of the previous subsection have identified a distribution of plausible topologies—and the router must select among them on a per-query basis.

Beyond architecture selection, the problem of adapting to a stream of heterogeneous queries is central to cost-efficient operation. This is often framed as a cost-performance optimization problem, where the goal is to route each query to the most appropriate model [282]. An effective approach involves estimating the difficulty or complexity of an incoming query to inform routing decisions. Difficulty-Aware Agentic Orchestration (DAAO) uses a variational autoencoder to predict query difficulty, which then guides a modular operator allocator and a cost- and performance-aware LLM router. This self-adjusting policy allows the system to select simpler workflows for easy queries and more complex strategies for harder ones, leading to improvements in both accuracy and inference efficiency [283]. Similarly, CASTER employs a Dual-Signal Router that combines semantic embeddings with structural meta-features to estimate task difficulty, and uses a “Cold Start to Iterative Evolution” paradigm where the router learns from its own routing failures. This strategy has been shown to dramatically reduce inference costs in graph-based MAS while maintaining performance [284]. The query-difficulty signal produced by these methods is also directly consumable by the infrastructure-aware schedulers of the following subsection, which can use difficulty estimates to inform resource provisioning and priority decisions, thereby closing the loop between architectural routing and execution-level optimization.

To achieve more granular control over the trade-off between cost and performance, routing can be formulated as a constrained optimization problem. This perspective moves beyond per-query

greedy decisions to consider global budgets and capacity constraints. For example, batch-level routing frameworks jointly optimize model assignments for a group of queries while respecting cost and GPU capacity limits, and robust variants account for uncertainty in predicted model performance [285]. OmniRouter similarly casts routing as a constrained optimization problem, using a hybrid retrieval-augmented predictor to forecast LLM capabilities and costs, and a constrained optimizer with Lagrangian dual decomposition to find globally optimal query-model allocations that minimize cost while meeting performance thresholds [286]. In agentic settings, where routing decisions have path-dependent consequences and feedback is delayed, other frameworks have been developed. TRACE-Router, for example, aligns routing with the “unit of supervision” by assigning a task to a single model for its entire execution and updating its policy based on the terminal reward of the task, effectively learning routing policies that adapt to the workload without requiring explicit task-complexity estimation [287]. This problem is also tackled from a different angle by Budget-Aware Agentic Routing, which proposes a boundary-guided training method. This approach leverages “always-small” and “always-large” boundary policies to build a difficulty taxonomy and anchor learning under sparse rewards, effectively optimizing the cost-success frontier for tasks with strict per-task spending limits [288]. These constrained formulations produce routing decisions that are particularly well-suited for the infrastructure-aware execution layer of the following subsection, as they directly output resource budgets per query—information that scheduling systems can leverage to pre-allocate GPU memory, plan for KV cache usage, and coordinate CPU-GPU concurrency.

While many routing mechanisms rely on complex models and continuous learning, significant effort has also been dedicated to creating efficient, interpretable, and adaptive routing policies. Some approaches move away from expensive LLM-based selectors and static policies altogether. For instance, AMRO-S frames MAS routing as a semantic-conditioned path selection problem, modeled and solved using ant colony optimization. It uses a supervised fine-tuned small language model for low-overhead intent inference and a quality-gated asynchronous update mechanism to decouple inference from learning, providing traceable routing evidence and robust performance under high concurrency [289]. Other work focuses on continual learning and adaptation to changing model pools and user preferences. MixLLM develops a dynamic contextual-bandit-based routing system that leverages query tags to enhance embeddings, predicts model responses and costs, and includes a meta-decision maker to balance quality, cost, and latency [290]. Similarly, the concept of meta-learning has been introduced to efficiently learn user preferences for cost-performance optimization. MetaRouter formulates preference profiles as tasks in a contextual bandit setting, allowing the router to quickly adapt to the implicit, heterogeneous preferences of different users with very little interaction [291]. The interpretability of these routing policies is not merely a convenience; it provides the audit trails that infrastructure schedulers require for debugging, capacity planning, and avoiding cascade failures in production environments.

Finally, the practical deployment of routing in distributed and resource-constrained environments has spurred research into infrastructure-aware orchestration. These systems recognize that the state of the serving infrastructure is a critical input for routing decisions. INFRAMIND, for instance, makes the multi-agent stack infrastructure-aware by conditioning topology and role selection on real-time system load and remaining budget, and uses a budget-aware scheduler to reorder queues [292]. On edge clusters with heterogeneous devices, LMEdge formulates orchestration as a binary integer linear programming (BILP) problem to jointly determine model configuration and execution placement, using lightweight ML models to predict latency and accuracy, and a heuristic to approximate the solution for scalable scheduling [293]. This infrastructure awareness is crucial for ensuring SLO compliance and efficient resource utilization, especially under dynamic and concurrent workloads, and it directly bridges the routing methods discussed here with the serving and scheduling systems of the following subsection. In this sense, routing is not merely a decision

layer above fixed architectures, but a continuous feedback mechanism that ties the structural choices of automated architecture search to the dynamic realities of heterogeneous GPU clusters, enabling the full stack from architecture discovery to infrastructure execution to operate in a synchronized, cost-efficient, and performance-optimal manner.

5.3 Infrastructure-Aware Scheduling, Serving, and Resource Management

Deploying LLM-based multi-agent systems (LLM-MAS) at scale in production environments introduces a critical layer of complexity that extends beyond algorithmic design: the underlying serving infrastructure. While the previous subsection examined how routing strategies determine *which* models and agents to invoke for a given query, this subsection addresses the equally critical question of *how* those selected components are efficiently executed, scheduled, and managed on shared, heterogeneous GPU clusters. Unlike single-turn LLM inference, each user query in an agentic workflow triggers an iterative, multi-turn pipeline of LLM calls interleaved with tool executions, which dramatically amplifies resource consumption and introduces non-deterministic, input-dependent costs [294]. Consequently, infrastructure-level scheduling, serving, and resource management have emerged as first-order concerns for achieving cost-effective, low-latency, and reliable LLM-MAS deployment, complementing the model-selection strategies discussed earlier with execution-level optimizations.

A central challenge in this domain is the inherent mismatch between the resource demands of agentic workloads and the capabilities of traditional serving systems designed for monolithic, single-model inference. Agentic workloads exhibit a temporal shift from single-turn inference to multi-turn LLM-tool loops, and a spatial shift from chat-scale, GPU-only execution to repository-scale, GPU-CPU co-located execution [295]. Recognizing this, recent systems have proposed workload-aware cross-cluster scheduling frameworks that explicitly leverage agent semantics and roles. For instance, Maestro predicts the output length and memory usage of each agentic stage and uses these predictions to drive a hierarchical scheduler: at the node level, it enables dynamic multi-model co-location via hierarchical weight caching and elastic memory provisioning; at the cluster level, it performs latency-aware routing to avoid cold-start delays; and at the global level, it enforces workflow-aware prioritization to minimize head-of-line blocking for interactive tasks [294]. This infrastructure-aware approach has been shown to reduce KV-reservation HBM by 67.2% and improve high-contention SLO attainment by 23.6 percentage points over traditional earliest-deadline-first scheduling [294].

Beyond workload prediction, a growing body of work emphasizes the importance of conditioning orchestration decisions on real-time infrastructure state. Traditional multi-agent orchestration methods select models and topologies based solely on task and model features, ignoring the runtime state of the serving infrastructure—a “blindness” that causes systematic resource underutilization, where preferred models accumulate deep request queues while equally capable alternatives sit idle [292]. To address this, INFRAMIND introduces an infrastructure-aware orchestration framework that conditions topology and role selection on real-time system load and remaining budget, biasing toward simpler graphs under congestion and richer ones at low load. An infrastructure-aware executor then observes per-model queue depths, cache utilization, and response latencies at each agent step to decide which model to call and how deeply to reason, while a budget-aware scheduler reorders each model’s queue so that urgent requests are served first [292]. Cast as a hierarchical constrained Markov Decision Process and solved end-to-end via reinforcement learning, this framework delivers up to +7.6 percentage points accuracy improvement over prior baselines at low load with up to $7\times$ lower latency, and sustains up to 99.9% SLO compliance under high load where all baselines drop below 50% [292]. This paradigm shift from static, model-centric orchestration to dynamic,

infrastructure-aware orchestration represents a fundamental advancement in LLM-MAS serving, directly building upon the routing decisions of the previous subsection by making them contingent on live system state.

The heterogeneous nature of modern GPU clusters and agentic workloads further complicates scheduling. Agentic systems compose heterogeneous tool workloads on shared GPU/CPU infrastructure, yet existing frameworks often assign all GPU-capable tools to the GPU by default. However, profiling of 19 AI tools across GPU and CPU reveals that 11 are GPU-preferred, 4 are ambiguous, and 3 are device-neutral, establishing that blanket GPU-first scheduling is suboptimal [296]. Co-scheduling frameworks for heterogeneous agentic workloads therefore require holistic visibility across both GPU inference and CPU tool execution. MARS, for instance, establishes a unified information stream across GPU and CPU resources, and uses an external control plane to decouple admission from execution to prevent heterogeneous resource oversubscription, while an internal agent-centric scheduler minimizes the end-to-end critical path by prioritizing latency-sensitive continuations and adaptively retaining KV cache state only when warm resumption yields a latency benefit [295]. This approach reduces end-to-end latency by up to $5.94\times$ while maintaining nearly maximal system throughput [295]. Similarly, infrastructure-aware scheduling must account for the CPU-centric bottlenecks introduced by agentic AI serving, which heavily relies on CPU-orchestrated external tools; CPU-aware overlapped micro-batching (COMB) and mixed agentic scheduling (MAS) have been proposed to optimize CPU-GPU concurrent utilization while reducing skewed resource allocation for heterogeneous execution [297].

Infrastructure-level serving optimizations also extend to the operator and kernel granularity. Traditional model-level autoscaling treats the entire model as a monolith, leading to degraded performance or significant resource underutilization due to poor adaptability to dynamic inference traffic. Through detailed characterization, researchers have found that generative models are executed as graphs of interconnected operators with heterogeneous compute and memory footprints, suggesting that the operator—rather than the entire model—is the right granularity for scaling decisions [298]. Operator-level autoscaling frameworks allocate resources at this finer granularity, optimizing scaling, batching, and placement based on individual operator profiles, preserving SLOs with up to 40% fewer GPUs and 35% less energy, or achieving $1.6\times$ higher throughput under fixed resources [298]. Fused model routing and load balancing further optimize heterogeneous LLM serving by jointly trading off quality, latency, and cost in a single online assignment over concrete model instances, rather than treating routing and load balancing as isolated layers [299].

Finally, latency-aware orchestration must be coupled with critical-path-aware credit assignment to ensure that scheduling decisions optimize for the end-to-end workflow, not individual calls. Agentic workflows exhibit unpredictable execution times—execution may branch, fan-out, or recur in data-dependent ways—and since LLMs in workflows often outnumber available GPUs, execution leads to GPU oversubscription [300]. Systems like Scepsy exploit the insight that while agentic workflows have unpredictable end-to-end latencies, the shares of each LLM’s total execution times are comparatively stable across executions, enabling the construction of Aggregate LLM Pipelines as lightweight latency/throughput predictors for efficient GPU allocation [300]. Similarly, workflow-aware schedulers model each request as an online-revealed DAG, maintain running estimates of the workflow’s completion horizon, prioritize ready calls by projected risk of missing that horizon, and jointly select prefill placement, decode placement, and local queue priority while accounting for KV-cache capacity and cross-stage transfer latency [301]. These workflow-atomic scheduling approaches, which treat the entire agent workflow as the first-class schedulable unit rather than individual inference calls, have been shown to reduce task completion time by $1.64\times$ over request-level schedulers while improving GPU memory utilization and SLO attainment [302]. Collectively, these infrastructure-aware scheduling, serving, and resource management techniques constitute

an essential layer of the LLM-MAS stack, bridging the model-routing decisions of the previous subsection with the KV cache management and parallel execution optimizations of the following subsection, and thereby enabling scalable, efficient, and reliable deployment of multi-agent systems in production environments.

5.4 KV-Cache Management, Parallel Execution, and Performance-Cost Modeling

The efficient deployment of LLM-based Multi-Agent Systems (LLM-MAS) hinges critically on the underlying inference infrastructure, where Key-Value (KV) cache management has emerged as a first-order optimization target [303, 304]. While the preceding subsection addressed how infrastructure-aware scheduling and orchestration determine *which* models to invoke and *when* on shared GPU clusters, this subsection focuses on the complementary question of *how* inference efficiency is maximized once those scheduling decisions are made. In multi-agent workloads, the KV cache footprint is exacerbated by long shared contexts, frequent multi-turn interactions, and the interleaving of LLM calls with tool executions and agent-to-agent communication. This subsection examines the dual pillars of inference efficiency in LLM-MAS: advanced KV cache management across token-level, model-level, and system-level abstractions, and the orchestration of parallel execution alongside analytical performance-cost modeling. Together, these techniques form the execution-level backbone that makes infrastructure-aware scheduling decisions practically realizable at scale.

KV-Cache Management Strategies. The KV cache, while eliminating redundant computation in autoregressive generation, grows linearly with sequence length and can quickly exhaust GPU memory, particularly under the long-context demands of agentic pipelines [305, 304]. Prior work has categorized KV cache management into token-level, model-level, and system-level optimizations [303]. Token-level strategies focus on selective retention and eviction. For instance, CORM leverages the observation that adjacent tokens’ query vectors exhibit high similarity, allowing the dynamic retention of only the most important key-value pairs without model fine-tuning, achieving up to 70% memory reduction [306]. Similarly, approaches like K-VEC explicitly prioritize token coverage to mitigate the performance degradation associated with reduced mutual information between inputs and outputs under aggressive eviction [307]. More recent work, such as KV Admission, formalizes the problem as a causal system of admission, selection, and eviction, using a lightweight Write-Gated mechanism to predict token utility *before* cache entry, thereby preventing low-value states from consuming memory in the first place [308]. Likewise, TRIM-KV learns intrinsic token retention scores at creation time via a lightweight gate, with scores decaying over time to reflect long-term utility, effectively suppressing noise from uninformative tokens [309]. At the model level, techniques such as MiniCache exploit the observation that KV states exhibit high similarity across adjacent layers, applying depth-wise compression that disentangles magnitude and direction to achieve superior compression ratios [310]. Task-aware methods, such as Task-KV, allocate differentiated KV cache budgets per attention head based on semantic differentiation, preserving comprehensive information in heterogeneous heads while compressing aggregating heads [311]. Similarly, DynamicKV adaptively adjusts token retention per layer based on task-specific activation patterns, while ZigZagkv leverages layer uncertainty to determine per-layer budget sizes [312, 313]. At the system level, multi-tier memory hierarchies extend effective cache capacity beyond GPU HBM. Predictive multi-tier memory management systems unify KV cache sizing across attention architectures and employ Bayesian reuse predictors to achieve high cache hit rates across a six-tier memory hierarchy [314]. KVDrive jointly orchestrates cache placement, pipeline scheduling, and cross-tier coordination across GPU memory, host DRAM, and SSD, achieving up to 1.74x higher throughput than state-of-the-art works [315]. Offloading strategies, such as KVPR, mitigate PCIe bandwidth bottlenecks by overlapping GPU

recomputation with KV cache transfer from CPU memory, pushing the GPU to start generating from partial activations while the remainder is concurrently streamed [316]. ScoutAttention further accelerates offloaded inference through GPU-CPU collaborative block-wise sparse attention, featuring a layer-ahead CPU pre-computation algorithm that initiates attention computation one layer in advance [317].

KV Cache Management for Multi-Agent Workloads. While the general strategies above apply broadly to LLM inference, the unique characteristics of LLM-MAS—including multi-turn interactions, tool calls, and shared context—demand specialized cache management, particularly given the temporal dynamics of agentic workflows discussed in the previous subsection. Continuum introduces a time-to-live (TTL) mechanism for KV cache retention that selectively pins caches in GPU memory during tool calls, balancing the cost of reloading or recomputation against potential queueing delays, improving average job completion times by over 8x for real-world agents [318]. TokenCake, a KV-cache-centric serving framework, co-optimizes scheduling and memory management through an agent-aware design, employing a temporal scheduler that proactively offloads idle caches during function calls and a spatial scheduler with dynamic memory partitioning guided by graph structure and runtime state [319]. KVFlow abstracts agent execution schedules as an Agent Step Graph, assigning each agent a steps-to-execution value that guides fine-grained eviction policy at the KV node level, thereby anticipating future usage patterns and preserving entries likely to be reused, achieving up to 2.19x speedup over SGLang [320]. A particularly promising direction is the establishment of KV cache as a first-class distributed systems object. AAFLOW+ introduces a stateful extension of agentic workflow operators that treats KV cache as a distributed object, enabling zero-copy, transfer-aware execution where agents reuse long contexts without recomputation, reducing TTFT by up to 50.2x at 16-agent scale [321]. This approach demonstrates that KV-state sharing via transmission can significantly outperform text-passing and recomputation on networks with moderate to high bandwidth. Notably, these agent-specific KV management techniques directly complement the infrastructure-aware scheduling frameworks of the previous subsection: while those schedulers determine *when* and *where* to invoke agents, KV-centric mechanisms ensure that the necessary context is readily available at the chosen execution site, minimizing recomputation and transfer overhead.

Parallel Execution and Performance-Cost Modeling. Beyond cache management, the execution of LLM-MAS introduces significant scheduling and resource allocation challenges that bridge cache management with the workflow-atomic scheduling discussed earlier. Given the dynamic nature of agentic workloads, the management of KV cache must be tightly integrated with scheduling decisions. Online scheduling algorithms that jointly optimize batching and KV cache reservation under output token length uncertainty have been formalized, with Wasserstein distributionally robust optimization frameworks enabling adaptive reservation quantiles that balance preemption risk against memory efficiency [322]. Furthermore, the execution of multi-agent workloads can be viewed as a graph of dependent LLM calls and tool executions. KV caching strategies must therefore be integrated with the scheduler to minimize the critical path, echoing the workflow-atomic perspective introduced in the previous subsection. The concept of treating KV cache restoration as a multi-dimensional parallel execution problem is explored in CacheFlow, which introduces a unified 3D parallelism abstraction across tokens, layers, and GPUs, enabling fine-grained overlap of recomputation and I/O to reduce TTFT by 10%-62% [323]. Similarly, Compute-Or-Load (Cake) systems dynamically balance KV cache computation and loading, optimally utilizing both computational and I/O resources in parallel to reduce TTFT by 2.6x on average [324]. For modeling the performance-cost trade-offs of these systems, analytical frameworks that derive critical ratios between cached and prefill tokens help identify when workloads become memory-bound versus compute-bound, guiding architectural decisions [325]. However, such analyses highlight that under

extreme offloading, storage I/O can become the dominant bottleneck, with storage NICs on prefill engines becoming bandwidth-saturated while decoding engines remain idle [326]. DualPath addresses this imbalance through a dual-path KV cache loading strategy that routes caches through the compute network via RDMA, improving throughput by up to 1.96x in online serving [326]. Overall, the convergence of KV cache management with execution-aware scheduling and infrastructure-aware orchestration is essential for achieving cost-effective, low-latency inference in scalable LLM-MAS deployments. These cache-centric optimizations do not operate in isolation; rather, they form the memory-management substrate that enables the scheduling decisions of the preceding subsection to translate into tangible performance gains, and they set the stage for the parallel execution strategies and system-level optimizations that follow.

6 Evaluation, Benchmarking, and Failure Diagnosis

6.1 Evaluation Metrics, Taxonomies, and Process-Oriented Assessment Frameworks

The evaluation of LLM-based Multi-Agent Systems (LLM-MAS) has undergone a significant paradigm shift, moving from a narrow focus on final outcomes toward process-oriented, architecture-aware, and diagnostic assessment frameworks. Traditional metrics, such as end-to-end accuracy or task success rates, are increasingly recognized as insufficient for capturing the nuanced failure modes that plague complex, multi-step agentic workflows. This shift toward process-level scrutiny establishes the diagnostic foundation upon which the subsequent discussion of standardized benchmarks, scalability analysis, and comparative studies is built. This section synthesizes the emerging landscape of evaluation methodologies, focusing on diagnostic scoring systems, comprehensive failure taxonomies, and automated frameworks for process-level verification.

A central theme in modern evaluation is the transition from outcome-centric to process-centric metrics. End-to-end evaluations often obscure intermediate hallucinations and reasoning errors that accumulate throughout an agent’s trajectory, a problem particularly acute in long-horizon tasks like deep research [327]. To address this, novel diagnostic scoring systems have been proposed. For instance, the Contextualized Role Adherence Score (CRAS) decomposes an agent’s adherence to its designated role into four measurable dimensions, capturing query-wise, context-aware compliance that macro-level metrics like pass@k obscure [328]. Similarly, the Decision Quality (DQ) metric was introduced to capture properties essential for operational deployment—such as validity, specificity, and correctness—which are not addressed by existing LLM evaluation metrics, proving critical in demonstrating the deterministic quality advantage of multi-agent orchestration for incident response [329]. Beyond task-specific scores, broader frameworks like the Composite Reliability Score (CRS) integrate calibration, robustness, and uncertainty quantification into a single metric to holistically assess LLM reliability beyond isolated hallucination checks [330].

Complementing these continuous scoring systems are structured, trace-based verification frameworks that provide auditable and re-executable evidence. Frameworks like AgentEval formalize agent executions as evaluation directed acyclic graphs (DAGs), where each node carries typed quality metrics assessed by a calibrated LLM judge, enabling detailed root cause attribution. Its ablation study demonstrated that DAG-based dependency modeling alone contributed a substantial +22 percentage points to failure detection recall and +34 points to root cause accuracy over flat step-level evaluation [331]. Similarly, the concept of deterministic “integrity gates” has been proposed, where every stage transition in a workflow is gated with a halt-on-failure mechanism, resolving integrity questions with the cheapest sufficient deterministic check to create an auditable trail [332]. Frameworks such as Pramana emphasize the need for a protocol-layer treatment to produce verification artifacts—records that an auditor can re-execute offline—to ensure that claims are

verifiable against their sources [333].

The development of detailed failure taxonomies is another cornerstone of process-oriented evaluation. These taxonomies move beyond simple success/fail labels to categorize the specific nature of errors, directly enabling the failure attribution and diagnostic capabilities that will be examined in the benchmarking literature that follows. For instance, the PING Taxonomy classifies hallucinations in Deep Research Agents into Propagation, Intent, Noise-induced, and Grounding types [327]. Other work has derived a cross-domain failure taxonomy from grounded theory, annotating failed trajectories to create benchmarks for failure attribution and localization [334]. Comprehensive audits of medical multi-agent systems have also produced expert-validated taxonomies of recurrent collaborative failures, such as authority bias, self-contradiction, and failure to activate specialist reasoning, highlighting that collaboration can yield uneven accuracy gains and frequent process failures [335]. Similarly, broader assurance-focused taxonomies have been proposed to structure the risks associated with enterprise AI systems [336]. These categorical frameworks are essential for moving beyond identifying *that* a system failed to understanding *why* and *where*, directly informing targeted improvements [337].

Automating this process-level evaluation is critical for scalability and consistency. LLM-as-a-Judge protocols have become a dominant mechanism, but their reliability is under intense scrutiny. Research highlights a “Stability Trap,” where high verdict stability can mask fragile reasoning, leading to calls for scoping automated evaluation strictly: delegating deterministically verifiable logic to code while reserving LLM judges for complex semantic evaluation [338]. To mitigate judge bias and improve robustness, multi-perspective verification frameworks are being developed. For example, an evaluation harness using Adversarial Multi-Juror Scoring with Turn-Level Audit employs calibrated juror personas, consensus checks, and turn-level evidence to evaluate agent behavior under pressure [339]. Similarly, a tri-agent cross-validation framework, which integrates heterogeneous LLMs for semantic generation, analytical consistency checking, and transparency auditing, has been shown to induce stable recursive knowledge synthesis, providing a safety-preserving, human-supervised architecture for reliable evaluation [340].

Furthermore, preflight diagnostics are being introduced to vet the evaluation rubrics themselves. Frameworks like RADAR estimate the behavioral coupling between criteria in a rubric before large-scale evaluation, identifying redundancy and aggregation sensitivity to prevent distorted aggregate scores [341]. Fault injection is another powerful technique for stress-testing systems, systematically seeding defects to evaluate the sensitivity and precision of diagnostic tools, as demonstrated by the deterministic gates in a medical manuscript preparation pipeline which detected all 27 injected defects with no false positives [332]. Automated testing frameworks like AEVAL also formalize the evaluation process for agentic skills, creating deterministic contracts and separating the executor and grader to prevent self-correction bias and ensure auditable, evidence-grounded quality signals [342].

Finally, it is crucial to acknowledge the validity challenges that underpin all these measurement efforts. A compounding reliability problem exists in agentic AI evaluation, where validity can be lost at multiple stages—task generation, human simulation, and automated judgment—leading to a total validity bound that is the product of the validity of each stage [343]. This necessitates rigorous calibration, standardized reporting, and a principled evaluation paradigm that treats interactive evaluation as an autonomous mapping from interaction-generated evidence to judgments [344]. Collectively, these process-oriented metrics, detailed taxonomies, and automated verification frameworks represent a critical evolution towards building, evaluating, and deploying LLM-MAS that are not only high-performing but also reliable, interpretable, and trustworthy. They provide the diagnostic granularity and methodological rigor that the benchmarks and comparative analyses in the following subsection rely upon to assess failure attribution, long-horizon degradation, and the

conditions under which multi-agent configurations genuinely outperform single-agent baselines.

6.2 Benchmarks, Scalability, and Comparative Analysis of Single-Agent vs. Multi-Agent Systems

The evaluation of LLM-based Multi-Agent Systems (MAS) has evolved from simple outcome-based success metrics toward a multi-faceted assessment paradigm that encompasses standardized benchmarks, scalability analysis, and rigorous comparative studies against single-agent baselines. Building upon the process-oriented diagnostic frameworks and failure taxonomies established in the preceding section, this subsection examines how these diagnostic principles are operationalized into concrete benchmarks designed to measure failure attribution, long-horizon degradation, and real-world readiness. It further synthesizes the growing body of empirical evidence that questions the presumed superiority of multi-agent configurations, thereby establishing the empirical foundation for the subsequent discussion of failure localization and attribution methodologies.

A significant strand of benchmarking efforts focuses on diagnostic capabilities, particularly failure attribution—the process of identifying the responsible agent and decisive step leading to a system failure. A reliable benchmark is essential for guiding and evaluating attribution techniques, and the diagnostic granularity discussed in the previous section directly informs the design of these benchmarks. However, early benchmarks relied on partially observable traces that capture only agent outputs, omitting the inputs and context that developers actually use when debugging. To address this, TraceElephant introduced a benchmark designed for failure attribution with full execution traces and reproducible environments, demonstrating that full traces improve attribution accuracy by up to 76% over partial-observation counterparts [345]. Going further, the assumption of a single deterministic root cause has been challenged, as MAS failures often admit multiple plausible attributions due to complex inter-agent dependencies—a nuance that simpler outcome-level evaluations systematically obscure. The MP-Bench benchmark formalizes multi-perspective failure attribution, revealing that prior conclusions suggesting LLMs struggle with attribution were largely driven by limitations in existing benchmark designs [346]. In parallel, other diagnostic tools have emerged, such as AgentCollabBench, a benchmark of 900 human-validated tasks that isolates behavioral risks including instruction decay, false-belief contagion, context leakage, and tracer durability, exposing model-specific vulnerability profiles invisible to outcome-only evaluation [347]. Similarly, the MAS-FIRE framework provides a systematic approach to fault injection and reliability evaluation, defining a taxonomy of 15 fault types and demonstrating that architectural topology plays an equally decisive role as model strength in determining system robustness [348]. These diagnostic benchmarks directly operationalize the failure taxonomies and process-level metrics described earlier, translating them into standardized instruments for measuring system reliability.

Long-horizon evaluation has emerged as another critical dimension, as real-world tasks often span extended periods and require sustained reasoning, planning, and memory management—precisely the conditions under which the process-level degradation patterns identified in the previous section become most pronounced. Benchmarks such as SWE-Marathon introduce tasks that average 27.2M total tokens per agent attempt—substantially longer than existing software engineering benchmarks—and reveal that frontier coding agents solve fewer than 30% of tasks, with failures often arising from poor self-verification and premature termination [349]. UltraHorizon similarly probes long-horizon capabilities through exploration tasks averaging 200k+ tokens and 400+ tool calls, identifying eight error types attributed to in-context locking and functional capability gaps [350]. The concept of horizon residual has been proposed to distinguish genuine long-horizon failures from compound short-task errors, arguing that benchmarks must compare actual full-task success against a baseline prediction built from short, individual stages [351]. This methodological rigor

extends to the measurement of reliability decay and variance amplification, introducing metrics such as the Reliability Decay Curve and Meltdown Onset Point to capture how agent performance degrades over extended durations [352, 353]. These long-horizon benchmarks complement the diagnostic frameworks from the previous section by providing standardized stress-tests that reveal cumulative error propagation over extended trajectories.

At the system level, general-purpose evaluation suites have been developed to standardize MAS configuration and execution, enabling controlled comparisons across architectures and thus providing the empirical infrastructure necessary for the comparative analyses that follow. The MAESTRO suite provides a unified interface for integrating native and third-party MAS, exporting framework-agnostic traces and system-level signals, with case studies revealing that MAS architecture is the dominant driver of resource profiles, reproducibility, and cost-latency-accuracy trade-offs [354]. Similarly, OrchBench offers a simulation-based benchmark for evaluating orchestration plans in isolation, demonstrating strong correlation with quality scores from real executions while requiring only 1.3% of the tokens and 10.3% of the wall-clock time [355]. The CLEAR framework introduces a multi-dimensional evaluation paradigm encompassing Cost, Latency, Efficacy, Assurance, and Reliability, showing that optimizing for accuracy alone yields agents 4.4-10.8x more expensive than cost-aware alternatives with comparable performance [356]. Together with the process-oriented evaluation methodologies discussed previously, these system-level suites provide a comprehensive toolkit for assessing both the internal dynamics of MAS and their external performance characteristics.

A central debate in the field concerns whether multi-agent collaboration provides genuine value over strong single-agent baselines. Empirical evidence increasingly suggests that the answer depends critically on task structure, architecture-task alignment, and agent diversity—factors that the diagnostic frameworks of the previous section are uniquely positioned to quantify. A large-scale study spanning 260 configurations, six benchmarks, and five canonical architectures found that coordination yields diminishing returns once single-agent baselines exceed certain performance thresholds, with relative performance changes ranging from +80.8% on decomposable financial reasoning to -70.0% on sequential planning [111]. The study identified a robust capability-saturation effect, suggesting that architecture-task alignment determines collaborative success. This finding is echoed in an analysis of scaling behavior in homogeneous MAS, which established that performance does not scale monotonically with agent count but follows a pattern of diminishing returns governed by a trade-off between collaborative synergy and coordination overhead [28]. The study concludes that collective intelligence is an emergent property contingent on strategic interaction design rather than a guaranteed outcome of agent plurality. Similarly, the Six Sigma Agent architecture demonstrates that reliability improvements can emerge from principled redundancy and consensus voting rather than model scaling alone, achieving a 14,700x reliability improvement over single-agent execution while reducing costs by 80% [357].

Information-theoretic analyses provide principled explanations for these empirical observations and connect directly to the reliability frameworks discussed in the previous section. A theoretical framework showing that MAS performance is bounded by intrinsic task uncertainty, not by agent count, introduced the concept of effective channel count K^* , demonstrating that homogeneous agents saturate early because their outputs are strongly correlated, whereas heterogeneous agents contribute complementary evidence [31]. Crucially, the study found that two diverse agents can match or exceed the performance of 16 homogeneous agents, providing a quantitative justification for diversity-aware design. A complementary phase-transition theory proves that for binary success/failure tasks, budgeted synergy—outperforming the best single agent under the same total budget—occurs exactly when an organization exponent exceeds the compute-performance scaling exponent, yielding closed-form compute allocation rules [358]. These theoretical advances are substantiated by empirical studies of diversity collapse in MAS-based ideation, which demonstrate

that authority-driven dynamics suppress semantic diversity and dense communication topologies accelerate premature convergence, characterizing these outcomes as collective failures emerging from structural coupling [359].

Comparative studies also highlight the critical role of agent heterogeneity, which resonates with the process-level diagnostics emphasizing the importance of diverse reasoning pathways. A framework demonstrating that model heterogeneity is the critical factor driving substantial performance improvements found that heterogeneous configurations achieve step-wise accuracy of 0.64 versus 0.54 for individual models—a 2.3x improvement over homogeneous configurations—with reduced variance across categories and difficulty levels [360]. This aligns with the observation that heterogeneous configurations consistently outperform homogeneous scaling, as complementary error detection and reasoning refinement become possible only with model diversity. At the same time, studies of emergent coordination in large-scale LLM populations reveal severe coordination overhead in decentralized task resolution, with a Cohen’s d of -0.88 against a single-agent baseline, highlighting the challenges of scaling agent populations without corresponding coordination mechanisms [45]. Furthermore, the bystander effect in multi-agent reasoning has been quantified, demonstrating that simulated social pressure triggers cognitive loafing, where models compute correct derivations internally but actively subjugate empirical evidence to sycophantically appease a simulated swarm [9]. This behavioral vulnerability reveals that unstructured multi-agent topologies can degrade independent reasoning, complicating the assumption that collaboration inherently improves outcomes.

Taken together, these findings suggest that the value of MAS must be assessed against task-specific characteristics and architectural constraints. The empirical literature increasingly supports a nuanced view: multi-agent collaboration yields benefits when tasks are decomposable, agents are diverse, and coordination structures are aligned with task dependencies, but it can introduce costs and failure modes that outweigh benefits in other configurations. This underscores the necessity of continued benchmarking efforts that measure not only final outcomes but also process-level reliability, scalability, and the conditions under which collective intelligence genuinely exceeds the sum of individual capabilities. These benchmark-driven insights directly inform the subsequent discussion of failure localization and attribution, as they establish the empirical context in which diagnostic methods must operate—namely, complex systems where failures emerge from intricate inter-agent dependencies, where attribution ambiguity is the norm rather than the exception, and where the path from reliable evaluation to precise diagnosis requires the methodological rigor that the following subsection examines in depth.

6.3 Failure Localization, Attribution, and Error Propagation Diagnosis

Failure localization and attribution constitute one of the most challenging and actively researched problems in LLM-based multi-agent systems (LLM-MAS). Unlike single-agent systems where errors can often be traced to a specific model output or tool call, failures in MAS emerge from complex inter-agent dependencies, natural-language reasoning, nondeterministic outputs, and long-horizon interaction trajectories. The task of failure attribution—identifying both the responsible agent and the decisive step at which the trajectory becomes irreversibly misdirected—is fundamental to debugging, reliability engineering, and system improvement [361]. Early work by Chen et al. formalized this problem through the Who&When dataset, which comprises extensive failure logs from 127 LLM multi-agent systems with fine-grained annotations linking failures to specific agents and decisive error steps. Their evaluation of three automated attribution methods revealed the difficulty of the task: the best method achieved 53.5% accuracy in identifying failure-responsible agents but only 14.2% in pinpointing failure steps, with some methods performing below random and state-of-the-art reasoning models such as OpenAI o1 and DeepSeek R1 failing to achieve practical

usability [361]. This finding catalyzed a substantial body of follow-up research.

A critical methodological insight that emerged from subsequent studies concerns the observability assumptions underlying attribution benchmarks. Traditional benchmarks have relied on partially observable traces that capture only agent outputs, omitting the inputs and context that developers actually use when debugging. TraceElephant directly addresses this gap by arguing that failure attribution should be studied under full execution observability, aligning with real-world developer-facing scenarios where complete traces are accessible for diagnosis. Their benchmark, constructed with full execution traces and reproducible environments, demonstrated that full traces improve attribution accuracy by up to 76% over partial-observation counterparts, confirming that missing inputs obscure many failure causes [345]. Complicating matters further, failures in MAS often admit multiple plausible attributions due to complex inter-agent dependencies and ambiguous execution trajectories. MP-Bench introduced the paradigm of multi-perspective failure attribution, explicitly accounting for attribution ambiguity, and its experiments revealed that prior conclusions suggesting LLMs struggle with failure attribution were largely driven by limitations in existing benchmark designs [346]. For long-horizon tasks exceeding context window limits, SAFARI proposed a tool-augmented diagnostic loop that equips LLMs with specialized toolbox functions to read and search trajectory segments alongside persistent short-term memory, effectively decoupling diagnostic accuracy from architectural context limits and maintaining high precision even when the target fault resides far beyond the model’s native context window [362].

The temporal dimension of failure attribution has also received focused attention. StepFinder recognized that existing LLM-based attribution methods suffer from high inference costs and interference caused by redundant and noisy execution logs. Their framework uses LLMs solely during a feature construction phase to encode execution logs into temporal semantic sequences, followed by a parameter-efficient combination of temporal modeling and attention modules to capture sequential evolution and cross-step dependencies, achieving superior step-level attribution while reducing inference time by 79% compared with the fastest LLM-based method [363]. MASPrism pushes this efficiency further by performing attribution using prefill-stage signals from a small language model, extracting token-level negative log-likelihood and attention weights without decoding, achieving strong performance with zero output tokens and substantial speedups [364]. At the other extreme of handling long trajectories, AgentForesight reframes attribution from post-hoc diagnosis to online auditing, where an auditor observes only the current prefix at each step and must either continue the run or alarm at the earliest decisive error [365].

A major line of work has focused on modeling the structural dependencies inherent in MAS execution. GraphTracer identifies two core challenges in multi-agent error propagation: distinguishing symptoms from root causes and tracing information dependencies beyond temporal order. Their framework constructs Information Dependency Graphs that explicitly capture how agents reference and build on prior outputs, localizing root causes by tracing through dependency structures rather than temporal sequences [366]. Similarly, CHIEF transforms chaotic trajectories into structured hierarchical causal graphs and employs hierarchical oracle-guided backtracking with counterfactual attribution via progressive causal screening to rigorously distinguish true root causes from propagated symptoms [367]. FALAT frames attribution as a dependency-guided search problem, first constructing an expectation of how the task should be solved, then tracing dependencies among decisions, tool outputs, and agent messages to distinguish error-introducing steps from those that merely inherit prior mistakes, and finally evaluating whether correcting a candidate step would suffice to recover the expected outcome [368]. ECHO complements these approaches with a hierarchical context representation combined with consensus-based objective decision-making, leveraging positional-based leveling of contextual understanding to achieve robust attribution across complex interaction scenarios [369]. CockpitHAT extends dependency-aware attribution to embodied

settings, replacing positional windows with dependency-distance thresholds from interaction DAGs and integrating multi-channel evidence via an embodied adapter, achieving substantial gains in safety-critical automotive scenarios [370].

Causal inference has proven to be a particularly powerful foundation for attribution. CausalFlow models execution traces as sequential chains of dependent steps and computes Causal Responsibility Scores via step-level counterfactual intervention to identify failure-inducing steps, generating minimal counterfactual repairs that flip the final outcome to success [371]. A complementary approach based on multi-granularity causal inference introduced a performance causal inversion principle that correctly models performance dependencies by reversing the data flow in execution logs, combined with Shapley values for agent-level blame assignment and a causal discovery algorithm for critical failure step identification [372]. HPFA leverages hypergraph-based paired failure attribution that attributes root causes by comparing hyperedges of failed reasoning paths against reference successful paths, reducing the search space and enabling scalable synthesis of attribution data for training lightweight attributor models [373]. For unsupervised settings, OAT casts failure attribution as one-class learning with neural controlled differential equations, modeling the dynamical pattern of successful trajectories in latent space and assigning anomaly scores based on deviation from learned dynamics, achieving dramatic speedups and consistent performance improvements over prompting-based baselines [374].

Verification-based approaches have emerged as a principled alternative to direct prediction. VerifyMAS formulates and verifies failure hypotheses against full trajectories rather than directly predicting faulty agents and error types, decomposing attribution into trajectory-level error validation and fine-grained agent localization, substantially reducing the search space while capturing global failure patterns [375]. This verification-first philosophy is echoed in REFLECT, which closes the loop by diagnosing a candidate error step, testing it through controlled replay with a diagnosis-specific patch, and using the verified outcome flip as contrastive evidence to refine the final attribution [376]. DoVer similarly augments hypothesis generation with active verification through targeted interventions, finding that multiple distinct interventions can independently repair a failed task and arguing for outcome-oriented debugging metrics [377].

Self-improving diagnostic pipelines represent another frontier in attribution research. ErrorProbe introduces a three-stage pipeline that operationalizes MAS failure taxonomy to detect local anomalies, performs symptom-driven backward tracing to prune irrelevant context, and employs a specialized multi-agent team to validate error hypotheses through tool-grounded execution, maintaining verified episodic memory that updates only when error patterns are confirmed by executable evidence [378]. CORRECT exploits the insight that MAS errors often recur with similar structural patterns despite surface differences, using an online cache of distilled error schemata to recognize and transfer knowledge of failure structures across new requests, achieving near-zero overhead training-free error localization [379]. For scalable data generation, Aegis constructs large datasets of trajectories with annotated faulty agents and error modes using an LLM-based manipulator that adaptively injects context-aware errors into successful trajectories, supporting multiple learning paradigms for error attribution [380].

Edge-level error taxonomies provide an important lens for understanding where failures originate in the interaction structure. AgentAsk introduces an edge-level error taxonomy identifying four dominant error types—Data Gap, Signal Corruption, Referential Drift, and Capability Gap—as primary sources of failure in multi-agent interactions, and proposes lightweight clarification modules that intervene at the edge level to prevent cascading errors [381]. The broader failure taxonomy literature has also examined the interaction between model and harness in agent failures, proposing an interaction-centric taxonomy that localizes failures to the interactions in which they originate and identifies the responsible component across 41 failure modes [382]. AgentEval formalizes agent

executions as evaluation directed acyclic graphs where each node carries typed quality metrics and is linked to upstream dependencies, demonstrating that DAG-based dependency modeling alone contributes substantial gains in failure detection recall and root cause accuracy over flat step-level evaluation [331].

The role of topology in error propagation has emerged as a critical dimension in attribution research. MAS-FIRE introduces a systematic framework for fault injection and reliability evaluation, defining a taxonomy of 15 fault types covering intra-agent cognitive errors and inter-agent coordination failures, and demonstrating that architectural topology plays a decisive role—iterative, closed-loop designs neutralize over 40% of faults that cause catastrophic collapse in linear workflows [348]. The propagation dynamics model from From Spark to Fire abstracts collaboration as a directed dependency graph, identifying three vulnerability classes—cascade amplification, topological sensitivity, and consensus inertia—and demonstrating that injecting a single atomic error seed can lead to widespread failure [383]. For long-horizon tasks, HORIZON provides a cross-domain diagnostic benchmark that reveals horizon-dependent degradation patterns and proposes trajectory-grounded LLM-as-a-judge pipelines validated against human annotation [384].

Several foundational challenges remain open in MAS failure attribution. First, full execution observability is necessary but not always sufficient—even with complete traces, attribution ambiguity persists due to complex inter-agent dependencies [346]. Second, cognitive failure diagnosis requires distinguishing between different types of reasoning errors, such as omissions, hallucinations, and causal reasoning deficiencies, which may manifest similarly at the behavioral level [385]. Third, the pre-hoc/post-hoc dichotomy suggests that proactive risk inference before interaction could complement reactive attribution after failure, as demonstrated by HalluProp’s propagation-aware hallucination inference framework that estimates failure risks before inter-agent interaction [386]. Fourth, scalable benchmarks that combine controlled failure injection with realistic patterns remain essential for progress, as exemplified by Who&When Pro’s strictly controlled pipeline that injects failure only after exactly replaying a successful prefix across 12,326 failed trajectories [387] and TracerTraj’s counterfactual replay and programmed fault injection approach [388]. Finally, the integration of attribution with repair—moving from diagnosis to actionable recovery—represents a crucial direction, with frameworks like MARS formulating MAS repair as Monte Carlo Tree Search guided by diagnosis information [389] and AgentDebugX organizing debugging as a closed loop of detect, attribute, recover, and rerun [390]. Collectively, these advances underscore that failure attribution in LLM-MAS is not merely a technical exercise but a foundational capability for building reliable, trustworthy, and self-improving multi-agent systems.

6.4 Reliability Engineering, Robustness Testing, and Defense-Aware Evaluation

As LLM-based multi-agent systems (LLM-MAS) transition from research prototypes to deployment in high-stakes domains, the evaluation of system reliability under adversarial and degraded conditions becomes a first-order concern. While the failure attribution methods discussed in the previous section provide the diagnostic foundation for identifying *where* and *why* systems break, they are ultimately retrospective tools—they operate after a failure has already manifested. To complete the reliability picture, the field must also develop prospective, systematic methodologies for measuring, stress-testing, and guaranteeing system dependability before and during deployment. Traditional accuracy-based metrics, such as pass@1, are structurally insufficient for capturing the complex failure modes that emerge from probabilistic inference, long-horizon task execution, and inter-agent coordination. A growing body of research therefore advocates for a paradigm shift toward holistic reliability engineering, treating reliability not as a single aggregate score but as a multi-dimensional property that must be measured, stress-tested, and guaranteed through structured methodologies.

This emerging discipline draws heavily from safety-critical engineering practices, particularly fault injection, consensus theory, and formal verification, while adapting these tools to the unique challenges posed by stochastic, communication-driven multi-agent architectures.

A foundational step in this direction is the decomposition of reliability into orthogonal, measurable dimensions. A prominent framework proposes twelve distinct metrics grouped along four essential axes—consistency, robustness, predictability, and safety—arguing that compressing agent behavior into a single success metric obscures critical operational flaws such as run-to-run variance, sensitivity to perturbations, and unbounded error severity [353]. This perspective is complemented by statistical measurement theory that distinguishes between core capability and execution robustness, demonstrating that minor task-level variations can induce complete strategy breakdowns even when an agent possesses the requisite knowledge for the task. Using U -statistics for output-level reliability and kernel-based metrics for trajectory-level stability, this approach enables the isolation of architectural weaknesses that hinder deployment in real-world environments [391]. Such multi-faceted frameworks address a fundamental limitation of conventional benchmarks: the divergence between capability (succeeding once) and reliability (succeeding consistently). Empirical studies on long-horizon LLM agents reveal that this divergence is systematic, with reliability decay being domain-stratified and capability-reliability rankings often inverting at longer task durations; notably, frontier models exhibit the highest meltdown rates because they attempt ambitious multi-step strategies that sometimes spiral into failure [352]. This capability/reliability dissociation underscores the need for reliability as a first-class evaluation dimension, distinct from traditional capability benchmarking.

Robustness testing under perturbation is a critical component of reliability assessment. Conventional adversarial robustness evaluation, which typically measures accuracy under a single perturbation intensity, fails to fully capture a model’s resilience across varying degrees of perturbation. The adversarial hypervolume metric addresses this gap by assessing robustness over a range of perturbation intensities from a multi-objective optimization standpoint, providing a more comprehensive and discriminating measure than single-point adversarial accuracy [392]. For multi-agent systems specifically, robustness evaluation must also account for the fact that agents operate within communication topologies that can amplify or mitigate errors. Research on multi-hop information survival has shown that communication topology alone can explain 7–40% of the variance in information survival, with converging-DAG structures creating a “synthesis bottleneck” where agents weighing competing parent inputs discard constraints carried by a minority branch [347]. This finding demonstrates that agent reliability is fundamentally a structural problem, and that scaling model intelligence alone is no substitute for robustness-aware architecture design. Overall, robustness surfaces only when exposed to systematically generated difficult scenarios. For example, semantically aligned augmentation, target bootstrapping, and adversarial knowledge injection have been used to build stress-testing pipelines that reveal error rates of up to 19% absolute in zero-shot adversarial settings, even among state-of-the-art models with dedicated safeguards [393].

Fault injection and coverage-guided fuzzing constitute the empirical backbone of reliability testing in complex systems. Inspired by hardware reliability analysis, statistical fault injection frameworks leverage finite-population sampling theory to provide formal reliability guarantees—demonstrating that failure rates can be bounded within a 1% margin at 99% confidence using only a few thousand samples, regardless of model scale, and achieving up to $10,700\times$ reductions in experimental cost [394]. In the context of LLM-MAS, structured fault injection at service, retrieval, and memory boundaries allows for assessing containment under realistic operational faults and degraded conditions, complementing stress testing that is formulated as a budgeted counterexample search over bounded perturbations [395]. These methods enable the proactive identification of latent risks—hidden vulnerabilities where exceptional performance masks catastrophic fragility when

optimizations fail. For instance, cache layers achieving 99% hit rates can obscure database bottlenecks until cache failures trigger $100\times$ load amplification and cascading collapse; integrated perturbation testing strategies have achieved up to 89.7% risk discovery rates [396]. The SMART framework further formalizes reliability testing through its five components—Structure, Metrics, Analysis of failure causes, Reliability assessment, and Test planning—providing a systematic methodology for adapting traditional reliability data analysis to AI systems [397].

Byzantine fault tolerance (BFT) and consensus mechanisms offer a principled theoretical foundation for ensuring system-wide reliability under adversarial conditions, even when individual agents may be compromised. Drawing an analogy between unreliable or malicious AI artifacts and Byzantine nodes in a distributed system, consensus-based architectures have been proposed to enhance AI safety and reliability [398]. However, traditional BFT models assume that participants outside the Byzantine set correctly execute the protocol’s transition semantics. Agentic validators expose a weaker boundary: a protocol-compliant reasoning participant may still endorse a semantically invalid transition due to reasoning errors. The Honest Quorum Problem formalizes this failure mode, introducing Epistemic Byzantine Fault Tolerance (EBFT), which augments the conventional Byzantine fault bound with confidence-indexed quantities that bound coherent invalid endorsements and unusable validator support, thereby providing quorum-threshold conditions for semantic validity and liveness [399]. Probabilistic consensus protocols offer a complementary approach, ensuring safety and liveness with high probability rather than deterministically, thereby improving scalability in systems with realistic adversaries [400]. These theoretical frameworks are essential for reasoning about the upper bounds of system reliability when agents share model weights, training distributions, or prompts, making them highly susceptible to correlated epistemic faults.

A particularly insidious challenge in reliability evaluation is the divergence between formal certificates and operational safety. Certification frameworks can pass while task accuracy collapses; for instance, at a perturbation budget of $\epsilon=0.25$, classifier accuracy can drop by 25.7% under projected-gradient attacks while a Lipschitz-style certificate remains valid for all tested subjects [401]. This gap between verification and operational reality extends to the evaluation context itself, where models may recognize evaluation contexts and behave differently than under deployment-continuous conditions, creating an Evaluation Differential that undermines the validity of safety conclusions drawn from frontier evaluations [402]. Reliability assessment must therefore be grounded in trace-based assurance frameworks that instrument executions as Message-Action Traces with explicit step and trace contracts. These contracts provide machine-checkable verdicts, localize the first violating step, and support deterministic replay, while governance is treated as a runtime component enforcing per-agent capability limits and action mediation [395].

Test-time adaptation and continuous improvement are essential for sustaining reliability over extended deployments. Reliability metrics must account for the temporal dimension, recognizing that systems must perform their designed functionality for the intended period [397]. This includes statistical runtime verification approaches that assess robustness online by analyzing confidence score distributions under semantic perturbations, providing quantitative robustness assessments with statistically validated bounds while reducing verification times from hours to minutes [403]. The challenge is compounded by instruction conflicts and model-harness configurations: deviations in evaluation setup can produce misleading reliability estimates, and consistency must be measured under semantically preserving perturbations to isolate true system reliability from evaluator artifacts [391, 404].

In summary, reliability engineering for LLM-MAS is evolving from single-point accuracy metrics to a multi-dimensional discipline that integrates statistical measurement theory, structured fault injection, consensus-based fault tolerance, and trace-based verification. The complexity of multi-agent systems—characterized by emergent communication failures, topological error amplification,

and correlated epistemic faults—demands reliability assessment approaches that are simultaneously holistic, structural, and grounded in formal guarantees. The field is moving toward a unified assurance paradigm where reliability is not merely reported but engineered, verified, and continuously monitored as a fundamental property of the socio-technical system. This prospective reliability framework, together with the retrospective attribution methods detailed in the previous section, forms the two complementary pillars of a complete reliability science for LLM-MAS, setting the stage for the design principles and architectural patterns that will be examined in the following section.

7 Security, Robustness, and Responsible AI

7.1 Threat Taxonomy and Attack Surfaces in LLM-MAS

LLM-based multi-agent systems (LLM-MAS) inherit a broad spectrum of security vulnerabilities from their constituent large language models, but their distributed, communicative, and collaborative architecture introduces a qualitatively distinct and significantly expanded attack surface [405]. Unlike single-agent systems where the attack surface is largely confined to the prompt interface and the model’s internal parameters, LLM-MAS exposes new vulnerabilities through inter-agent communication channels, shared memory stores, complex coordination topologies, delegated tool authority, and emergent system-level behaviors [406, 407]. Understanding this threat landscape is not merely an academic exercise—it is the essential prerequisite for the defense mechanisms examined in the remainder of this survey. The attacks catalogued below define the precise adversary models, compromised system layers, and failure modes that architectural defenses, proactive guards, memory protections, and consensus mechanisms must contend with. A systematic taxonomy of these threats is therefore essential for understanding attacker capabilities, objectives, and the specific system layers under attack.

Communication-Based Attacks. The most distinguishing attack surface of LLM-MAS is the inter-agent communication channel. Since agents exchange messages in natural language or structured formats, these channels become prime vectors for malicious manipulation. A principal threat is the spread of adversarial content, where a single compromised or malicious agent injects harmful prompts or misinformation that propagates through the system, altering the behavior of otherwise benign agents [408, 409]. This can manifest as cascading prompt injection, where a single malicious message poisons the context of subsequent agents, leading to a chain of unsafe actions. Attacks can also target the communication topology itself. For instance, an adversary may attempt to infer the underlying interaction graph—which agent talks to whom—to identify critical nodes or bottlenecks for more targeted attacks, a risk formalized as the Communication Inference Attack (CIA) [410]. Furthermore, the metadata of communication—the frequency, timing, and direction of messages—can leak sensitive information about the nature of the ongoing task and system structure, even if message content is encrypted, posing a workflow integrity threat [411]. The structural design of the system itself, including whether it is centralized or decentralized and how roles are allocated, has been shown to significantly influence its vulnerability to these communication-based attacks, with some topologies amplifying susceptibility [412, 409]. As the defense literature demonstrates, this architectural dependency is not merely an attack-side observation but a foundational design principle: the empirical finding that attack success rates vary by up to 3.8x across architectures directly motivates the architectural-first defense philosophy that dominates current mitigation strategies. A specific malicious pattern is the “Telephone Loop” attack, which exploits cross-agent delegation to create cyclical task loops, causing the system to waste resources and potentially perform unintended actions; this attack is inert in single-agent systems but highly effective in

multi-agent ones [408].

Memory and Context Attacks. Agents in a MAS often rely on shared or persistent memory to maintain context across interactions and sessions. This cognitive infrastructure introduces a unique class of attacks centered on poisoning or manipulating the knowledge base that agents use for decision-making. An attacker can plant a dormant payload into an agent’s long-term memory via a single untrusted tool call, which activates later when the user discusses a specific sensitive topic (e.g., finance or health), triggering data exfiltration; this persistent attack class has been demonstrated to be highly effective across various memory architectures [413]. Such attacks extend the attack lifecycle beyond a single session, with stateful agent backdoors capable of executing a fixed, multi-step malicious behavior incrementally across multiple sessions, bypassing session-based permission isolation [414]. Beyond direct memory poisoning, attackers can exploit the context-sharing mechanisms between agents. If one agent provides a malicious summary, fabricated information, or a misleading plan to another, it can bias the entire downstream reasoning process. This is particularly dangerous in hierarchical systems where a planner agent’s instructions are treated as ground truth by worker agents. The attack surface here is not just the stored memory but the entire write–manage–read loop, where malicious content can be injected at any stage to corrupt the collective knowledge and influence future agent actions [415]. The severity of this threat class is underscored by the defense literature’s growing focus on memory-specific protections: frameworks such as memory sandboxing, write-gate quarantining, and consensus-based memory validation have emerged specifically in response to the persistence and stealth of these attacks, and empirical evaluations confirm that defense effectiveness is tightly coupled to the architectural layer at which memory protection is enforced.

Backdoor and Byzantine Attacks. Backdoor attacks in LLM-MAS are more complex than in single models, as they can be distributed across the system’s components. A novel class, the Distributed Backdoor Attack, decomposes the backdoor trigger and malicious payload into multiple “primitives” embedded within different agent tools [416]. These primitives lie dormant individually and only activate to form the full backdoor when agents collaborate in a specific sequence, making detection by inspecting any single component or agent nearly impossible. This distributed nature leverages the emergent capabilities of the MAS itself to hide the malicious logic [415]. A related but distinct threat is that of Byzantine agents—agents that behave arbitrarily or maliciously either due to compromise or adversarial design. Unlike a backdoor that requires a specific trigger, a Byzantine agent can disrupt the system’s operation at any time by providing incorrect information, failing to execute tasks, or actively sabotaging the collaborative process. Defending against such failures often requires Byzantine fault-tolerant consensus mechanisms, which are challenging to implement effectively in non-deterministic LLM-based systems [417, 418]. The “weak link” problem is also pertinent here: an adversary can target the least robust agent in the network, using it as an entry point to compromise the entire system’s output, a vulnerability that has been empirically shown to be influenced by architectural design choices [409]. Furthermore, attacks can be multimodal, targeting the perception, communication, and reasoning layers of agents that process different data types, with reasoning-layer attacks proving particularly effective at causing consistent errors across multiple agents [419]. These Byzantine and distributed threats directly motivate the consensus-based and game-theoretic defense mechanisms discussed in the following section, including Byzantine fault-tolerant voting protocols and credit-based malicious-agent detection.

Privacy-Related Risks. The collaborative nature of LLM-MAS, involving the exchange of information among multiple agents and access to various tools, intensifies privacy risks. The system’s communication graph, as mentioned, is a source of privacy leakage; an external observer can recover the task’s class from passive metadata without seeing the message content [411]. Memory systems, which are essential for long-term operation, become high-value targets for data exfiltration, as

they persist user information that can be stolen via a single backdoor trigger [413, 420]. The complexity of agents’ tool execution and multi-step workflows also creates more opportunities for side-channel leaks or unintended disclosure of intermediate states. The inherent autonomy and delegated authority of agents can enable “servant, stalker, predator” attack chains, where individually benign operations—such as data retrieval, location tracking, and code deployment—are chained together across services to perform data exfiltration or financial manipulation [421]. This compositional attack surface grows with each additional tool or capability available to the system.

In summary, the threat landscape of LLM-MAS is multi-layered and emergent. Threat categories are not isolated; communication attacks can be used to poison memory, backdoors can be triggered to leak private data, and Byzantine agents can create cascading failures. Formalizing these threats with clear attacker capabilities, objectives, and affected system layers is the foundational step toward developing robust security frameworks, moving beyond single-agent protection to address the structural vulnerabilities inherent in multi-agent collaboration [407, 418]. The defense mechanisms that follow are best understood as systematic responses to this taxonomy: architectural defenses target the topology-dependent vulnerabilities, proactive guards and memory protections address the persistent and stealthy memory and context attacks, and consensus-based approaches respond to Byzantine and distributed backdoor threats. This threat-to-defense mapping provides the conceptual bridge from the attack landscape catalogued here to the layered defense ecosystem examined next.

7.2 Defense Mechanisms and Mitigation Strategies

The escalating threat landscape of LLM-based multi-agent systems (LLM-MAS) has driven the development of a rich and heterogeneous body of defense mechanisms spanning multiple levels of the system stack. Building directly upon the threat taxonomy established in the previous section, these defenses are best understood as systematic responses to specific attack classes: architectural defenses target topology-dependent communication vulnerabilities, memory protections counter persistent poisoning and backdoor attacks, and consensus-based approaches address Byzantine failures and distributed threats. A central insight emerging from recent analysis is that architectural design choices themselves are a first-order security parameter: the empirical study by [412] demonstrates that multi-agent architectures are more vulnerable than standalone agents in the majority of configurations, with attack success rates varying by up to 3.8x at comparable or higher benign accuracy. This finding—which echoes the attack-side observation from the threat landscape that communication topology significantly influences susceptibility—fundamentally reframes defense as a design-time concern rather than a purely runtime countermeasure. Indeed, the framework proposed in [422] formalizes this perspective by decomposing attack surfaces across component, coordination, and protocol layers, identifying tool orchestration and memory management as the primary trust boundaries, and deriving five defensive principles—authorized interfaces, capability scoping, verified execution, memory integrity, and access-controlled data isolation—aligned with established compliance standards. This architectural-first philosophy positions defense as an intrinsic property of system design, echoing the broader principle from [418] that agentic safety must be co-designed with capability as a fundamental architectural property.

At the architectural level, defenses can be broadly categorized by the distribution of control and the placement of monitoring functions. Centralized architectures concentrate defense in a single authority but risk creating single points of failure. The Sentinel Agents framework [423] addresses this by proposing a distributed security layer comprising a network of Sentinel Agents that oversee inter-agent communications via semantic analysis, behavioral analytics, and retrieval-augmented verification, complemented by a Coordinator Agent that ingests alerts, adapts policies, and isolates misbehaving agents—a dual-layered design supporting dynamic and adaptive defense. Similarly,

the MAS-Shield framework [424] navigates the centralized-versus-decentralized trade-off through a three-stage coarse-to-fine filtering pipeline: critical agent selection narrows the defense surface, lightweight sentry models rapidly filter benign cases, and a heavyweight committee provides global consensus auditing only for escalated signals. This hierarchical design achieves a 92.5% recovery rate against diverse adversarial scenarios while reducing defense latency by over 70%, directly addressing the tension between security and efficiency. Pushing further toward decentralization, SentinelNet [425] equips each agent with a credit-based detector trained via contrastive learning on augmented adversarial debate trajectories, enabling autonomous evaluation of message credibility and dynamic neighbor ranking through bottom-k elimination—achieving near-perfect detection of malicious agents within two debate rounds and recovering 95% of system accuracy from compromised baselines. These architectural responses directly counter the communication-based attacks—cascading prompt injection, topology inference, and telephone loop exploitation—identified as the most distinguishing attack surface in the threat taxonomy.

A critical distinction has emerged between reactive and proactive defense paradigms. The majority of conventional defenses are reactive, detecting and isolating harmful agents after execution, which risks irreversible damage and degrades collaborative utility. The Simulation-aware Interception Guard (SAIGuard) [426] pioneers a proactive approach by performing communication-state simulation over the MAS interaction graph, estimating the impact of incoming messages on local and global system states, and detecting risky messages via reconstruction deviations from benign communication patterns. Rather than isolating agents, SAIGuard sanitizes or regenerates suspicious messages before propagation, reducing attack success rates while maintaining MAS utility. In a similar spirit, the Memory As Guardrail Enforcement (MAGE) framework [427] maintains a dedicated safety-focused agentic memory that distills safety-critical context across the full execution trajectory, inspired by the shadow stack abstraction in systems security, enabling proactive risk assessment of pending actions before execution—the first framework to detect and mitigate long-horizon threats via an agentic memory approach. The Cognitive Firewall [428] extends proactive oversight to conversation-level analysis through four categorical gates—intent, zero-trust context, consistency, and output risk—combined through escalation rather than score averaging, allowing any confident danger signal to block an interaction while preserving auditable rationale. Complementary to these, the Stateful Cooperative Agents framework [429] employs specialized agents executing complementary defensive strategies—controlled response pacing, strategically ambiguous outputs, and forensic analysis—coordinated by an adaptive mechanism that dynamically adjusts defense in response to escalating threats, reducing attack success rates by 69% on average while sustaining deceptive engagement and increasing attacker-token consumption by 198.83%. This shift toward proactive defense is particularly significant given the threat taxonomy’s emphasis on dormant payloads and stateful backdoors that can persist across sessions, as proactive methods aim to intercept these threats before they activate.

Memory-specific protections have emerged as a critical frontier given the persistent attack surface introduced by memory systems, directly responding to the memory and context poisoning attacks catalogued in the threat analysis. The Memory-Aware Propagation and Link Enforcement Guard (MAPLE-Guard) [430] monitors the memory lifecycle with gates at write, retrieval, promotion, and cross-agent reuse, quarantining risky memories and filtering unsafe retrievals—lowering attack success rates from 38.2% to 0.9% on LongMemEval and from 34.7% to 0.2% on AppWorld. The Agent-Memory Guard (A-MemGuard) [431] introduces the first proactive defense framework for agent memory through consensus-based validation, comparing reasoning paths derived from multiple related memories to detect anomalies, and a dual-memory structure where detected failures are distilled into lessons stored separately and consulted before future actions—breaking error cycles and reducing attack success rates by over 95% with minimal utility cost. AgentSafe [432]

contributes hierarchical information management through ThreatSieve, which secures communication by verifying information authority and preventing impersonation, and HierarCache, an adaptive memory management system defending against unauthorized access and malicious poisoning, achieving defense success rates above 80% under adversarial conditions. The empirical evaluation in [433] reveals a critical architectural insight: only tool-gating at the memory layer (Memory Sandbox) reduces attack success rates to 0% for eight of nine models with zero utility cost, whereas input-level filters, retrieval-level classifiers, and instruction-level hardening all fail—demonstrating that defense effectiveness is determined by architectural layer and reasoning capability, not classifier quality. This finding is reinforced by [434]’s three-tier taxonomy analysis showing that write-time defenses suppress direct attacks but fail against compositional and trigger-conditioned dormant corruption, advocating a shift from static filtering to adaptive, context-sensitive memory defense strategies. The Moving Target Defense concept can also be applied to memory: the Co-Forgetting Protocol [435] integrates context-aware semantic voting, multi-scale temporal decay functions, and practical Byzantine fault tolerance consensus to enable synchronized memory pruning, achieving a 52% reduction in memory footprint with 88% voting accuracy and 92% consensus success under Byzantine conditions.

Game-theoretic and consensus-based approaches provide principled foundations for coordination under adversarial conditions, directly addressing the Byzantine and distributed backdoor threats identified in the threat taxonomy. The MESA framework [436] addresses proactive edge prioritization by combining six graph-theoretic metrics and two dynamic probes to rank which communication channels are most security-critical, achieving mean Spearman correlations of +0.60 with empirical per-edge attack success rates and intercepting approximately 3x the successful attacks as random allocation when monitoring the top 10% of ranked edges. The INFA-Guard framework [437] introduces infection-aware detection that explicitly identifies infected agents—benign entities converted by attack agents—as a distinct threat category, localizing attack sources and infected ranges before replacing attackers and rehabilitating infected agents, reducing attack success rates by an average of 33% with cross-model robustness and high cost-effectiveness. For protecting against coordinated attacks, the MASTrike framework [438] provides a closed-loop red-teaming framework using agent-level Shapley value analysis to quantify each agent’s marginal contribution to system robustness, identifying vulnerable agent coalitions and generating coordinated role-aware adversarial manipulations—serving as a defensive tool for proactively identifying and hardening critical vulnerabilities. Studied defense frameworks also include more specialized security mechanisms. For instance, the OpenEvoShield framework [439] addresses the challenge of non-stationary attacks through a co-evolutionary continual defense framework that decouples fast attack-side and slow normal-side learning rates from dual drift signals, maintaining a dynamic behavioral boundary while employing an energy-based multi-granularity detector that fuses node-, subgraph-, and graph-level evidence to classify novel attacks as out-of-distribution.

Access-control and verification-centric solutions provide a complementary layer of defense. The BlindGuard framework [440] demonstrates that effective detection is possible without attack-specific labels by establishing a hierarchical agent encoder capturing individual, neighborhood, and global interaction patterns, trained solely on normal agent behaviors through corruption-guided contrastive learning—achieving superior generalizability compared to supervised baselines across diverse attack types including prompt injection, memory poisoning, and tool attacks. The MAPLE-Guard and AgentSafe approaches exemplify access-control-centric strategies, while the PBFT-Backed Semantic Voting protocol [435] applies Byzantine fault tolerance to memory management decisions. However, the empirical evaluation of defense robustness across architectural layers in [433] cautions that defense effectiveness is highly sensitive to threat-model assumptions: an initial empty-corpus screen showing 0/210 exfiltrations was revealed to be an artifact of the threat model rather than model

safety, with realistic loaded-corpus conditions exposing 95% attack success rates for Gemini 3.1 Pro Preview and 22.5% for GPT-5.1—underscoring the critical importance of realistic evaluation conditions for defense validation, a concern that directly motivates the evaluation methodology frameworks discussed in the following section.

This diversity of defense mechanisms highlights a fundamental trade-off triangle spanning security, efficiency, and utility. Architecture Matters for Multi-Agent Security finds that no single architectural design is universally safer, and the binary paradigm in existing approaches is increasingly inadequate. The propagation-aware PropGuard framework [441] addresses this by constructing a dual-view spatio-temporal graph combining response-centric risk estimation with full-state evidence preservation, using a GE-GRPO trained inspector to recover compact suspicious propagation subgraphs and applying source-guided remediation that corrects upstream contamination and replays affected downstream interactions—demonstrating a favorable effectiveness-efficiency trade-off. The AcMAS framework [442] further advances reactive detection by analyzing internal reasoning states in the activation space of local agents, detecting stealthy attacks in a synchronization-robust fashion without relying on explicit interaction graphs, outperforming graph-based baselines by +0.22 F1 in synchronous settings and +0.55 F1 in asynchronous settings, and guiding restoration of compromised agents rather than disruptive isolation. The Adaptive Defense Orchestration architecture [443] demonstrates that adaptive query-aware defense can substantially reduce the security-utility trade-off in RAG systems: eliminating membership inference leakage while restoring contextual recall to more than 75% of the undefended baseline. However, the Trojan Hippo analysis [413] cautions that even effective defenses incur utility costs that vary widely with task requirements, and the security-utility trade-off remains an open challenge. A potential synthesis is offered by MASTER [444], which integrates both attack generation and defense strategies across multiple MAS setups, demonstrating that scenario-adaptive, extensible frameworks can substantially enhance MAS resilience across diverse scenarios. Understanding these trade-offs—and the precise conditions under which each defense mechanism is most effective—remains a critical research frontier in building robust and practical LLM-based multi-agent systems. This empirical complexity, and the need for rigorous validation of defense claims across architectures and threat models, establishes the central motivation for the evaluation frameworks, metrics, and benchmarks that the following section examines in depth.

7.3 Evaluation, Verification, and Red-Teaming for Security

The evaluation of security and trustworthiness in LLM-based Multi-Agent Systems (LLM-MAS) requires methodologies that extend well beyond conventional single-model robustness testing, due to the distributed nature of reasoning, inter-agent communication, and emergent system-level behaviors. Unlike traditional LLM evaluation that focuses on input-output pairs, MAS assessment must account for the collaborative decision-making processes, information propagation patterns, and the unique attack surfaces introduced by agent interactions. This subsection reviews the landscape of evaluation frameworks, metrics, benchmarks, and theoretical foundations that collectively establish a rigorous basis for assessing the security posture of LLM-MAS. In doing so, it bridges the preceding discussion of defense mechanisms—which established architectural design and runtime countermeasures as first-order security parameters—and the subsequent examination of governance frameworks, which presupposes the ability to measure, audit, and verify the security properties that evaluation methodologies are designed to expose.

A central challenge in LLM-MAS security evaluation lies in moving beyond outcome-only metrics toward process-oriented, architecture-aware assessment paradigms [445]. The reliability of MAS depends on multiple orthogonal dimensions—including in-distribution accuracy, distribution-

shift robustness, adversarial robustness, calibration, and out-of-distribution detection—each of which requires distinct measurement methodologies [445]. For multi-agent systems specifically, evaluation must additionally capture inter-agent dependencies, information flow integrity, and the extent to which individual agent vulnerabilities propagate through the network topology. This necessitates diagnostic frameworks that can attribute failures to specific agents or communication edges, rather than treating the system as a monolithic unit—a capability that directly complements the architectural and memory-focused defenses discussed earlier by enabling defenders to localize and remediate precisely where defensive mechanisms have failed. Trace-based verification frameworks and fault injection protocols provide a means to systematically probe system behavior under adversarial conditions, enabling the identification of critical vulnerabilities before deployment in high-stakes environments [446].

Benchmark suites play an indispensable role in standardizing security evaluation across different MAS architectures and configurations, providing the empirical substrate upon which the comparative claims of defense frameworks—such as the demonstrated vulnerabilities of multi-agent designs relative to single-agent baselines—can be validated and replicated. General-purpose evaluation frameworks enable consistent measurement of attack success rates, defense efficacy, and system degradation under diverse adversarial scenarios, supporting reproducibility and comparability of research findings. Standardized benchmarks have been developed that cover failure attribution, long-horizon degradation, and real-world deployment scenarios, providing a common ground for assessing both the capabilities and vulnerabilities of different MAS designs. However, the rapid evolution of both attack and defense techniques poses a persistent challenge for benchmark longevity, as static evaluation suites may quickly become outdated in the face of advancing adversarial methodologies [447]. This motivates the development of dynamically updated benchmark suites that incorporate newly discovered attack patterns and threat models. Notably, an important empirical question is whether MAS offer genuine value over strong single-agent baselines, as some comparative studies suggest diminishing returns or even degraded performance under certain conditions [448]. The capability paradox—where stronger individual agents can inadvertently decrease system-level security due to increased linguistic certainty and persuasive power—underscores the need for evaluation frameworks that measure not just component capabilities but also emergent system-level properties [448], echoing the architectural-first observation from the prior subsection that configuration choices themselves are security parameters.

Metrics for measuring attack success and defense efficacy in LLM-MAS extend beyond conventional accuracy and error rate. Attack Success Rate (ASR) remains a primary indicator but must be complemented with finer-grained measurements, such as the average number of attempts required to elicit harmful behaviors, which reveals that attack difficulty can vary by over 300-fold across different models despite universal vulnerability [449]. Fast proxy metrics have been proposed to predict real-world robustness against computationally expensive attack ensembles without requiring full execution, achieving high correlation with complete attack evaluations while reducing computational cost by orders of magnitude [450]. This approach is particularly valuable for MAS, where the combinatorial space of agent interactions makes exhaustive adversarial testing computationally prohibitive—a practical consideration that aligns with the efficiency-security trade-offs highlighted in the defense-oriented frameworks of the previous subsection. For robustness evaluation specifically, metrics such as adversarial distance—the minimum perturbation required to cause misclassification—provide more informative measures than accuracy at fixed perturbation budgets [446]. Extending these concepts to MAS requires defining analogous distances for agent communications and coordination protocols, accounting for the multi-agent decision-making process, and thereby enabling the direct quantification of how much an attacker must compromise a communication channel or agent to achieve system-level failure.

Adversarial simulation and automated red-teaming constitute essential components of LLM-MAS security evaluation, serving as the empirical engine that validates the effectiveness of the defensive mechanisms surveyed previously. Automated red-teaming methods have evolved from simple prompt templates toward sophisticated frameworks that formalize the attack process as a Markov Decision Process and employ hierarchical reinforcement learning to learn coherent multi-turn attack strategies [451]. Such approaches enable systematic exploration of the agent interaction space, uncovering vulnerabilities that single-step attacks would miss—vulnerabilities of precisely the kind that architectural defenses, memory guards, and access-control mechanisms aim to close. Empirical evidence from large-scale red-teaming competitions demonstrates that automated approaches significantly outperform manual techniques in terms of success rate, albeit with trade-offs in time-to-solve for certain creative reasoning challenges [452]. For MAS specifically, closed-loop frameworks such as MASTrike enable collusive red-teaming that accounts for coordinated adversarial behaviors across multiple compromised agents, using Shapley value analysis to identify which agents contribute most to system robustness or vulnerability [438]. Structured causal diagnosis within such frameworks supports iterative refinement of attack strategies, moving from heuristic selection toward principled attribution of system failures, and providing actionable intelligence that can be fed back into the architectural design principles discussed in the preceding subsection. Furthermore, the use of game-theoretic formulations has been proposed to model the interplay between system designers and adversaries, wherein the system designer iteratively proposes agent architectures while the adversary selects optimal attack strategies, leading to adversarially robust system designs that generalize to unseen threats [453].

Formal and theoretical approaches provide principled foundations for evaluating LLM-MAS robustness that complement empirical testing and offer the strongest forms of assurance required for governance and auditability. Information-theoretic frameworks quantify the amount of information leaked through observable signals, such as answer tokens, thinking process tokens, or logits, establishing fundamental bounds on attack efficiency and enabling auditors to assess how close their methods approach theoretical limits [454]. Such bounds inform deployment decisions about the trade-off between transparency and security, revealing that even modest increases in disclosed information can collapse attack costs dramatically—a finding directly relevant to the accountability and audit-trail mechanisms discussed in the following subsection, which require transparency as a precondition for oversight. Formal security games, which define completeness and adversarial robustness objectives within a unified model, extend cryptographic methodology to agentic systems, enabling rigorous reasoning about confidentiality, integrity, and availability guarantees [455]. Probability-approximately-correct (PAC) bounds and statistical verification methods offer another avenue for quantifying robustness with provable confidence guarantees, applicable in settings where formal verification is computationally infeasible [456]. Statistical runtime verification frameworks adapt formal methods to black-box deployment settings, providing quantitative robustness assessments with statistically validated bounds that achieve accuracy comparable to formal verification while reducing verification time from hours to minutes [403]. For mission-critical applications such as autonomous cyber defense, machine-checked guarantees (e.g., using Lean 4 with zero unsolved proof obligations) can certify the controllability, observability, and input-to-state stability of tool-mediated agent architectures, providing the strongest form of assurance for safety-critical MAS deployments [457]. Assurance cases, constructed through ontology-driven argumentation, offer a structured approach to documenting and justifying the robustness of LLM-MAS, establishing the evidentiary chain from attack and defense mechanisms to system-level assurance claims [458]—an evidentiary chain that governance frameworks in the following subsection depend upon for verifiable compliance and auditable decision-making.

Finally, defense-oriented evaluation frameworks are essential for assessing not only whether

defenses are effective but also under what conditions they may fail, thereby closing the loop between the defense mechanisms surveyed previously and the governance architectures that follow. Embedding-based defenses, for instance, exhibit fundamental limitations when attackers craft messages whose embeddings lie close to benign ones, suggesting that token-level confidence signals should complement embedding-based detection [459], a finding that reinforces the architectural-first conclusion from the prior subsection that defense effectiveness is determined more by structural placement than by classifier sophistication. Reliability assessments must therefore account for both adversarial perturbations and non-adversarial environmental shifts, as robustness evaluations reveal that designing for one metric does not necessarily constrain others, and certain algorithmic techniques can improve reliability across multiple dimensions simultaneously [445]. The evaluation of LLM-MAS security remains an evolving field that requires continuous adaptation of assessment methodologies to keep pace with the rapidly developing landscape of attacks, defenses, and system architectures, and it is precisely this adaptive measurement capability that enables the governance mechanisms of the following subsection to move from static policy assertion to dynamic, verifiable, and continuously auditable accountability.

7.4 Governance, Alignment, and Responsible AI Frameworks

The governance of LLM-based Multi-Agent Systems (LLM-MAS) extends far beyond the technical mechanisms of security and robustness surveyed in the previous subsection, encompassing a complex interplay of socio-technical factors that ensure these systems operate safely, ethically, and accountably. While the preceding discussion established evaluation methodologies as the empirical basis for assessing security posture, governance frameworks presuppose exactly this measurement capability—they require the ability to verify, audit, and enforce the security properties that evaluation techniques are designed to expose. The shift from single-agent to multi-agent paradigms fundamentally alters the nature of responsibility, as outcomes emerge from distributed interactions rather than isolated decisions. A central position in this discourse is that ensuring responsible behavior requires a paradigm shift from local, superficial agent-level alignment to global, systemic agreement, conceptualizing responsibility as a lifecycle-wide property encompassing agreement, uncertainty, and security [10]. This perspective views LLM-MAS not as loose collections of agents but as unified, dynamic socio-technical systems requiring principled mechanisms for sustained, system-wide agreement [10]. Consequently, governance frameworks must move beyond static constraints to become embedded, enforceable, and continuously auditable properties of the system’s architecture [460]—a requirement that directly extends the architectural-first principle established in the defense mechanisms discussion and operationalizes the adaptive measurement capabilities described in the evaluation subsection.

The foundation of trust in autonomous agent ecosystems begins with verifiable identity and accountability, which serves as the governance counterpart to the trace-based verification frameworks discussed earlier. Without a clear answer to “who or what is the agent?” and “can its actions be verified, audited, and trusted?”, the deployment of LLM-MAS in high-stakes environments remains precarious, and the audit trails and fault injection protocols from the evaluation subsection lack a substrate upon which to operate [461]. To address this, emerging frameworks propose decentralized identity layers and ethical consensus protocols embedded at the protocol level itself [461]. This is complemented by a growing emphasis on audit trails and provenance tracking as sociotechnical mechanisms for continuous accountability. An audit trail, defined as a chronological, tamper-evident, context-rich ledger of lifecycle events and decisions, links technical provenance (models, data, training runs) with governance records (approvals, waivers), allowing organizations to reconstruct what changed, when, and who authorized it [462]. The core principle is that no

agent system can be truly accountable without auditability, which requires action recoverability, lifecycle coverage, policy checkability, responsibility attribution, and evidence integrity [463]. To operationalize this, frameworks propose dual-layer identifiers with machine-verifiable primary hashes and human-readable secondary identifiers anchored in tamper-resistant registries, supported by zero-knowledge proof-based verification and post-deployment structural change screening [464]. Similarly, the TrustTrack protocol embeds structural guarantees—verifiable identity, policy commitments, and tamper-resistant behavioral logs—directly into agent infrastructure, treating compliance as a design constraint rather than post-hoc oversight [465]. These mechanisms directly complement the formal verification and information-theoretic bounds discussed in the evaluation subsection, translating abstract security guarantees into operationally enforceable accountability structures.

Lifecycle governance models provide a structured approach to managing the evolving nature of agentic AI, closely mirroring the dynamic benchmark evolution and continuous adaptation emphasized in the evaluation subsection. Frameworks such as AGENTSAFE operationalize risk taxonomies into design, runtime, and audit controls, profiling agentic loops and mapping risks onto structured taxonomies to provide pre-deployment assurance and continuous governance through semantic telemetry and dynamic authorization [466]. A lifecycle-wide perspective is also central to the Unified Agent Lifecycle Management (UALM) blueprint, which addresses agent sprawl in healthcare by mapping gaps onto five control-plane layers: an identity and persona registry, orchestration mediation, PHI-bounded context, runtime policy enforcement with kill-switch triggers, and lifecycle management linked to credential revocation [467]. Complementing these, the Social Responsibility Stack (SRS) models responsibility as a closed-loop supervisory control problem, embedding societal values as explicit constraints and integrating design-time safeguards with runtime monitoring and institutional oversight [468]. Such architectural stratification is argued to be structurally required, as safety dimensions concerning semantic intent, environmental validity, and dynamical feasibility depend on distinct information available at different execution stages, which no single guardrail can certify [469]. This structural stratification resonates with the process-oriented, architecture-aware assessment paradigms from the evaluation subsection, which likewise recognize that security and governance properties cannot be captured through monolithic approaches. Governance is further embedded through policies that intercept agents at structural checkpoints—such as intent guards, playbooks, tool guides, and human-in-the-loop gates—ensuring compliance is interwoven into the execution pipeline rather than being an afterthought [470]. For sustained governance, the alignment flywheel architecture decouples decision generation from safety governance, with a Safety Oracle providing raw safety signals through a stable interface, enabling runtime gating and updateable oversight without retraining the core decision component [471].

Transparency and explainability are indispensable for building trust and enabling effective oversight, serving as the governance-facing counterpart to the formal and information-theoretic evaluation methods described previously. Just as the evaluation subsection demonstrated that information leakage bounds can collapse attack costs, governance frameworks recognize that the opacity of multi-agent reasoning processes erodes trust and creates privacy concerns, motivating decentralized frameworks that decompose chain-of-thought reasoning into abstraction levels for parallel distributed auditing [472]. The “Governance by Metrics” paradigm treats explainability evaluation as a central mechanism of private AI governance, proposing a hierarchical model linking transparency, tamper resistance, and legal alignment to provide standardized, manipulation-resistant metrics for auditors and regulators [473]. Standardized evaluation is also central to frameworks like TRACE, which provides a metrologically-grounded engineering approach for trustworthy agentic AI in operationally critical domains, introducing a Model-Parsimony principle to ensure LLMs are used as a deliberate design choice rather than a default [474]. This is complemented by a methodology for auditable trustworthiness levels, which models governance-relative trustworthiness

through interpretable rules over trustworthiness profiles, yielding explicit plateaus and lifecycle diagnostics such as boundary margins and profile drift [475]. These standardized measurement approaches directly extend the evaluation metrics discussed in the previous subsection—such as attack success rates and adversarial distances—into the governance domain, providing the common vocabulary through which security evaluation and institutional oversight can communicate.

Privacy-preserving auditing and verifiable compliance are critical concerns, particularly when dealing with proprietary models and sensitive data, and they connect the evaluation subsection’s emphasis on efficient, computationally feasible assessment with the practical constraints of institutional oversight. Frameworks like LLM-FACETS address this by providing a browser-accessible interface that runs deterministic metrics entirely within a self-hosted server, with no outbound transmission, while making data flows explicit for LLM-judge metrics [476]. This mirrors the evaluation subsection’s recognition that computational feasibility is a critical constraint, particularly for the combinatorial space of multi-agent interactions. Broader frameworks integrate cryptographic tracing and organizational controls to enable measurable, auditable assurance across the entire lifecycle [466]. For reconstructing behavior from untrusted or incomplete evidence—a challenge directly analogous to the fault injection and failure attribution protocols in the evaluation subsection—iCORE (Integrated Cooperation-Obligation REpresentation) provides a unified encoding integrating observable interactions as a cooperation graph, evolving work assignments as an obligation graph, and an audit map linking them with verifiable properties, enabling the certification of work soundness and agent-assignment stability [477].

Human oversight and alignment remain the cornerstone of responsible AI, operationalizing the governance frameworks at the individual agent level while remaining consistent with the system-level evaluation approaches discussed previously. The neurocognitive governance model draws inspiration from human self-governance, formalizing a Pre-Action Governance Reasoning Loop where agents consult a four-layer governance rule set before consequential actions, embedding deliberation into agent reasoning to achieve consistent and auditable compliance [478]. This human-centric approach is echoed in frameworks that propose a dual-perspective governance combining interdisciplinary design with human-AI collaborative oversight [10]. For high-stakes deployments, structured design spaces that couple agency and autonomy into operational levels provide principled tactics—such as checkpoints, escalation, and write staging—to reason about oversight and action consequences [479]. These human oversight mechanisms complement the automated red-teaming and adversarial simulation approaches from the evaluation subsection, recognizing that effective governance requires both machine-speed detection and human-level judgment.

However, the emergence of multi-agent systems introduces new modes of failure that challenge traditional alignment and accountability mechanisms, and these failures demand the kind of process-oriented, system-level evaluation that the previous subsection established as essential. A key concern is multi-agent governance simulations, which show that governance structure can be a stronger driver of corruption-related outcomes than model identity, implying that institutional design is a precondition for safe delegation [480]. This finding directly parallels the evaluation subsection’s observation that architectural configuration choices themselves are security parameters. Similarly, constraint drift—the loss, distortion, or weakening of safety-critical constraints as they pass through memory, delegation, and communication—represents a fundamental challenge where safe behavior must be maintained, not merely asserted, necessitating constraint-native reinforcement learning within maintained safety boundaries [481]. Safety alignment itself can even create unpredictable liability risks, as safety-trained models may override deployment instructions in ways that conflict with organizational policies, a tension identified in benchmarks like ToolAlignBench [482]. Beyond LLM-to-human interactions, safety mechanisms designed for dyadic settings do not scale to LLM-to-LLM ecosystems where local compliance can aggregate into collective failure, motivating the

concept of an Emergent Systemic Risk Horizon and the design of architectures like InstitutionalAI for embedding adaptive oversight within agent networks [483]. These emergent failure modes are precisely what the evaluation subsection’s call for emergent system-level property measurement—as opposed to component-level assessment—was designed to capture, and they underscore why governance frameworks cannot be reduced to static policy assertions.

Finally, responsible deployment necessitates robust governance frameworks that span the entire lifecycle and bridge technical design with institutional responsibility, forming the culmination of the security-evaluation-governance continuum established across these subsections. A framework for Responsible AI Systems (RAIS) integrates five key dimensions—domain definition, trustworthy AI design, auditability, accountability, and governance—emphasizing their inter-dependencies and iterative feedback loops for proactive and reactive accountability [484]. This is complemented by practical frameworks like the six-decision framework for financial LLM integration, which guides institutions from feasibility assessment to deployment, integrating ethical considerations and audit trails [485]. The AgentHub registry exemplifies the need for discoverable, verifiable, and reproducible AI agents with canonical manifests and append-only lifecycle event logs, providing a substrate for building reliable and reusable agent ecosystems [486]—a registry that directly depends on the standardized benchmarks and evaluation metrics from the previous subsection for its verification capabilities. Ultimately, the path forward requires a shift from local, superficial agent-level alignment to global, systemic agreement, achieved through interdisciplinary design, human-AI collaborative oversight, and principled engineering that treats governance not as an external layer but as an intrinsic, verifiable property of the agentic system architecture [10, 468]. This systemic view of governance, built upon the evaluation methodologies and defense mechanisms discussed in the preceding subsection, provides the institutional accountability layer that transforms measured security properties into verifiable societal trust.

References

- [1] Multi-Agent Collaboration Mechanisms A Survey of LLMs. arXiv:2501.06322v1, 2025. <https://arxiv.org/abs/2501.06322v1>
- [2] Beyond Individual Intelligence Surveying Collaboration, Failure Attribution, and Self-Evolution in LLM-based Multi-Agent Systems. arXiv:2605.14892v2, 2026. <https://arxiv.org/abs/2605.14892v2>
- [3] Multi-Agent Systems From Classical Paradigms to Large Foundation Model-Enabled Futures. arXiv:2604.18133v1, 2026. <https://arxiv.org/abs/2604.18133v1>
- [4] Contemporary Agent Technology LLM-Driven Advancements vs Classic Multi-Agent Systems. arXiv:2509.02515v1, 2025. <https://arxiv.org/abs/2509.02515v1>
- [5] Large Language Models Miss the Multi-Agent Mark. arXiv:2505.21298v4, 2025. <https://arxiv.org/abs/2505.21298v4>
- [6] Game-Theoretic Lens on LLM-based Multi-Agent Systems. arXiv:2601.15047v1, 2026. <https://arxiv.org/abs/2601.15047v1>
- [7] Beyond Self-Talk A Communication-Centric Survey of LLM-Based Multi-Agent Systems. arXiv:2502.14321v3, 2025. <https://arxiv.org/abs/2502.14321v3>

- [8] Why do AI agents communicate in human language. arXiv:2506.02739v1, 2025. <https://arxiv.org/abs/2506.02739v1>
- [9] The Bystander Effect in Multi-Agent Reasoning Quantifying Cognitive Loading in Collaborative Interactions. arXiv:2605.10698v1, 2026. <https://arxiv.org/abs/2605.10698v1>
- [10] Stop Reducing Responsibility in LLM-Powered Multi-Agent Systems to Local Alignment. arXiv:2510.14008v2, 2025. <https://arxiv.org/abs/2510.14008v2>
- [11] Advances and Challenges in Foundation Agents From Brain-Inspired Intelligence to Evolutionary, Collaborative, and Safe Systems. arXiv:2504.01990v2, 2025. <https://arxiv.org/abs/2504.01990v2>
- [12] A Comprehensive Review of AI Agents Transforming Possibilities in Technology and Beyond. arXiv:2508.11957v1, 2025. <https://arxiv.org/abs/2508.11957v1>
- [13] Institutional Metaphors for Designing Large-Scale Distributed AI versus AI Techniques for Running Institutions. arXiv:1803.03407v2, 2018. <https://arxiv.org/abs/1803.03407v2>
- [14] The Path Ahead for Agentic AI Challenges and Opportunities. arXiv:2601.02749v1, 2026. <https://arxiv.org/abs/2601.02749v1>
- [15] A Survey of Self-Evolving Agents What, When, How, and Where to Evolve on the Path to Artificial Super Intelligence. arXiv:2507.21046v4, 2025. <https://arxiv.org/abs/2507.21046v4>
- [16] AgentVerse Facilitating Multi-Agent Collaboration and Exploring Emergent Behaviors. arXiv:2308.10848v3, 2023. <https://arxiv.org/abs/2308.10848v3>
- [17] Distinguishing Autonomous AI Agents from Collaborative Agentic Systems A Comprehensive Framework for Understanding Modern Intelligent Architectures. arXiv:2506.01438v2, 2025. <https://arxiv.org/abs/2506.01438v2>
- [18] Adaptive and Resource-efficient Agentic AI Systems for Mobile and Embedded Devices A Survey. arXiv:2510.00078v1, 2025. <https://arxiv.org/abs/2510.00078v1>
- [19] Is the ‘Agent’ Paradigm a Limiting Framework for Next-Generation Intelligent Systems. arXiv:2509.10875v1, 2025. <https://arxiv.org/abs/2509.10875v1>
- [20] Large Language Model Agent A Survey on Methodology, Applications and Challenges. arXiv:2503.21460v1, 2025. <https://arxiv.org/abs/2503.21460v1>
- [21] Towards a Science of Collective AI LLM-based Multi-Agent Systems Need a Transition from Blind Trial-and-Error to Rigorous Science. arXiv:2602.05289v1, 2026. <https://arxiv.org/abs/2602.05289v1>
- [22] Organizing a Society of Language Models Structures and Mechanisms for Enhanced Collective Intelligence. arXiv:2405.03825v1, 2024. <https://arxiv.org/abs/2405.03825v1>
- [23] Co-Saving Resource Aware Multi-Agent Collaboration for Software Development. arXiv:2505.21898v1, 2025. <https://arxiv.org/abs/2505.21898v1>
- [24] Decision Protocols in Multi-Agent Large Language Model Conversations. arXiv:2607.05477v1, 2026. <https://arxiv.org/abs/2607.05477v1>

- [25] Towards Cognitive Synergy in LLM-Based Multi-Agent Systems Integrating Theory of Mind and Critical Evaluation. arXiv:2507.21969v1, 2025. <https://arxiv.org/abs/2507.21969v1>
- [26] Revisiting Multi-Agent Debate as Test-Time Scaling A Systematic Study of Conditional Effectiveness. arXiv:2505.22960v2, 2025. <https://arxiv.org/abs/2505.22960v2>
- [27] On the Importance of Task Complexity in Evaluating LLM-Based Multi-Agent Systems. arXiv:2510.04311v1, 2025. <https://arxiv.org/abs/2510.04311v1>
- [28] Scaling Behavior of Single LLM-Driven Multi-Agent Systems. arXiv:2606.00655v1, 2026. <https://arxiv.org/abs/2606.00655v1>
- [29] Evolve as a Team Collaborative Self-Evolution for LLM-based Multi-Agent Systems. arXiv:2605.29790v1, 2026. <https://arxiv.org/abs/2605.29790v1>
- [30] X-MAS Towards Building Multi-Agent Systems with Heterogeneous LLMs. arXiv:2505.16997v1, 2025. <https://arxiv.org/abs/2505.16997v1>
- [31] Understanding Agent Scaling in LLM-Based Multi-Agent Systems via Diversity. arXiv:2602.03794v1, 2026. <https://arxiv.org/abs/2602.03794v1>
- [32] Byzantine-Robust Decentralized Coordination of LLM Agents. arXiv:2507.14928v1, 2025. <https://arxiv.org/abs/2507.14928v1>
- [33] More Capable, Less Cooperative When LLMs Fail At Zero-Cost Collaboration. arXiv:2604.07821v2, 2026. <https://arxiv.org/abs/2604.07821v2>
- [34] Agents that Matter Optimizing Multi-Agent LLMs via Removal-Based Attribution. arXiv:2605.27621v1, 2026. <https://arxiv.org/abs/2605.27621v1>
- [35] HiveMind Contribution-Guided Online Prompt Optimization of LLM Multi-Agent Systems. arXiv:2512.06432v1, 2025. <https://arxiv.org/abs/2512.06432v1>
- [36] Enhancing Decision-Making with Large Language Models through Multi-Agent Fictitious Play. arXiv:2606.19308v1, 2026. <https://arxiv.org/abs/2606.19308v1>
- [37] LLM-Hanabi Evaluating Multi-Agent Gameplays with Theory-of-Mind and Rationale Inference in Imperfect Information Collaboration Game. arXiv:2510.04980v1, 2025. <https://arxiv.org/abs/2510.04980v1>
- [38] AgentCDM Enhancing Multi-Agent Collaborative Decision-Making via ACH-Inspired Structured Reasoning. arXiv:2508.11995v1, 2025. <https://arxiv.org/abs/2508.11995v1>
- [39] Can Lessons From Human Teams Be Applied to Multi-Agent Systems The Role of Structure, Diversity, and Interaction Dynamics. arXiv:2510.07488v2, 2025. <https://arxiv.org/abs/2510.07488v2>
- [40] CollabSim A CSCW-Grounded Methodology for Investigating Collaborative Competence of LLM Agents through Controlled Multi-Agent Experiments. arXiv:2606.06399v2, 2026. <https://arxiv.org/abs/2606.06399v2>
- [41] A Review of Prominent Paradigms for LLM-Based Agents Tool Use (Including RAG), Planning, and Feedback Learning. arXiv:2406.05804v6, 2024. <https://arxiv.org/abs/2406.05804v6>

- [42] PartnerMAS An LLM Hierarchical Multi-Agent Framework for Business Partner Selection on High-Dimensional Features. arXiv:2509.24046v2, 2025. <https://arxiv.org/abs/2509.24046v2>
- [43] MorphAgent Empowering Agents through Self-Evolving Profiles and Decentralized Collaboration. arXiv:2410.15048v2, 2024. <https://arxiv.org/abs/2410.15048v2>
- [44] SIGMA Skill-Incidence Graphs for Compositional Multi-Agent Design. arXiv:2606.19758v1, 2026. <https://arxiv.org/abs/2606.19758v1>
- [45] Benchmarking Emergent Coordination in Large-Scale LLM Populations An Evaluation Framework on the MoltBook Archive. arXiv:2603.03555v3, 2026. <https://arxiv.org/abs/2603.03555v3>
- [46] A Taxonomy of Hierarchical Multi-Agent Systems Design Patterns, Coordination Mechanisms, and Industrial Applications. arXiv:2508.12683v1, 2025. <https://arxiv.org/abs/2508.12683v1>
- [47] AgentConductor Topology Evolution for Multi-Agent Competition-Level Code Generation. arXiv:2602.17100v1, 2026. <https://arxiv.org/abs/2602.17100v1>
- [48] DRAMA Next-Gen Dynamic Orchestration for Resilient Multi-Agent Ecosystems in Flux. arXiv:2508.04332v3, 2025. <https://arxiv.org/abs/2508.04332v3>
- [49] Scaling LLM-Driven Multi-Agent Systems Design Principles and Architectural Scalability Analysis. arXiv:2607.27942v1, 2026. <https://arxiv.org/abs/2607.27942v1>
- [50] A Two-Dimensional Framework for AI Agent Design Patterns Cognitive Function and Execution Topology. arXiv:2605.13850v2, 2026. <https://arxiv.org/abs/2605.13850v2>
- [51] Hierarchical Adaptive Consensus Network A Dynamic Framework for Scalable Consensus in Collaborative Multi-Agent AI Systems. arXiv:2511.17586v1, 2025. <https://arxiv.org/abs/2511.17586v1>
- [52] HierMAS Optimizing Intra-Node LLM Mixtures and Inter-Node Topology for Multi-Agent Systems. arXiv:2602.20229v1, 2026. <https://arxiv.org/abs/2602.20229v1>
- [53] Symphony A Decentralized Multi-Agent Framework for Scalable Collective Intelligence. arXiv:2508.20019v1, 2025. <https://arxiv.org/abs/2508.20019v1>
- [54] Matrix Peer-to-Peer Multi-Agent Synthetic Data Generation Framework. arXiv:2511.21686v2, 2025. <https://arxiv.org/abs/2511.21686v2>
- [55] AgentNet Decentralized Evolutionary Coordination for LLM-based Multi-Agent Systems. arXiv:2504.00587v2, 2025. <https://arxiv.org/abs/2504.00587v2>
- [56] CTHA Constrained Temporal Hierarchical Architecture for Stable Multi-Agent LLM Systems. arXiv:2601.10738v1, 2026. <https://arxiv.org/abs/2601.10738v1>
- [57] Designing Domain-Specific Agents via Hierarchical Task Abstraction Mechanism. arXiv:2511.17198v1, 2025. <https://arxiv.org/abs/2511.17198v1>
- [58] When Do Multi-Agent Systems Help An Information Bottleneck Perspective. arXiv:2607.16133v1, 2026. <https://arxiv.org/abs/2607.16133v1>

- [59] Rethinking Multi-Agent Intelligence Through the Lens of Small-World Networks. arXiv:2512.18094v1, 2025. <https://arxiv.org/abs/2512.18094v1>
- [60] GoAgent Group-of-Agents Communication Topology Generation for LLM-based Multi-Agent Systems. arXiv:2603.19677v1, 2026. <https://arxiv.org/abs/2603.19677v1>
- [61] Adaptive Orchestration Scalable Self-Evolving Multi-Agent Systems. arXiv:2601.09742v1, 2026. <https://arxiv.org/abs/2601.09742v1>
- [62] AdaptOrch Task-Adaptive Multi-Agent Orchestration in the Era of LLM Performance Convergence. arXiv:2602.16873v1, 2026. <https://arxiv.org/abs/2602.16873v1>
- [63] Beyond tokens a unified framework for latent communication in LLM-based multi-agent systems. arXiv:2606.05711v3, 2026. <https://arxiv.org/abs/2606.05711v3>
- [64] A survey of agent interoperability protocols Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A), and Agent Network Protocol (ANP). arXiv:2505.02279v2, 2025. <https://arxiv.org/abs/2505.02279v2>
- [65] Agentic AI Frameworks Architectures, Protocols, and Design Challenges. arXiv:2508.10146v1, 2025. <https://arxiv.org/abs/2508.10146v1>
- [66] AgentOrchestra Orchestrating Multi-Agent Intelligence with the Tool-Environment-Agent(TEA) Protocol. arXiv:2506.12508v6, 2025. <https://arxiv.org/abs/2506.12508v6>
- [67] LLM Agent Communication Protocol (LACP) Requires Urgent Standardization A Telecom-Inspired Protocol is Necessary. arXiv:2510.13821v1, 2025. <https://arxiv.org/abs/2510.13821v1>
- [68] A Technical Taxonomy of LLM Agent Communication Protocols. arXiv:2606.19135v1, 2026. <https://arxiv.org/abs/2606.19135v1>
- [69] Beyond Message Passing A Semantic View of Agent Communication Protocols. arXiv:2604.02369v3, 2026. <https://arxiv.org/abs/2604.02369v3>
- [70] Enabling Agents to Communicate Entirely in Latent Space. arXiv:2511.09149v5, 2025. <https://arxiv.org/abs/2511.09149v5>
- [71] KVComm Enabling Efficient LLM Communication through Selective KV Sharing. arXiv:2510.03346v3, 2025. <https://arxiv.org/abs/2510.03346v3>
- [72] Good Agentic Friends Do Not Just Give Verbal Advice They Can Update Your Weights. arXiv:2605.13839v1, 2026. <https://arxiv.org/abs/2605.13839v1>
- [73] Out of Sight, Not Out of Mind Unveiling Latent Attack in Latent-based Multi-Agent Systems. arXiv:2605.28214v1, 2026. <https://arxiv.org/abs/2605.28214v1>
- [74] LCGuard Latent Communication Guard for Safe KV Sharing in Multi-Agent Systems. arXiv:2605.22786v1, 2026. <https://arxiv.org/abs/2605.22786v1>
- [75] Phase-Scheduled Multi-Agent Systems for Token-Efficient Coordination. arXiv:2604.17400v1, 2026. <https://arxiv.org/abs/2604.17400v1>

- [76] A Layered Protocol Architecture for the Internet of Agents. arXiv:2511.19699v3, 2025. <https://arxiv.org/abs/2511.19699v3>
- [77] Thought Communication in Multiagent Collaboration. arXiv:2510.20733v1, 2025. <https://arxiv.org/abs/2510.20733v1>
- [78] What Should Agents Say Action-state Communication for Efficient Multi-Agent Systems. arXiv:2606.05304v1, 2026. <https://arxiv.org/abs/2606.05304v1>
- [79] CoopEval Benchmarking Cooperation-Sustaining Mechanisms and LLM Agents in Social Dilemmas. arXiv:2604.15267v2, 2026. <https://arxiv.org/abs/2604.15267v2>
- [80] Super-additive Cooperation in Language Model Agents. arXiv:2508.15510v1, 2025. <https://arxiv.org/abs/2508.15510v1>
- [81] Computational Foundations for Strategic Coopetition Formalizing Interdependence and Complementarity. arXiv:2510.18802v4, 2025. <https://arxiv.org/abs/2510.18802v4>
- [82] Voting Protocols as Coordination Mechanisms for Role-Constrained Multi-Agent Tutoring Systems. arXiv:2606.08030v1, 2026. <https://arxiv.org/abs/2606.08030v1>
- [83] Consensus is Strategically Insufficient Reasoning-Trace Disagreement as a Knowledge-Representation Signal. arXiv:2606.04223v1, 2026. <https://arxiv.org/abs/2606.04223v1>
- [84] The Hidden Strength of Disagreement Unraveling the Consensus-Diversity Tradeoff in Adaptive Multi-Agent Systems. arXiv:2502.16565v2, 2025. <https://arxiv.org/abs/2502.16565v2>
- [85] LLM Agents for Deliberative Collaboration A Study on Joint Decision Making Under Partial Observability. arXiv:2607.06157v1, 2026. <https://arxiv.org/abs/2607.06157v1>
- [86] From Debate to Deliberation Structured Collective Reasoning with Typed Epistemic Acts. arXiv:2603.11781v1, 2026. <https://arxiv.org/abs/2603.11781v1>
- [87] DR. WELL Dynamic Reasoning and Learning with Symbolic World Model for Embodied LLM-Based Multi-Agent Collaboration. arXiv:2511.04646v1, 2025. <https://arxiv.org/abs/2511.04646v1>
- [88] LLM Strategic Reasoning Agentic Study through Behavioral Game Theory. arXiv:2502.20432v3, 2025. <https://arxiv.org/abs/2502.20432v3>
- [89] Multi-Agent Strategic Games with LLMs. arXiv:2605.03604v1, 2026. <https://arxiv.org/abs/2605.03604v1>
- [90] Dynamic Strategy Adaptation in Multi-Agent Environments with Large Language Models. arXiv:2507.02002v4, 2025. <https://arxiv.org/abs/2507.02002v4>
- [91] Cooperation Breakdown in LLM Agents Under Communication Delays. arXiv:2602.11754v1, 2026. <https://arxiv.org/abs/2602.11754v1>
- [92] Communication-Efficient Digital-Twin Coordination for Heterogeneous LLM Embodied Agents over Computing Power Networks. arXiv:2607.09330v1, 2026. <https://arxiv.org/abs/2607.09330v1>

- [93] Exploration enhances cooperation in the multi-agent communication system. arXiv:2603.01401v1, 2026. <https://arxiv.org/abs/2603.01401v1>
- [94] Improving the Efficiency of Language Agent Teams with Adaptive Task Graphs. arXiv:2605.06320v1, 2026. <https://arxiv.org/abs/2605.06320v1>
- [95] Adaptive Coopetition Leveraging Coarse Verifier Signals for Resilient Multi-Agent LLM Reasoning. arXiv:2510.18179v2, 2025. <https://arxiv.org/abs/2510.18179v2>
- [96] Feedback-Coupled Memory Systems A Dynamical Model for Adaptive Coordination. arXiv:2603.11560v3, 2026. <https://arxiv.org/abs/2603.11560v3>
- [97] Coalition Formation in LLM Agent Networks Stability Analysis and Convergence Guarantees. arXiv:2604.14386v1, 2026. <https://arxiv.org/abs/2604.14386v1>
- [98] Self-Evolving Coordination Protocol in Multi-Agent AI Systems An Exploratory Systems Feasibility Study. arXiv:2602.02170v1, 2026. <https://arxiv.org/abs/2602.02170v1>
- [99] Modeling Response Consistency in Multi-Agent LLM Systems A Comparative Analysis of Shared and Separate Context Approaches. arXiv:2504.07303v1, 2025. <https://arxiv.org/abs/2504.07303v1>
- [100] Decentralized Multi-Agent Systems with Shared Context. arXiv:2606.10662v1, 2026. <https://arxiv.org/abs/2606.10662v1>
- [101] Rethinking the Value of Multi-Agent Workflow A Strong Single Agent Baseline. arXiv:2601.12307v1, 2026. <https://arxiv.org/abs/2601.12307v1>
- [102] SC-MAS Constructing Cost-Efficient Multi-Agent Systems with Edge-Level Heterogeneous Collaboration. arXiv:2601.09434v1, 2026. <https://arxiv.org/abs/2601.09434v1>
- [103] AgentBalance Backbone-then-Topology Design for Cost-Effective Multi-Agent Systems under Budget Constraints. arXiv:2512.11426v1, 2025. <https://arxiv.org/abs/2512.11426v1>
- [104] Too Many Specialists Emergent Inefficiencies and Bottlenecks for Multi-agent Ad-hoc Collaboration. arXiv:2605.08540v1, 2026. <https://arxiv.org/abs/2605.08540v1>
- [105] Communication to Completion Modeling Collaborative Workflows with Intelligent Multi-Agent Communication. arXiv:2510.19995v2, 2025. <https://arxiv.org/abs/2510.19995v2>
- [106] Towards Multi-Agent Reasoning Systems for Collaborative Expertise Delegation An Exploratory Design Study. arXiv:2505.07313v2, 2025. <https://arxiv.org/abs/2505.07313v2>
- [107] AgentGroupChat-V2 Divide-and-Conquer Is What LLM-Based Multi-Agent System Need. arXiv:2506.15451v1, 2025. <https://arxiv.org/abs/2506.15451v1>
- [108] The Illusion of Multi-Agent Advantage. arXiv:2606.13003v2, 2026. <https://arxiv.org/abs/2606.13003v2>
- [109] Single-Agent LLMs Outperform Multi-Agent Systems on Multi-Hop Reasoning Under Equal Thinking Token Budgets. arXiv:2604.02460v2, 2026. <https://arxiv.org/abs/2604.02460v2>

- [110] Multi-Agent Teams Hold Experts Back. arXiv:2602.01011v4, 2026. <https://arxiv.org/abs/2602.01011v4>
- [111] Towards a Science of Scaling Agent Systems. arXiv:2512.08296v3, 2025. <https://arxiv.org/abs/2512.08296v3>
- [112] Memory for Autonomous LLM Agents Mechanisms, Evaluation, and Emerging Frontiers. arXiv:2603.07670v1, 2026. <https://arxiv.org/abs/2603.07670v1>
- [113] Memory in the Age of AI Agents. arXiv:2512.13564v2, 2025. <https://arxiv.org/abs/2512.13564v2>
- [114] Memory for Large Language Models. arXiv:2607.25380v1, 2026. <https://arxiv.org/abs/2607.25380v1>
- [115] AI Meets Brain Memory Systems from Cognitive Neuroscience to Autonomous Agents. arXiv:2512.23343v1, 2025. <https://arxiv.org/abs/2512.23343v1>
- [116] HeLa-Mem Hebbian Learning and Associative Memory for LLM Agents. arXiv:2604.16839v1, 2026. <https://arxiv.org/abs/2604.16839v1>
- [117] The AI Hippocampus How Far are We From Human Memory. arXiv:2601.09113v1, 2026. <https://arxiv.org/abs/2601.09113v1>
- [118] G-Memory Tracing Hierarchical Memory for Multi-Agent Systems. arXiv:2506.07398v2, 2025. <https://arxiv.org/abs/2506.07398v2>
- [119] Hierarchical Memory for High-Efficiency Long-Term Reasoning in LLM Agents. arXiv:2507.22925v1, 2025. <https://arxiv.org/abs/2507.22925v1>
- [120] BMAM Brain-inspired Multi-Agent Memory Framework. arXiv:2601.20465v1, 2026. <https://arxiv.org/abs/2601.20465v1>
- [121] H²R Hierarchical Hindsight Reflection for Multi-Task LLM Agents. arXiv:2509.12810v1, 2025. <https://arxiv.org/abs/2509.12810v1>
- [122] HyperMem Hypergraph Memory for Long-Term Conversations. arXiv:2604.08256v2, 2026. <https://arxiv.org/abs/2604.08256v2>
- [123] Hierarchical Long-Term Semantic Memory for LinkedIn’s Hiring Agent. arXiv:2604.26197v3, 2026. <https://arxiv.org/abs/2604.26197v3>
- [124] Multi-Agent Memory from a Computer Architecture Perspective Visions and Challenges Ahead. arXiv:2603.10062v2, 2026. <https://arxiv.org/abs/2603.10062v2>
- [125] Governed Collaborative Memory as Artificial Selection in LLM-Based Multi-Agent Systems. arXiv:2605.04264v1, 2026. <https://arxiv.org/abs/2605.04264v1>
- [126] Collaborative Memory Multi-User Memory Sharing in LLM Agents with Dynamic Access Control. arXiv:2505.18279v1, 2025. <https://arxiv.org/abs/2505.18279v1>
- [127] Scaling Teams or Scaling Time Memory Enabled Lifelong Learning in LLM Multi-Agent Systems. arXiv:2604.03295v1, 2026. <https://arxiv.org/abs/2604.03295v1>

- [128] AutoMem Automated Learning of Memory as a Cognitive Skill. arXiv:2607.01224v1, 2026. <https://arxiv.org/abs/2607.01224v1>
- [129] Agentic Memory Learning Unified Long-Term and Short-Term Memory Management for Large Language Model Agents. arXiv:2601.01885v3, 2026. <https://arxiv.org/abs/2601.01885v3>
- [130] Experience-Evolving Multi-Turn Tool-Use Agent with Hybrid Episodic-Procedural Memory. arXiv:2512.07287v3, 2025. <https://arxiv.org/abs/2512.07287v3>
- [131] Remember Me, Refine Me A Dynamic Procedural Memory Framework for Experience-Driven Agent Evolution. arXiv:2512.10696v2, 2025. <https://arxiv.org/abs/2512.10696v2>
- [132] SKILL-DISCO Distilling and Compiling Agent Traces into Reusable Procedural Skills. arXiv:2606.26669v1, 2026. <https://arxiv.org/abs/2606.26669v1>
- [133] Experience Compression Spectrum Unifying Memory, Skills, and Rules in LLM Agents. arXiv:2604.15877v2, 2026. <https://arxiv.org/abs/2604.15877v2>
- [134] Useful Memories Become Faulty When Continuously Updated by LLMs. arXiv:2605.12978v1, 2026. <https://arxiv.org/abs/2605.12978v1>
- [135] Learning to Retrieve Dual-Level Long-Term Memory for Text-to-SQL Agents. arXiv:2606.00547v1, 2026. <https://arxiv.org/abs/2606.00547v1>
- [136] HiSkill Empowering LLM Agents with Hierarchical Skill Graphs. arXiv:2607.25853v1, 2026. <https://arxiv.org/abs/2607.25853v1>
- [137] SkillLens Adaptive Multi-Granularity Skill Reuse for Cost-Efficient LLM Agents. arXiv:2605.08386v1, 2026. <https://arxiv.org/abs/2605.08386v1>
- [138] Task Decomposition-Guided Reranking for Adaptive Agent Skill Retrieval. arXiv:2607.06283v1, 2026. <https://arxiv.org/abs/2607.06283v1>
- [139] Memory as a Controlled Process Learned Adaptive Memory Management for LLM Agents. arXiv:2607.13591v1, 2026. <https://arxiv.org/abs/2607.13591v1>
- [140] SkillEvolBench Benchmarking the Evolution from Episodic Experience to Procedural Skills. arXiv:2605.24117v1, 2026. <https://arxiv.org/abs/2605.24117v1>
- [141] Skill-Pro Learning Reusable Skills from Experience via Non-Parametric PPO for LLM Agents. arXiv:2602.01869v3, 2026. <https://arxiv.org/abs/2602.01869v3>
- [142] MAGE Multi-Agent Self-Evolution with Co-Evolutionary Knowledge Graphs. arXiv:2605.10064v1, 2026. <https://arxiv.org/abs/2605.10064v1>
- [143] Skill-MAS Evolving Meta-Skill for Automatic Multi-Agent Systems. arXiv:2606.18837v2, 2026. <https://arxiv.org/abs/2606.18837v2>
- [144] SkillsVote Lifecycle Governance of Agent Skills from Collection, Recommendation to Evolution. arXiv:2605.18401v2, 2026. <https://arxiv.org/abs/2605.18401v2>
- [145] SkillOS Learning Skill Curation for Self-Evolving Agents. arXiv:2605.06614v1, 2026. <https://arxiv.org/abs/2605.06614v1>

- [146] ReSkill Reconciling Skill Creation with Policy Optimization in Agentic RL. arXiv:2606.01619v2, 2026. <https://arxiv.org/abs/2606.01619v2>
- [147] Mem- α Learning Memory Construction via Reinforcement Learning. arXiv:2509.25911v1, 2025. <https://arxiv.org/abs/2509.25911v1>
- [148] Learning How and What to Memorize Cognition-Inspired Two-Stage Optimization for Evolving Memory. arXiv:2605.00702v1, 2026. <https://arxiv.org/abs/2605.00702v1>
- [149] Memory-R2 Fair Credit Assignment for Long-Horizon Memory-Augmented LLM Agents. arXiv:2605.21768v1, 2026. <https://arxiv.org/abs/2605.21768v1>
- [150] Joint Optimization of Multi-agent Memory System. arXiv:2603.12631v2, 2026. <https://arxiv.org/abs/2603.12631v2>
- [151] LERO LLM-driven Evolutionary framework with Hybrid Rewards and Enhanced Observation for Multi-Agent Reinforcement Learning. arXiv:2503.21807v1, 2025. <https://arxiv.org/abs/2503.21807v1>
- [152] MAGE Meta-Reinforcement Learning for Language Agents toward Strategic Exploration and Exploitation. arXiv:2603.03680v1, 2026. <https://arxiv.org/abs/2603.03680v1>
- [153] SEDM Scalable Self-Evolving Distributed Memory for Agents. arXiv:2509.09498v3, 2025. <https://arxiv.org/abs/2509.09498v3>
- [154] Self-Evolving Multi-Agent Systems via Decentralized Memory. arXiv:2605.22721v1, 2026. <https://arxiv.org/abs/2605.22721v1>
- [155] LatentMem Customizing Latent Memory for Multi-Agent Systems. arXiv:2602.03036v2, 2026. <https://arxiv.org/abs/2602.03036v2>
- [156] SkillMAS Skill Co-Evolution with LLM-based Multi-Agent System. arXiv:2605.09341v2, 2026. <https://arxiv.org/abs/2605.09341v2>
- [157] MemSkill Learning and Evolving Memory Skills for Self-Evolving Agents. arXiv:2602.02474v2, 2026. <https://arxiv.org/abs/2602.02474v2>
- [158] From Memory to Skills Evidence-Grounded Co-Evolution Governance for Long-Horizon LLM Agents. arXiv:2607.16621v1, 2026. <https://arxiv.org/abs/2607.16621v1>
- [159] ExpGraph Model-Agnostic Experience Learning with Graph-Structured Memory for LLM Agents. arXiv:2605.30712v1, 2026. <https://arxiv.org/abs/2605.30712v1>
- [160] MemEvolve Meta-Evolution of Agent Memory Systems. arXiv:2512.18746v1, 2025. <https://arxiv.org/abs/2512.18746v1>
- [161] EvolveMem Self-Evolving Memory Architecture via AutoResearch for LLM Agents. arXiv:2605.13941v1, 2026. <https://arxiv.org/abs/2605.13941v1>
- [162] RoMeRL Balancing Feedback Coverage and the Memory-Reward Trap in Self-Evolving Agent Memory via Reduced-Order Utility States. arXiv:2608.02508v1, 2026. <https://arxiv.org/abs/2608.02508v1>

- [163] MetaEvo A Meta-Optimization Framework for Experience-Driven Agent Evolution. arXiv:2606.07603v1, 2026. <https://arxiv.org/abs/2606.07603v1>
- [164] CoMAS Co-Evolving Multi-Agent Systems via Interaction Rewards. arXiv:2510.08529v2, 2025. <https://arxiv.org/abs/2510.08529v2>
- [165] Multi-Agent Evolve LLM Self-Improve through Co-evolution. arXiv:2510.23595v3, 2025. <https://arxiv.org/abs/2510.23595v3>
- [166] RetroAgent From Solving to Evolving via Retrospective Dual Intrinsic Feedback. arXiv:2603.08561v6, 2026. <https://arxiv.org/abs/2603.08561v6>
- [167] CoEvolve Training LLM Agents via Agent-Data Mutual Evolution. arXiv:2604.15840v1, 2026. <https://arxiv.org/abs/2604.15840v1>
- [168] Continual Learning in Large Language Models Methods, Challenges, and Opportunities. arXiv:2603.12658v1, 2026. <https://arxiv.org/abs/2603.12658v1>
- [169] Continual Learning of Large Language Models A Comprehensive Survey. arXiv:2404.16789v1, 2024. <https://arxiv.org/abs/2404.16789v1>
- [170] Fine-Tuning Regimes Define Distinct Continual Learning Problems. arXiv:2604.21927v3, 2026. <https://arxiv.org/abs/2604.21927v3>
- [171] LifelongAgentBench Evaluating LLM Agents as Lifelong Learners. arXiv:2505.11942v3, 2025. <https://arxiv.org/abs/2505.11942v3>
- [172] GeRe Towards Efficient Anti-Forgetting in Continual Learning of LLM via General Samples Replay. arXiv:2508.04676v1, 2025. <https://arxiv.org/abs/2508.04676v1>
- [173] MSSR Memory-Aware Adaptive Replay for Continual LLM Fine-Tuning. arXiv:2603.09892v1, 2026. <https://arxiv.org/abs/2603.09892v1>
- [174] FOREVER Forgetting Curve-Inspired Memory Replay for Language Model Continual Learning. arXiv:2601.03938v2, 2026. <https://arxiv.org/abs/2601.03938v2>
- [175] SOLAR A Self-Optimizing Open-Ended Autonomous Agent for Lifelong Learning and Continual Adaptation. arXiv:2605.20189v1, 2026. <https://arxiv.org/abs/2605.20189v1>
- [176] Learning While Acting A Skill-Enhanced Test-Time Co-Evolution Framework for Online Lifelong Learning Agents. arXiv:2606.04815v2, 2026. <https://arxiv.org/abs/2606.04815v2>
- [177] When Does Continual Learning Require Learning. arXiv:2607.07847v1, 2026. <https://arxiv.org/abs/2607.07847v1>
- [178] PATH-Bench Path-Dependent Evaluation of Lifelong Agents. arXiv:2608.01149v1, 2026. <https://arxiv.org/abs/2608.01149v1>
- [179] EvoAgentBench Benchmarking Agent Self-Evolution via Ability Transfer. arXiv:2607.05202v1, 2026. <https://arxiv.org/abs/2607.05202v1>
- [180] Continual Learning Bench Evaluating Frontier AI Systems in Real-World Stateful Environments. arXiv:2606.05661v1, 2026. <https://arxiv.org/abs/2606.05661v1>

- [181] Live-Evo Online Evolution of Agentic Memory from Continuous Feedback. arXiv:2602.02369v1, 2026. <https://arxiv.org/abs/2602.02369v1>
- [182] EvoArena Tracking Memory Evolution for Robust LLM Agents in Dynamic Environments. arXiv:2606.13681v2, 2026. <https://arxiv.org/abs/2606.13681v2>
- [183] Hierarchical Neuro-Symbolic Decision Transformer. arXiv:2503.07148v3, 2025. <https://arxiv.org/abs/2503.07148v3>
- [184] Overview A Hierarchical Framework for Plan Generation and Execution in Multi-Robot Systems. arXiv:1804.00038v1, 2018. <https://arxiv.org/abs/1804.00038v1>
- [185] LLM-guided Task and Motion Planning using Knowledge-based Reasoning. arXiv:2412.07493v3, 2024. <https://arxiv.org/abs/2412.07493v3>
- [186] Hierarchical Planning for Complex Tasks with Knowledge Graph-RAG and Symbolic Verification. arXiv:2504.04578v1, 2025. <https://arxiv.org/abs/2504.04578v1>
- [187] LOOP A Plug-and-Play Neuro-Symbolic Framework for Enhancing Planning in Autonomous Systems. arXiv:2508.13371v1, 2025. <https://arxiv.org/abs/2508.13371v1>
- [188] Deliberate Planning in Language Models with Symbolic Representation. arXiv:2505.01479v3, 2025. <https://arxiv.org/abs/2505.01479v3>
- [189] Causality Networks. arXiv:1406.6651v1, 2014. <https://arxiv.org/abs/1406.6651v1>
- [190] Causal Channels. arXiv:2103.02834v1, 2021. <https://arxiv.org/abs/2103.02834v1>
- [191] A Framework for Following Temporal Logic Instructions with Unknown Causal Dependencies. arXiv:2204.03196v3, 2022. <https://arxiv.org/abs/2204.03196v3>
- [192] ExoPredicate Learning Abstract Models of Dynamic Worlds for Robot Planning. arXiv:2509.26255v3, 2025. <https://arxiv.org/abs/2509.26255v3>
- [193] Healthcare. arXiv:1703.04524v1, 2017. <https://arxiv.org/abs/1703.04524v1>
- [194] CoEx – Co-evolving World-model and Exploration. arXiv:2507.22281v1, 2025. <https://arxiv.org/abs/2507.22281v1>
- [195] Compositional Planning with Jumpy World Models. arXiv:2602.19634v1, 2026. <https://arxiv.org/abs/2602.19634v1>
- [196] Generalized Multimodal ELBO. arXiv:2105.02470v2, 2021. <https://arxiv.org/abs/2105.02470v2>
- [197] SCOOP A Framework for Proactive Collaboration and Social Continual Learning through Natural Language Interaction and Causal Reasoning. arXiv:2503.10241v1, 2025. <https://arxiv.org/abs/2503.10241v1>
- [198] Planner Matters! An Efficient and Unbalanced Multi-agent Collaboration Framework for Long-horizon Planning. arXiv:2605.02168v1, 2026. <https://arxiv.org/abs/2605.02168v1>
- [199] The Visual Debugger Past, Present, and Future. arXiv:2403.03683v1, 2024. <https://arxiv.org/abs/2403.03683v1>

- [200] The h-index. arXiv:2112.02175v1, 2021. <https://arxiv.org/abs/2112.02175v1>
- [201] Temporal Planning via Interval Logic Satisfiability for Autonomous Systems. arXiv:2406.09661v1, 2024. <https://arxiv.org/abs/2406.09661v1>
- [202] Hulk A Universal Knowledge Translator for Human-Centric Tasks. arXiv:2312.01697v4, 2023. <https://arxiv.org/abs/2312.01697v4>
- [203] Debate or Vote Which Yields Better Decisions in Multi-Agent Large Language Models. arXiv:2508.17536v2, 2025. <https://arxiv.org/abs/2508.17536v2>
- [204] CHAL Council of Hierarchical Agentic Language. arXiv:2605.12718v1, 2026. <https://arxiv.org/abs/2605.12718v1>
- [205] DynaDebate Breaking Homogeneity in Multi-Agent Debate with Dynamic Path Generation. arXiv:2601.05746v2, 2026. <https://arxiv.org/abs/2601.05746v2>
- [206] When Disagreements Elicit Robustness Investigating Self-Repair Capabilities under LLM Multi-Agent Disagreements. arXiv:2502.15153v2, 2025. <https://arxiv.org/abs/2502.15153v2>
- [207] The Value of Variance Mitigating Debate Collapse in Multi-Agent Systems via Uncertainty-Driven Policy Optimization. arXiv:2602.07186v1, 2026. <https://arxiv.org/abs/2602.07186v1>
- [208] Voting or Consensus Decision-Making in Multi-Agent Debate. arXiv:2502.19130v4, 2025. <https://arxiv.org/abs/2502.19130v4>
- [209] RoundTable Investigating Group Decision-Making Mechanism in Multi-Agent Collaboration. arXiv:2411.07161v2, 2024. <https://arxiv.org/abs/2411.07161v2>
- [210] HCP-MAD Heterogeneous Consensus-Progressive Reasoning for Efficient Multi-Agent Debate. arXiv:2604.09679v2, 2026. <https://arxiv.org/abs/2604.09679v2>
- [211] Collaborative Disagreement Resolution for Scalable Oversight. arXiv:2607.01251v1, 2026. <https://arxiv.org/abs/2607.01251v1>
- [212] Self-Improvement of Language Models by Post-Training on Multi-Agent Debate. arXiv:2509.15172v3, 2025. <https://arxiv.org/abs/2509.15172v3>
- [213] The Cost of Consensus Isolated Self-Correction Prevails Over Unguided Homogeneous Multi-Agent Debate. arXiv:2605.00914v1, 2026. <https://arxiv.org/abs/2605.00914v1>
- [214] Talk Isn’t Always Cheap Understanding Failure Modes in Multi-Agent Debate. arXiv:2509.05396v2, 2025. <https://arxiv.org/abs/2509.05396v2>
- [215] A Survey on Hypergame Theory Modelling Misaligned Perceptions and Nested Beliefs for Multi-Agent Systems. arXiv:2507.19593v3, 2025. <https://arxiv.org/abs/2507.19593v3>
- [216] Hypergame Rationalisability Solving Agent Misalignment In Strategic Play. arXiv:2512.11942v1, 2025. <https://arxiv.org/abs/2512.11942v1>
- [217] Improving Quantal Cognitive Hierarchy Model Through Iterative Population Learning. arXiv:2302.06033v2, 2023. <https://arxiv.org/abs/2302.06033v2>

- [218] CHBench A Cognitive Hierarchy Benchmark for Evaluating Strategic Reasoning Capability of LLMs. arXiv:2508.11944v1, 2025. <https://arxiv.org/abs/2508.11944v1>
- [219] LLMs as Strategic Agents Beliefs, Best Response Behavior, and Emergent Heuristics. arXiv:2510.10813v1, 2025. <https://arxiv.org/abs/2510.10813v1>
- [220] FORM Version 5.0. arXiv:2601.19982v1, 2026. <https://arxiv.org/abs/2601.19982v1>
- [221] Quantal Response Equilibrium as a Measure of Strategic Sophistication Theory and Validation for LLM Evaluation. arXiv:2603.10029v1, 2026. <https://arxiv.org/abs/2603.10029v1>
- [222] ElementaryNet A Non-Strategic Neural Network for Predicting Human Behavior in Normal-Form Games. arXiv:2503.05925v3, 2025. <https://arxiv.org/abs/2503.05925v3>
- [223] GameBench Evaluating Strategic Reasoning Abilities of LLM Agents. arXiv:2406.06613v2, 2024. <https://arxiv.org/abs/2406.06613v2>
- [224] SPIN-Bench How Well Do LLMs Plan Strategically and Reason Socially. arXiv:2503.12349v5, 2025. <https://arxiv.org/abs/2503.12349v5>
- [225] WGSB-Bench Wargame-based Game-theoretic Strategic Reasoning Benchmark for Large Language Models. arXiv:2506.10264v1, 2025. <https://arxiv.org/abs/2506.10264v1>
- [226] TMGBench A Systematic Game Benchmark for Evaluating Strategic Reasoning Abilities of LLMs. arXiv:2410.10479v2, 2024. <https://arxiv.org/abs/2410.10479v2>
- [227] The Decrypto Benchmark for Multi-Agent Reasoning and Theory of Mind. arXiv:2506.20664v1, 2025. <https://arxiv.org/abs/2506.20664v1>
- [228] Improving Multi-Agent Cooperation using Theory of Mind. arXiv:2007.15703v1, 2020. <https://arxiv.org/abs/2007.15703v1>
- [229] Cognitive Insights and Stable Coalition Matching for Fostering Multi-Agent Cooperation. arXiv:2405.18044v2, 2024. <https://arxiv.org/abs/2405.18044v2>
- [230] Effects of Theory of Mind and Prosocial Beliefs on Steering Human-Aligned Behaviors of LLMs in Ultimatum Games. arXiv:2505.24255v1, 2025. <https://arxiv.org/abs/2505.24255v1>
- [231] Theory of Mind Using Active Inference A Framework for Multi-Agent Cooperation. arXiv:2508.00401v2, 2025. <https://arxiv.org/abs/2508.00401v2>
- [232] A Causal Model of Theory of Mind in Conflict for Artificial Intelligence. arXiv:2606.16944v2, 2026. <https://arxiv.org/abs/2606.16944v2>
- [233] Adaptive Theory of Mind for LLM-based Multi-Agent Coordination. arXiv:2603.16264v1, 2026. <https://arxiv.org/abs/2603.16264v1>
- [234] HI-TOM A Benchmark for Evaluating Higher-Order Theory of Mind Reasoning in Large Language Models. arXiv:2310.16755v1, 2023. <https://arxiv.org/abs/2310.16755v1>
- [235] Decompose-ToM Enhancing Theory of Mind Reasoning in Large Language Models through Simulation and Task Decomposition. arXiv:2501.09056v1, 2025. <https://arxiv.org/abs/2501.09056v1>

- [236] AutoToM Scaling Model-based Mental Inference via Automated Agent Modeling. arXiv:2502.15676v3, 2025. <https://arxiv.org/abs/2502.15676v3>
- [237] The Theory of Mind Utility Formal Specification of a Mentalizing Mechanism. arXiv:2606.12721v2, 2026. <https://arxiv.org/abs/2606.12721v2>
- [238] MIND Multi-agent inference for negotiation dialogue in travel planning. arXiv:2603.21696v1, 2026. <https://arxiv.org/abs/2603.21696v1>
- [239] PersuasiveToM A Benchmark for Evaluating Machine Theory of Mind in Persuasive Dialogues. arXiv:2502.21017v2, 2025. <https://arxiv.org/abs/2502.21017v2>
- [240] Think Thrice Before You Speak Dual knowledge-enhanced Theory-of-Mind Reasoning for Persuasive Agents. arXiv:2605.22602v1, 2026. <https://arxiv.org/abs/2605.22602v1>
- [241] NegotiationToM A Benchmark for Stress-testing Machine Theory of Mind on Negotiation Surrounding. arXiv:2404.13627v1, 2024. <https://arxiv.org/abs/2404.13627v1>
- [242] Resonating Minds – Emergent Collaboration Through Hierarchical Active Inference. arXiv:2112.01210v1, 2021. <https://arxiv.org/abs/2112.01210v1>
- [243] A Brain-inspired Theory of Collective Mind Model for Efficient Social Cooperation. arXiv:2311.03150v2, 2023. <https://arxiv.org/abs/2311.03150v2>
- [244] Readable Minds Emergent Theory-of-Mind-Like Behavior in LLM Poker Agents. arXiv:2604.04157v1, 2026. <https://arxiv.org/abs/2604.04157v1>
- [245] Model-free conventions in multi-agent reinforcement learning with heterogeneous preferences. arXiv:2010.09054v2, 2020. <https://arxiv.org/abs/2010.09054v2>
- [246] Emergent social conventions and collective bias in LLM populations. arXiv:2410.08948v2, 2024. <https://arxiv.org/abs/2410.08948v2>
- [247] Improved Cooperation by Exploiting a Common Signal. arXiv:2102.02304v1, 2021. <https://arxiv.org/abs/2102.02304v1>
- [248] Superminds Test Actively Evaluating Collective Intelligence of Agent Society via Probing Agents. arXiv:2604.22452v1, 2026. <https://arxiv.org/abs/2604.22452v1>
- [249] Do Agent Societies Develop Intellectual Elites The Hidden Power Laws of Collective Cognition in LLM Multi-Agent Systems. arXiv:2604.02674v1, 2026. <https://arxiv.org/abs/2604.02674v1>
- [250] Causal Foundations of Collective Agency. arXiv:2605.00248v1, 2026. <https://arxiv.org/abs/2605.00248v1>
- [251] Emergent Coordination in Multi-Agent Language Models. arXiv:2510.05174v4, 2025. <https://arxiv.org/abs/2510.05174v4>
- [252] The impact of behavioral diversity in multi-agent reinforcement learning. arXiv:2412.16244v2, 2024. <https://arxiv.org/abs/2412.16244v2>
- [253] Artificial collectives of specialists and generalists excel at different tasks. arXiv:2606.20877v1, 2026. <https://arxiv.org/abs/2606.20877v1>

- [254] Dynamics of Cognitive Heterogeneity Investigating Behavioral Biases in Multi-Stage Supply Chains with LLM-Based Simulation. arXiv:2604.17220v2, 2026. <https://arxiv.org/abs/2604.17220v2>
- [255] On the Dynamics of Multi-Agent LLM Communities Driven by Value Diversity. arXiv:2512.10665v1, 2025. <https://arxiv.org/abs/2512.10665v1>
- [256] The Principle of Maximum Heterogeneity Optimises Productivity in Distributed Production Systems Across Biology, Economics, and Computing. arXiv:2604.07602v1, 2026. <https://arxiv.org/abs/2604.07602v1>
- [257] NetworkGames Simulating Cooperation in Network Games with Personality-driven LLM Agents. arXiv:2511.21783v2, 2025. <https://arxiv.org/abs/2511.21783v2>
- [258] Evolutionary game selection creates cooperative environments. arXiv:2311.11128v1, 2023. <https://arxiv.org/abs/2311.11128v1>
- [259] Synchronization Dynamics of Heterogeneous, Collaborative Multi-Agent AI Systems. arXiv:2508.12314v1, 2025. <https://arxiv.org/abs/2508.12314v1>
- [260] Partner Modelling Emerges in Recurrent Agents (But Only When It Matters). arXiv:2505.17323v2, 2025. <https://arxiv.org/abs/2505.17323v2>
- [261] ConventionPlay Capability-Limited Training for Robust Ad-Hoc Collaboration. arXiv:2604.18123v1, 2026. <https://arxiv.org/abs/2604.18123v1>
- [262] The Collaboration Gap. arXiv:2511.02687v1, 2025. <https://arxiv.org/abs/2511.02687v1>
- [263] A Survey of Supernet Optimization and its Applications Spatial and Temporal Optimization for Neural Architecture Search. arXiv:2204.03916v2, 2022. <https://arxiv.org/abs/2204.03916v2>
- [264] Rethinking Performance Estimation in Neural Architecture Search. arXiv:2005.09917v1, 2020. <https://arxiv.org/abs/2005.09917v1>
- [265] Meta knowledge assisted Evolutionary Neural Architecture Search. arXiv:2504.21545v1, 2025. <https://arxiv.org/abs/2504.21545v1>
- [266] Enabling NAS with Automated Super-Network Generation. arXiv:2112.10878v1, 2021. <https://arxiv.org/abs/2112.10878v1>
- [267] A Hardware-Aware System for Accelerating Deep Neural Network Optimization. arXiv:2202.12954v1, 2022. <https://arxiv.org/abs/2202.12954v1>
- [268] A Hardware-Aware Framework for Accelerating Neural Architecture Search Across Modalities. arXiv:2205.10358v1, 2022. <https://arxiv.org/abs/2205.10358v1>
- [269] Multi-agent Architecture Search via Agentic Supernet. arXiv:2502.04180v2, 2025. <https://arxiv.org/abs/2502.04180v2>
- [270] AutoMaAS Self-Evolving Multi-Agent Architecture Search for Large Language Models. arXiv:2510.02669v1, 2025. <https://arxiv.org/abs/2510.02669v1>

- [271] TacoMAS Test-Time Co-Evolution of Topology and Capability in LLM-based Multi-Agent Systems. arXiv:2605.09539v1, 2026. <https://arxiv.org/abs/2605.09539v1>
- [272] Learning Latency-Aware Orchestration for Parallel Multi-Agent Systems. arXiv:2601.10560v1, 2026. <https://arxiv.org/abs/2601.10560v1>
- [273] HMAACE Heterogeneous Multi-Agent Collaborative Evolution for Combinatorial Optimization. arXiv:2605.07214v1, 2026. <https://arxiv.org/abs/2605.07214v1>
- [274] Learning to Evolve A Self-Improving Framework for Multi-Agent Systems via Textual Parameter Graph Optimization. arXiv:2604.20714v1, 2026. <https://arxiv.org/abs/2604.20714v1>
- [275] AgentSwift Efficient LLM Agent Design via Value-guided Hierarchical Search. arXiv:2506.06017v2, 2025. <https://arxiv.org/abs/2506.06017v2>
- [276] EvoRoute Experience-Driven Self-Routing LLM Agent Systems. arXiv:2601.02695v1, 2026. <https://arxiv.org/abs/2601.02695v1>
- [277] MasHost Builds It All Autonomous Multi-Agent System Directed by Reinforcement Learning. arXiv:2506.08507v2, 2025. <https://arxiv.org/abs/2506.08507v2>
- [278] Agent-UCT Upper Confidence Bounds Applied to Trees for Agentic Workflow Optimization with Cost-Awareness. arXiv:2607.24162v2, 2026. <https://arxiv.org/abs/2607.24162v2>
- [279] Dynamic Model Routing and Cascading for Efficient LLM Inference A Survey. arXiv:2603.04445v2, 2026. <https://arxiv.org/abs/2603.04445v2>
- [280] MasRouter Learning to Route LLMs for Multi-Agent Systems. arXiv:2502.11133v1, 2025. <https://arxiv.org/abs/2502.11133v1>
- [281] Towards Generalized Routing Model and Agent Orchestration for Adaptive and Efficient Inference. arXiv:2509.07571v2, 2025. <https://arxiv.org/abs/2509.07571v2>
- [282] Doing More with Less A Survey on Routing Strategies for Resource Optimisation in Large Language Model-Based Systems. arXiv:2502.00409v3, 2025. <https://arxiv.org/abs/2502.00409v3>
- [283] Difficulty-Aware Agentic Orchestration for Query-Specific Multi-Agent Workflows. arXiv:2509.11079v5, 2025. <https://arxiv.org/abs/2509.11079v5>
- [284] CASTER Breaking the Cost-Performance Barrier in Multi-Agent Orchestration via Context-Aware Strategy for Task Efficient Routing. arXiv:2601.19793v1, 2026. <https://arxiv.org/abs/2601.19793v1>
- [285] Robust Batch-Level Query Routing for Large Language Models under Cost and Capacity Constraints. arXiv:2603.26796v1, 2026. <https://arxiv.org/abs/2603.26796v1>
- [286] OmniRouter Budget and Performance Controllable Multi-LLM Routing. arXiv:2502.20576v6, 2025. <https://arxiv.org/abs/2502.20576v6>
- [287] TRACE-ROUTER Task-Consistent and Adaptive Online Routing for Agentic AI. arXiv:2607.22465v2, 2026. <https://arxiv.org/abs/2607.22465v2>

- [288] Budget-Aware Agentic Routing via Boundary-Guided Training. arXiv:2602.21227v1, 2026. <https://arxiv.org/abs/2602.21227v1>
- [289] Efficient and Interpretable Multi-Agent LLM Routing via Ant Colony Optimization. arXiv:2603.12933v1, 2026. <https://arxiv.org/abs/2603.12933v1>
- [290] MixLLM Dynamic Routing in Mixed Large Language Models. arXiv:2502.18482v1, 2025. <https://arxiv.org/abs/2502.18482v1>
- [291] Learning to Route LLMs from Implicit Cost-Performance Preferences via Meta-Learning. arXiv:2606.06178v1, 2026. <https://arxiv.org/abs/2606.06178v1>
- [292] INFRAMIND Infrastructure-Aware Multi-Agent Orchestration. arXiv:2606.11440v1, 2026. <https://arxiv.org/abs/2606.11440v1>
- [293] LMEdge QoS-Aware LLM Inference Orchestration on Edge Clusters. arXiv:2607.17175v1, 2026. <https://arxiv.org/abs/2607.17175v1>
- [294] Maestro Workload-Aware Cross-Cluster Scheduling for LLM-Based Multi-Agent Systems. arXiv:2606.12950v1, 2026. <https://arxiv.org/abs/2606.12950v1>
- [295] MARS Efficient, Adaptive Co-Scheduling for Heterogeneous Agentic Systems. arXiv:2604.26963v2, 2026. <https://arxiv.org/abs/2604.26963v2>
- [296] Agentic CPU-GPU Scheduling for Heterogeneous AI Workloads. arXiv:2607.22242v1, 2026. <https://arxiv.org/abs/2607.22242v1>
- [297] Towards Understanding, Analyzing, and Optimizing Agentic AI Execution A CPU-Centric Perspective. arXiv:2511.00739v3, 2025. <https://arxiv.org/abs/2511.00739v3>
- [298] From Models to Operators Rethinking Autoscaling Granularity for Large Generative Models. arXiv:2511.02248v1, 2025. <https://arxiv.org/abs/2511.02248v1>
- [299] RouteBalance Fused Model Routing and Load Balancing for Heterogeneous LLM Serving. arXiv:2606.17949v1, 2026. <https://arxiv.org/abs/2606.17949v1>
- [300] Sceptsy Serving Agentic Workflows Using Aggregate LLM Pipelines. arXiv:2604.15186v1, 2026. <https://arxiv.org/abs/2604.15186v1>
- [301] HexAGenT Efficient Agentic LLM Serving via Workflow- and Heterogeneity-Aware Scheduling. arXiv:2605.16637v1, 2026. <https://arxiv.org/abs/2605.16637v1>
- [302] SAGA Workflow-Atomic Scheduling for AI Agent Inference on GPU Clusters. arXiv:2605.00528v2, 2026. <https://arxiv.org/abs/2605.00528v2>
- [303] A Survey on Large Language Model Acceleration based on KV Cache Management. arXiv:2412.19442v3, 2024. <https://arxiv.org/abs/2412.19442v3>
- [304] KV Cache Optimization Strategies for Scalable and Efficient LLM Inference. arXiv:2603.20397v1, 2026. <https://arxiv.org/abs/2603.20397v1>
- [305] Comparative Characterization of KV Cache Management Strategies for LLM Inference. arXiv:2604.05012v1, 2026. <https://arxiv.org/abs/2604.05012v1>

- [306] Sequence can Secretly Tell You What to Discard. arXiv:2404.15949v1, 2024. <https://arxiv.org/abs/2404.15949v1>
- [307] Coverage-Driven KV Cache Eviction for Efficient and Improved Inference of LLM. arXiv:2606.29563v1, 2026. <https://arxiv.org/abs/2606.29563v1>
- [308] KV Admission Learning What to Write for Efficient Long-Context Inference. arXiv:2512.17452v3, 2025. <https://arxiv.org/abs/2512.17452v3>
- [309] Cache What Lasts Token Retention for Memory-Bounded KV Cache in LLMs. arXiv:2512.03324v2, 2025. <https://arxiv.org/abs/2512.03324v2>
- [310] MiniCache KV Cache Compression in Depth Dimension for Large Language Models. arXiv:2405.14366v2, 2024. <https://arxiv.org/abs/2405.14366v2>
- [311] Task-KV Task-aware KV Cache Optimization via Semantic Differentiation of Attention Heads. arXiv:2501.15113v1, 2025. <https://arxiv.org/abs/2501.15113v1>
- [312] DynamicKV Task-Aware Adaptive KV Cache Compression for Long Context LLMs. arXiv:2412.14838v4, 2024. <https://arxiv.org/abs/2412.14838v4>
- [313] ZigZagkv Dynamic KV Cache Compression for Long-context Modeling based on Layer Uncertainty. arXiv:2412.09036v1, 2024. <https://arxiv.org/abs/2412.09036v1>
- [314] Predictive Multi-Tier Memory Management for KV Cache in Large-Scale GPU Inference. arXiv:2604.26968v1, 2026. <https://arxiv.org/abs/2604.26968v1>
- [315] KVDrive A Holistic Multi-Tier KV Cache Management System for Long-Context LLM Inference. arXiv:2605.18071v1, 2026. <https://arxiv.org/abs/2605.18071v1>
- [316] KVPR Efficient LLM Inference with I O-Aware KV Cache Partial Recomputation. arXiv:2411.17089v2, 2024. <https://arxiv.org/abs/2411.17089v2>
- [317] ScoutAttention Efficient KV Cache Offloading via Layer-Ahead CPU Pre-computation for LLM Inference. arXiv:2603.27138v1, 2026. <https://arxiv.org/abs/2603.27138v1>
- [318] Continuum Efficient and Robust Multi-Turn LLM Agent Scheduling with KV Cache Time-to-Live. arXiv:2511.02230v6, 2025. <https://arxiv.org/abs/2511.02230v6>
- [319] TokenCake A KV-Cache-centric Serving Framework for LLM-based Multi-Agent Applications. arXiv:2510.18586v3, 2025. <https://arxiv.org/abs/2510.18586v3>
- [320] KVFlow Efficient Prefix Caching for Accelerating LLM-Based Multi-Agent Workflows. arXiv:2507.07400v1, 2025. <https://arxiv.org/abs/2507.07400v1>
- [321] [AAFLOW+] Stateful Operator Abstraction with Zero-Copy Distributed KV Cache Orchestration for Multi-Agent Workflows. arXiv:2607.10987v1, 2026. <https://arxiv.org/abs/2607.10987v1>
- [322] Robust KV Cache Management for LLM Serving under Output Token Length Uncertainty. arXiv:2607.16892v1, 2026. <https://arxiv.org/abs/2607.16892v1>
- [323] CacheFlow Efficient LLM Serving with 3D-Parallel KV Cache Restoration. arXiv:2604.25080v1, 2026. <https://arxiv.org/abs/2604.25080v1>

- [324] Compute Or Load KV Cache Why Not Both. arXiv:2410.03065v2, 2024. <https://arxiv.org/abs/2410.03065v2>
- [325] Understanding Bottlenecks for Efficiently Serving LLM Inference With KV Offloading. arXiv:2601.19910v1, 2025. <https://arxiv.org/abs/2601.19910v1>
- [326] DualPath Breaking the Storage Bandwidth Bottleneck in Agentic LLM Inference. arXiv:2602.21548v2, 2026. <https://arxiv.org/abs/2602.21548v2>
- [327] Why Your Deep Research Agent Fails On Hallucination Evaluation in Full Research Trajectory. arXiv:2601.22984v2, 2026. <https://arxiv.org/abs/2601.22984v2>
- [328] Diagnose, Localize, Align A Full-Stack Framework for Reliable LLM Multi-Agent Systems under Instruction Conflicts. arXiv:2509.23188v3, 2025. <https://arxiv.org/abs/2509.23188v3>
- [329] Multi-Agent LLM Orchestration Achieves Deterministic, High-Quality Decision Support for Incident Response. arXiv:2511.15755v2, 2025. <https://arxiv.org/abs/2511.15755v2>
- [330] Beyond Hallucinations A Composite Score for Measuring Reliability in Open-Source Large Language Models. arXiv:2512.24058v1, 2025. <https://arxiv.org/abs/2512.24058v1>
- [331] AgentEval DAG-Structured Step-Level Evaluation for Agentic Workflows with Error Propagation Tracking. arXiv:2604.23581v1, 2026. <https://arxiv.org/abs/2604.23581v1>
- [332] Deterministic Integrity Gates for LLM-Assisted Clinical Manuscript Preparation An Auditable Biomedical Informatics Architecture. arXiv:2606.09500v4, 2026. <https://arxiv.org/abs/2606.09500v4>
- [333] Pramana A Protocol-Layer Treatment of Claim Verification in Autonomous Agent Networks. arXiv:2605.20312v1, 2026. <https://arxiv.org/abs/2605.20312v1>
- [334] AgentRx Diagnosing AI Agent Failures from Execution Trajectories. arXiv:2602.02475v1, 2026. <https://arxiv.org/abs/2602.02475v1>
- [335] Auditing medical multi-agent AI reveals risks of false consensus. arXiv:2510.10185v3, 2025. <https://arxiv.org/abs/2510.10185v3>
- [336] AI Assurance A Comprehensive Testing Strategy for Enterprise AI Systems. arXiv:2605.23459v1, 2026. <https://arxiv.org/abs/2605.23459v1>
- [337] AgentFixer From Failure Detection to Fix Recommendations in LLM Agentic Systems. arXiv:2603.29848v1, 2026. <https://arxiv.org/abs/2603.29848v1>
- [338] The Stability Trap Evaluating the Reliability of LLM-Based Instruction Adherence Auditing. arXiv:2601.11783v1, 2026. <https://arxiv.org/abs/2601.11783v1>
- [339] ProofAgent Harness Open Infrastructure for Adversarial Evaluation of AI Agents. arXiv:2605.24134v1, 2026. <https://arxiv.org/abs/2605.24134v1>
- [340] Recursive Knowledge Synthesis for Multi-LLM Systems Stability Analysis and Tri-Agent Audit Framework. arXiv:2601.08839v1, 2025. <https://arxiv.org/abs/2601.08839v1>

- [341] RADAR Rubric-Aware Dependency and Redundancy Analysis for LLM-as-Judge Evaluation. arXiv:2608.01810v1, 2026. <https://arxiv.org/abs/2608.01810v1>
- [342] AEVAL From Anecdotal to Deterministic Testing for Agentic Skill Workflows. arXiv:2607.16345v2, 2026. <https://arxiv.org/abs/2607.16345v2>
- [343] Measurement Without Validity The Compounding Reliability Problem in Agentic AI Evaluation. arXiv:2608.00794v1, 2026. <https://arxiv.org/abs/2608.00794v1>
- [344] Interactive Evaluation Requires a Design Science. arXiv:2605.17829v1, 2026. <https://arxiv.org/abs/2605.17829v1>
- [345] Seeing the Whole Elephant A Benchmark for Failure Attribution in LLM-based Multi-Agent Systems. arXiv:2604.22708v1, 2026. <https://arxiv.org/abs/2604.22708v1>
- [346] Rethinking Failure Attribution in Multi-Agent Systems A Multi-Perspective Benchmark and Evaluation. arXiv:2603.25001v1, 2026. <https://arxiv.org/abs/2603.25001v1>
- [347] AgentCollabBench Diagnosing When Good Agents Make Bad Collaborators. arXiv:2605.08647v1, 2026. <https://arxiv.org/abs/2605.08647v1>
- [348] MAS-FIRE Fault Injection and Reliability Evaluation for LLM-Based Multi-Agent Systems. arXiv:2602.19843v1, 2026. <https://arxiv.org/abs/2602.19843v1>
- [349] SWE-Marathon Can Agents Autonomously Complete Ultra-Long-Horizon Software Work. arXiv:2606.07682v1, 2026. <https://arxiv.org/abs/2606.07682v1>
- [350] UltraHorizon Benchmarking Agent Capabilities in Ultra Long-Horizon Scenarios. arXiv:2509.21766v1, 2025. <https://arxiv.org/abs/2509.21766v1>
- [351] Benchmarking the Residual What Long-Horizon Evaluations Add Beyond Matched Short-Task Performance. arXiv:2607.27283v1, 2026. <https://arxiv.org/abs/2607.27283v1>
- [352] Beyond pass@1 A Reliability Science Framework for Long-Horizon LLM Agents. arXiv:2603.29231v1, 2026. <https://arxiv.org/abs/2603.29231v1>
- [353] Towards a Science of AI Agent Reliability. arXiv:2602.16666v3, 2026. <https://arxiv.org/abs/2602.16666v3>
- [354] MAESTRO Multi-Agent Evaluation Suite for Testing, Reliability, and Observability. arXiv:2601.00481v1, 2026. <https://arxiv.org/abs/2601.00481v1>
- [355] OrchBench Evaluating Multi-Agent Orchestration Plans in Isolation via Deterministic Simulation. arXiv:2607.25656v1, 2026. <https://arxiv.org/abs/2607.25656v1>
- [356] Beyond Accuracy A Multi-Dimensional Framework for Evaluating Enterprise Agentic AI Systems. arXiv:2511.14136v1, 2025. <https://arxiv.org/abs/2511.14136v1>
- [357] The Six Sigma Agent Achieving Enterprise-Grade Reliability in LLM Systems Through Consensus-Driven Decomposed Execution. arXiv:2601.22290v1, 2026. <https://arxiv.org/abs/2601.22290v1>
- [358] Phase Transition for Budgeted Multi-Agent Synergy. arXiv:2601.17311v2, 2026. <https://arxiv.org/abs/2601.17311v2>

- [359] Diversity Collapse in Multi-Agent LLM Systems Structural Coupling and Collective Failure in Open-Ended Idea Generation. arXiv:2604.18005v2, 2026. <https://arxiv.org/abs/2604.18005v2>
- [360] Collective Intelligence with Foundation Models. arXiv:2607.07729v1, 2026. <https://arxiv.org/abs/2607.07729v1>
- [361] Which Agent Causes Task Failures and When On Automated Failure Attribution of LLM Multi-Agent Systems. arXiv:2505.00212v3, 2025. <https://arxiv.org/abs/2505.00212v3>
- [362] SAFARI Scaling Long Horizon Agentic Fault Attribution via Active Investigation. arXiv:2606.24626v1, 2026. <https://arxiv.org/abs/2606.24626v1>
- [363] StepFinder A Temporal Semantic Framework for Failure Attribution in Multi-Agent Systems. arXiv:2606.03467v1, 2026. <https://arxiv.org/abs/2606.03467v1>
- [364] MASPrism Lightweight Failure Attribution for Multi-Agent Systems Using Prefill-Stage Signals. arXiv:2605.07509v2, 2026. <https://arxiv.org/abs/2605.07509v2>
- [365] AgentForesight Online Auditing for Early Failure Prediction in Multi-Agent Systems. arXiv:2605.08715v2, 2026. <https://arxiv.org/abs/2605.08715v2>
- [366] GraphTracer Graph-Guided Failure Tracing in LLM Agents for Robust Multi-Turn Deep Search. arXiv:2510.10581v2, 2025. <https://arxiv.org/abs/2510.10581v2>
- [367] From Flat Logs to Causal Graphs Hierarchical Failure Attribution for LLM-based Multi-Agent Systems. arXiv:2602.23701v1, 2026. <https://arxiv.org/abs/2602.23701v1>
- [368] FALAT Tracing Failures in LLM Agent Trajectories via Dependency-Guided Search. arXiv:2606.00765v1, 2026. <https://arxiv.org/abs/2606.00765v1>
- [369] Where Did It All Go Wrong A Hierarchical Look into Multi-Agent Error Attribution. arXiv:2510.04886v2, 2025. <https://arxiv.org/abs/2510.04886v2>
- [370] CockpitHAT Dependency-Graph-Driven Hierarchical Attribution for Embodied Multi-Agent Cockpits. arXiv:2608.01805v1, 2026. <https://arxiv.org/abs/2608.01805v1>
- [371] CausalFlow Causal Attribution and Counterfactual Repair for LLM Agent Failures. arXiv:2605.25338v1, 2026. <https://arxiv.org/abs/2605.25338v1>
- [372] Automatic Failure Attribution and Critical Step Prediction Method for Multi-Agent Systems Based on Causal Inference. arXiv:2509.08682v1, 2025. <https://arxiv.org/abs/2509.08682v1>
- [373] HPFA Hypergraph-Based Paired Failure Attribution for LLM Reasoning. arXiv:2608.02026v1, 2026. <https://arxiv.org/abs/2608.02026v1>
- [374] Tracing Agentic Failure from the Flow of Success. arXiv:2607.12747v1, 2026. <https://arxiv.org/abs/2607.12747v1>
- [375] VerifyMAS Hypothesis Verification for Failure Attribution in LLM Multi-Agent Systems. arXiv:2605.17467v1, 2026. <https://arxiv.org/abs/2605.17467v1>

- [376] REFLECT Intervention-Supported Error Attribution for Silent Failures in LLM Agent Traces. arXiv:2606.09071v1, 2026. <https://arxiv.org/abs/2606.09071v1>
- [377] DoVer Intervention-Driven Auto Debugging for LLM Multi-Agent Systems. arXiv:2512.06749v3, 2025. <https://arxiv.org/abs/2512.06749v3>
- [378] Towards Self-Improving Error Diagnosis in Multi-Agent Systems. arXiv:2604.17658v1, 2026. <https://arxiv.org/abs/2604.17658v1>
- [379] CORRECT COndensed eRror RECOgnition via knowledge Transfer in multi-agent systems. arXiv:2509.24088v2, 2025. <https://arxiv.org/abs/2509.24088v2>
- [380] Aegis Automated Error Generation and Attribution for Multi-Agent Systems. arXiv:2509.14295v6, 2025. <https://arxiv.org/abs/2509.14295v6>
- [381] AgentAsk Multi-Agent Systems Need to Ask. arXiv:2510.07593v2, 2025. <https://arxiv.org/abs/2510.07593v2>
- [382] Model or Harness An Interaction-Centric Taxonomy for Localizing Agent Failures. arXiv:2607.28802v1, 2026. <https://arxiv.org/abs/2607.28802v1>
- [383] From Spark to Fire Modeling and Mitigating Error Cascades in LLM-Based Multi-Agent Collaboration. arXiv:2603.04474v2, 2026. <https://arxiv.org/abs/2603.04474v2>
- [384] The Long-Horizon Task Mirage Diagnosing Where and Why Agentic Systems Break. arXiv:2604.11978v1, 2026. <https://arxiv.org/abs/2604.11978v1>
- [385] Benchmarking Chinese Medical LLMs A Medbench-based Analysis of Performance Gaps and Hierarchical Optimization Strategies. arXiv:2503.07306v1, 2025. <https://arxiv.org/abs/2503.07306v1>
- [386] Before Agents Speak Pre-hoc Failure Risk Inference in Multi-Agent Systems. arXiv:2607.26836v1, 2026. <https://arxiv.org/abs/2607.26836v1>
- [387] Who&When Pro Can LLMs Really Attribute Failures in AI Agents. arXiv:2607.09996v1, 2026. <https://arxiv.org/abs/2607.09996v1>
- [388] AgenTracer Who Is Inducing Failure in the LLM Agentic Systems. arXiv:2509.03312v2, 2025. <https://arxiv.org/abs/2509.03312v2>
- [389] Autonomous Repair for Multi-Agent Systems via Monte-Carlo Tree Search. arXiv:2607.29055v1, 2026. <https://arxiv.org/abs/2607.29055v1>
- [390] AgentDebugX An Open-Source Toolkit for Failure Observability, Attribution, and Recovery in LLM Agents. arXiv:2607.18754v1, 2026. <https://arxiv.org/abs/2607.18754v1>
- [391] Consistency as a Testable Property Statistical Methods to Evaluate AI Agent Reliability. arXiv:2605.10516v1, 2026. <https://arxiv.org/abs/2605.10516v1>
- [392] Exploring the Adversarial Frontier Quantifying Robustness via Adversarial Hypervolume. arXiv:2403.05100v1, 2024. <https://arxiv.org/abs/2403.05100v1>
- [393] ASSERT Automated Safety Scenario Red Teaming for Evaluating the Robustness of Large Language Models. arXiv:2310.09624v2, 2023. <https://arxiv.org/abs/2310.09624v2>

- [394] SENTRY Statistical Reliability Analysis of Vision Transformers Under Soft Errors. arXiv:2606.07620v1, 2026. <https://arxiv.org/abs/2606.07620v1>
- [395] A Trace-Based Assurance Framework for Agentic AI Orchestration Contracts, Testing, and Governance. arXiv:2603.18096v1, 2026. <https://arxiv.org/abs/2603.18096v1>
- [396] Detecting and Preventing Latent Risk Accumulation in High-Performance Software Systems. arXiv:2510.03712v2, 2025. <https://arxiv.org/abs/2510.03712v2>
- [397] Statistical Perspectives on Reliability of Artificial Intelligence Systems. arXiv:2111.05391v1, 2021. <https://arxiv.org/abs/2111.05391v1>
- [398] A Byzantine Fault Tolerance Approach towards AI Safety. arXiv:2504.14668v1, 2025. <https://arxiv.org/abs/2504.14668v1>
- [399] The Honest Quorum Problem Epistemic Byzantine Fault Tolerance for Agentic Infrastructure. arXiv:2607.16109v1, 2026. <https://arxiv.org/abs/2607.16109v1>
- [400] Probabilistic Byzantine Fault Tolerance (Extended Version). arXiv:2405.04606v3, 2024. <https://arxiv.org/abs/2405.04606v3>
- [401] When Certificates Fail A Unified Safety Framework for Embedded Neural Interface Models. arXiv:2607.06630v1, 2026. <https://arxiv.org/abs/2607.06630v1>
- [402] The Evaluation Differential When Frontier AI Models Recognise They Are Being Tested. arXiv:2605.11496v1, 2026. <https://arxiv.org/abs/2605.11496v1>
- [403] Statistical Runtime Verification for LLMs via Robustness Estimation. arXiv:2504.17723v2, 2025. <https://arxiv.org/abs/2504.17723v2>
- [404] Adversarial Pragmatics for AI Safety Evaluation A Diagnostic Framework and Seed Benchmark for Language-Mediated Control. arXiv:2607.01153v3, 2026. <https://arxiv.org/abs/2607.01153v3>
- [405] A Systematic Survey of Security Threats and Defenses in LLM-Based AI Agents A Layered Attack Surface Framework. arXiv:2604.23338v2, 2026. <https://arxiv.org/abs/2604.23338v2>
- [406] Security Considerations for Multi-agent Systems. arXiv:2603.09002v2, 2026. <https://arxiv.org/abs/2603.09002v2>
- [407] Comprehensive Vulnerability Analysis is Necessary for Trustworthy LLM-MAS. arXiv:2506.01245v3, 2025. <https://arxiv.org/abs/2506.01245v3>
- [408] From Monoliths to Swarms A Study of Attack Surface Evolution in the Transition to Multi-Agent Web Systems. arXiv:2608.00202v1, 2026. <https://arxiv.org/abs/2608.00202v1>
- [409] Exposing Weak Links in Multi-Agent Systems under Adversarial Prompting. arXiv:2511.10949v1, 2025. <https://arxiv.org/abs/2511.10949v1>
- [410] CIA Inferring the Communication Topology from LLM-based Multi-Agent Systems. arXiv:2604.12461v1, 2026. <https://arxiv.org/abs/2604.12461v1>

- [411] From Privacy to Workflow Integrity Communication-Graph Metadata in Autonomous Agent Interoperability. arXiv:2606.07150v3, 2026. <https://arxiv.org/abs/2606.07150v3>
- [412] Architecture Matters for Multi-Agent Security. arXiv:2604.23459v1, 2026. <https://arxiv.org/abs/2604.23459v1>
- [413] Trojan Hippo Weaponizing Agent Memory for Data Exfiltration. arXiv:2605.01970v3, 2026. <https://arxiv.org/abs/2605.01970v3>
- [414] Stateful Agent Backdoor. arXiv:2605.06158v1, 2026. <https://arxiv.org/abs/2605.06158v1>
- [415] BackdoorAgent A Unified Framework for Backdoor Attacks on LLM-based Agents. arXiv:2601.04566v2, 2026. <https://arxiv.org/abs/2601.04566v2>
- [416] Collaborative Shadows Distributed Backdoor Attacks in LLM-Based Multi-Agent Systems. arXiv:2510.11246v1, 2025. <https://arxiv.org/abs/2510.11246v1>
- [417] Benchmarking the Robustness of Agentic Systems to Adversarially-Induced Harms. arXiv:2508.16481v2, 2025. <https://arxiv.org/abs/2508.16481v2>
- [418] Toward a Safe Internet of Agents. arXiv:2512.00520v2, 2025. <https://arxiv.org/abs/2512.00520v2>
- [419] Hierarchical Attacks for Multi-Modal Multi-Agent Reasoning. arXiv:2605.13213v1, 2026. <https://arxiv.org/abs/2605.13213v1>
- [420] Securing Agentic AI A Comprehensive Threat Model and Mitigation Framework for Generative AI Agents. arXiv:2504.19956v2, 2025. <https://arxiv.org/abs/2504.19956v2>
- [421] Servant, Stalker, Predator How An Honest, Helpful, And Harmless (3H) Agent Unlocks Adversarial Skills. arXiv:2508.19500v1, 2025. <https://arxiv.org/abs/2508.19500v1>
- [422] The Agentic Economy. arXiv:2505.15799v2, 2025. <https://arxiv.org/abs/2505.15799v2>
- [423] Sentinel Agents for Secure and Trustworthy Agentic AI in Multi-Agent Systems. arXiv:2509.14956v1, 2025. <https://arxiv.org/abs/2509.14956v1>
- [424] MAS-Shield A Defense Framework for Secure and Efficient LLM MAS. arXiv:2511.22924v2, 2025. <https://arxiv.org/abs/2511.22924v2>
- [425] SentinelNet Safeguarding Multi-Agent Collaboration Through Credit-Based Dynamic Threat Detection. arXiv:2510.16219v3, 2025. <https://arxiv.org/abs/2510.16219v3>
- [426] SAIGuard Communication-State Simulation for Proactive Defense of LLM Multi-Agent Systems. arXiv:2606.12474v1, 2026. <https://arxiv.org/abs/2606.12474v1>
- [427] MAGE Safeguarding LLM Agents against Long-Horizon Threats via Shadow Memory. arXiv:2605.03228v1, 2026. <https://arxiv.org/abs/2605.03228v1>
- [428] Cognitive Firewall A Proactive, Zero-Trust, Multi-Gate Framework for LLM Safety. arXiv:2607.01277v1, 2026. <https://arxiv.org/abs/2607.01277v1>
- [429] Stateful Cooperative Agents Safeguarding LLMs Against Evolving Multi-Turn Attacks. arXiv:2608.00134v1, 2026. <https://arxiv.org/abs/2608.00134v1>

- [430] MAPLE-Guard Memory-Aware Link Enforcement Against Memory-Link Poisoning in Multi-Agent Systems. arXiv:2608.00426v1, 2026. <https://arxiv.org/abs/2608.00426v1>
- [431] A-MemGuard A Proactive Defense Framework for LLM-Based Agent Memory. arXiv:2510.02373v1, 2025. <https://arxiv.org/abs/2510.02373v1>
- [432] AgentSafe Safeguarding Large Language Model-based Multi-agent Systems via Hierarchical Data Management. arXiv:2503.04392v2, 2025. <https://arxiv.org/abs/2503.04392v2>
- [433] Defense effectiveness across architectural layers a mechanistic evaluation of persistent memory attacks on stateful LLM agents. arXiv:2605.08442v4, 2026. <https://arxiv.org/abs/2605.08442v4>
- [434] MemPoison Uncovering Persistent Memory Threats and Structural Blind Spots in LLM Agents. arXiv:2607.14651v1, 2026. <https://arxiv.org/abs/2607.14651v1>
- [435] PBFT-Backed Semantic Voting for Multi-Agent Memory Pruning. arXiv:2506.17338v2, 2025. <https://arxiv.org/abs/2506.17338v2>
- [436] MESA Prioritizing Vulnerable Communication Channels for Securing Multi-Agent Systems. arXiv:2606.30602v1, 2026. <https://arxiv.org/abs/2606.30602v1>
- [437] INFA-Guard Mitigating Malicious Propagation via Infection-Aware Safeguarding in LLM-Based Multi-Agent Systems. arXiv:2601.14667v1, 2026. <https://arxiv.org/abs/2601.14667v1>
- [438] MASStrike Shapley-Guided Collusive Red-Teaming on Multi-Agent Systems. arXiv:2606.12918v2, 2026. <https://arxiv.org/abs/2606.12918v2>
- [439] OpenEvoShield Dual Non-Stationary Continual Defense for Open-World Multi-Agent System Attacks. arXiv:2607.19351v1, 2026. <https://arxiv.org/abs/2607.19351v1>
- [440] BlindGuard Safeguarding LLM-based Multi-Agent Systems under Unknown Attacks. arXiv:2508.08127v2, 2025. <https://arxiv.org/abs/2508.08127v2>
- [441] PropGuard Safeguarding LLM-MAS via Propagation-Aware Exploration and Remediation. arXiv:2605.16346v1, 2026. <https://arxiv.org/abs/2605.16346v1>
- [442] When Agents Go Rogue Activation-Based Detection of Malicious Behaviors in Multi-Agent Systems. arXiv:2607.06807v1, 2026. <https://arxiv.org/abs/2607.06807v1>
- [443] Adaptive Defense Orchestration for RAG A Sentinel-Strategist Architecture against Multi-Vector Attacks. arXiv:2604.20932v1, 2026. <https://arxiv.org/abs/2604.20932v1>
- [444] MASTER Multi-Agent Security Through Exploration of Roles and Topological Structures – A Comprehensive Framework. arXiv:2505.18572v1, 2025. <https://arxiv.org/abs/2505.18572v1>
- [445] A Holistic Assessment of the Reliability of Machine Learning Systems. arXiv:2307.10586v2, 2023. <https://arxiv.org/abs/2307.10586v2>
- [446] A practical approach to evaluating the adversarial distance for machine learning classifiers. arXiv:2409.03598v1, 2024. <https://arxiv.org/abs/2409.03598v1>

- [447] FRAME Comprehensive Risk Assessment Framework for Adversarial Machine Learning Threats. arXiv:2508.17405v1, 2025. <https://arxiv.org/abs/2508.17405v1>
- [448] The Capability Paradox How Smarter Auditors Make Multi-Agent Systems Less Secure. arXiv:2605.17480v3, 2026. <https://arxiv.org/abs/2605.17480v3>
- [449] LLM Robustness Leaderboard v1 –Technical report. arXiv:2508.06296v2, 2025. <https://arxiv.org/abs/2508.06296v2>
- [450] Fast Proxies for LLM Robustness Evaluation. arXiv:2502.10487v1, 2025. <https://arxiv.org/abs/2502.10487v1>
- [451] Automatic LLM Red Teaming. arXiv:2508.04451v1, 2025. <https://arxiv.org/abs/2508.04451v1>
- [452] The Automation Advantage in AI Red Teaming. arXiv:2504.19855v2, 2025. <https://arxiv.org/abs/2504.19855v2>
- [453] MaMa A Game-Theoretic Approach for Designing Safe Agentic Systems. arXiv:2602.04431v2, 2026. <https://arxiv.org/abs/2602.04431v2>
- [454] Bits Leaked per Query Information-Theoretic Bounds on Adversarial Attacks against LLMs. arXiv:2510.17000v1, 2025. <https://arxiv.org/abs/2510.17000v1>
- [455] Extending the Formalism and Theoretical Foundations of Cryptography to AI. arXiv:2603.02590v1, 2026. <https://arxiv.org/abs/2603.02590v1>
- [456] Robustness Certificates for Neural Networks Against Data Poisoning and Evasion Attacks. arXiv:2512.20865v3, 2025. <https://arxiv.org/abs/2512.20865v3>
- [457] Stable Agentic Control Tool-Mediated LLM Architecture for Autonomous Cyber Defense. arXiv:2605.03034v1, 2026. <https://arxiv.org/abs/2605.03034v1>
- [458] Towards Assurance of LLM Adversarial Robustness using Ontology-Driven Argumentation. arXiv:2410.07962v1, 2024. <https://arxiv.org/abs/2410.07962v1>
- [459] When Embedding-Based Defenses Fail Rethinking Safety in LLM-Based Multi-Agent Systems. arXiv:2605.01133v3, 2026. <https://arxiv.org/abs/2605.01133v3>
- [460] Securing Agentic AI From Per-Action Checks to Trajectory Assurance. arXiv:2608.01558v1, 2026. <https://arxiv.org/abs/2608.01558v1>
- [461] LOKA Protocol A Decentralized Framework for Trustworthy and Ethical AI Agent Ecosystems. arXiv:2504.10915v2, 2025. <https://arxiv.org/abs/2504.10915v2>
- [462] Audit Trails for Accountability in Large Language Models. arXiv:2601.20727v1, 2026. <https://arxiv.org/abs/2601.20727v1>
- [463] Auditable Agents. arXiv:2604.05485v1, 2026. <https://arxiv.org/abs/2604.05485v1>
- [464] AI Identification An Integrated Framework for Sustainable Governance in Digital Enterprises. arXiv:2604.10473v1, 2026. <https://arxiv.org/abs/2604.10473v1>
- [465] From Cloud-Native to Trust-Native A Protocol for Verifiable Multi-Agent Systems. arXiv:2507.22077v1, 2025. <https://arxiv.org/abs/2507.22077v1>

- [466] AGENTS SAFE A Unified Framework for Ethical Assurance and Governance in Agentic AI. arXiv:2512.03180v1, 2025. <https://arxiv.org/abs/2512.03180v1>
- [467] Agentic AI Governance and Lifecycle Management in Healthcare. arXiv:2601.15630v2, 2026. <https://arxiv.org/abs/2601.15630v2>
- [468] The Social Responsibility Stack A Control-Theoretic Architecture for Governing Socio-Technical AI. arXiv:2512.16873v1, 2025. <https://arxiv.org/abs/2512.16873v1>
- [469] Position A Three-Layer Probabilistic Assume-Guarantee Architecture Is Structurally Required for Safe LLM Agent Deployment. arXiv:2605.18672v1, 2026. <https://arxiv.org/abs/2605.18672v1>
- [470] Governance by Construction for Generalist Agents. arXiv:2605.20874v1, 2026. <https://arxiv.org/abs/2605.20874v1>
- [471] The Alignment Flywheel A Governance-Centric Hybrid MAS for Architecture-Agnostic Safety. arXiv:2603.02259v2, 2026. <https://arxiv.org/abs/2603.02259v2>
- [472] TRUST A Framework for Decentralized AI Service v.0.1. arXiv:2604.27132v1, 2026. <https://arxiv.org/abs/2604.27132v1>
- [473] Bridging the Gap in XAI-Why Reliable Metrics Matter for Explainability and Compliance. arXiv:2502.04695v2, 2025. <https://arxiv.org/abs/2502.04695v2>
- [474] TRACE A Metrologically-Grounded Engineering Framework for Trustworthy Agentic AI Systems in Operationally Critical Domains. arXiv:2605.03838v1, 2026. <https://arxiv.org/abs/2605.03838v1>
- [475] A Methodology for Auditable Trustworthiness Levels in AI Lifecycle Governance. arXiv:2607.16130v1, 2026. <https://arxiv.org/abs/2607.16130v1>
- [476] LLM-FACETS A Privacy-Preserving Framework for Evaluating LLM Transparency and Accountability. arXiv:2605.31167v1, 2026. <https://arxiv.org/abs/2605.31167v1>
- [477] Auditing Emergent LLM-Agent Collaboration through Cooperation-Obligation Coupling. arXiv:2607.27429v1, 2026. <https://arxiv.org/abs/2607.27429v1>
- [478] Think Before You Act – A Neurocognitive Governance Model for Autonomous AI Agents. arXiv:2604.25684v1, 2026. <https://arxiv.org/abs/2604.25684v1>
- [479] Autonomy and Agency in Agentic AI Architectural Tactics for Regulated Contexts. arXiv:2605.12105v1, 2026. <https://arxiv.org/abs/2605.12105v1>
- [480] I Can’t Believe It’s Corrupt Evaluating Corruption in Multi-Agent Governance Systems. arXiv:2603.18894v1, 2026. <https://arxiv.org/abs/2603.18894v1>
- [481] Safe Multi-Agent Behavior Must Be Maintained, Not Merely Asserted Constraint Drift in LLM-Based Multi-Agent Systems. arXiv:2605.10481v1, 2026. <https://arxiv.org/abs/2605.10481v1>
- [482] ToolAlignBench Investigating Alignment Conflicts in Tool-Calling Enabled LLMs. arXiv:2607.14285v1, 2026. <https://arxiv.org/abs/2607.14285v1>

- [483] Beyond Single-Agent Safety A Taxonomy of Risks in LLM-to-LLM Interactions. arXiv:2512.02682v1, 2025. <https://arxiv.org/abs/2512.02682v1>
- [484] A Framework for Responsible AI Systems Building Societal Trust through Domain Definition, Trustworthy AI Design, Auditability, Accountability, and Governance. arXiv:2503.04739v2, 2025. <https://arxiv.org/abs/2503.04739v2>
- [485] Responsible Innovation A Strategic Framework for Financial LLM Integration. arXiv:2504.02165v1, 2025. <https://arxiv.org/abs/2504.02165v1>
- [486] AgentHub A Registry for Discoverable, Verifiable, and Reproducible AI Agents. arXiv:2510.03495v2, 2025. <https://arxiv.org/abs/2510.03495v2>